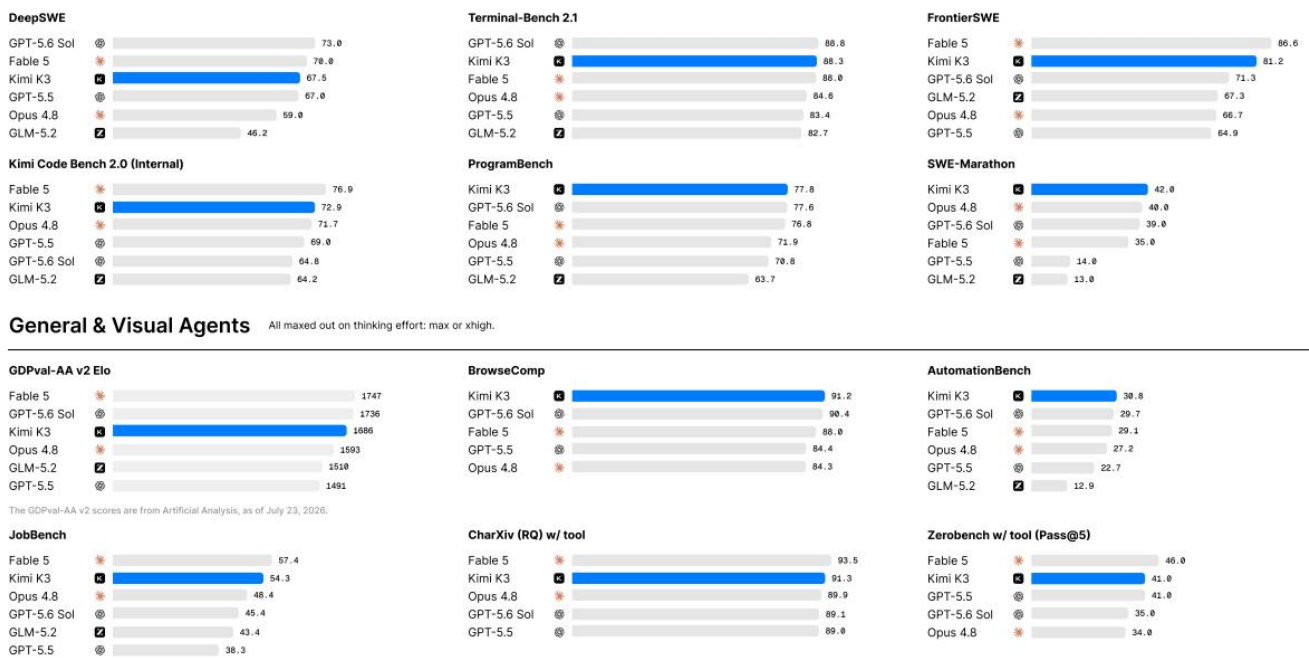




KIMI K3: 开放前沿智能

KIMI K3 技术报告

Kimi 团队

摘要

我们提出 Kimi K3，一个拥有 2.8 万亿参数的 Mixture-of-Experts 模型，激活参数为 1040 亿，具备原生视觉能力和 100 万 token 的上下文窗口。Kimi K3 基于 Kimi Delta Attention [63] 和 Attention Residuals [57]，这两项技术分别改善了沿序列长度和模型深度的信息流动。结合 Stable LatentMoE（每个 token 在 896 个路由专家中有效激活 16 个）以及经过优化的训练和数据方案，这些进展使整体扩展效率相比 Kimi K2 [58] 提升了约 2.5 倍。后训练方面，重点介绍了在通用、智能体和编程领域的强化学习，以及多个推理努力级别，从而实现了组合泛化和稳健的长期执行能力。在 2.8T 规模下，Kimi K3 得到了多个领域基础设施进步的支持：面向 KDA 的算法-系统协同设计、具有高效内存管理的完美均衡专家并行训练、支持持久化 rollout 和沙箱状态的百万 token 智能体强化学习，以及部署创新。

广泛的评估表明，Kimi K3 在长期编程、智能体、知识、推理和视觉任务上均达到了前沿水平。虽然其整体性能仍落后于最强大的闭源模型 Claude Fable 5 和 GPT-5.6 Sol，但 Kimi K3 始终优于我们评估套件中的其他开源和闭源模型。我们发布完整的 Kimi K3 模型权重，以促进未来研究，并加速前沿智能的更广泛部署和应用。

编程 AI 在思考努力上达到最大值：max 或 xhigh。

通用和视觉智能体 AI 在思考努力上达到最大值：max 或 xhigh。

注：Table 5 的结果包含可能的回退。GPT-5.6 Sol 的结果包含可能的网络防护。

图 1: Kimi K3 主要结果。

1 引言

在大语言模型 (LLM) 发展的相当长一段时期里，扩展意味着在部署前投入更多计算资源，通过训练更大的模型处理更多数据 [54, 45]。推理模型的兴起将测试时计算确立为第二维度上的扩展：OpenAI 的 o 系列扩展了强化学习和测试时推理 [84, 83]；Anthropic 的扩展思考模型分配自适应思考预算，并将推理与工具使用交织进行 [6, 7]；DeepSeek-R1 [40] 和 Kimi K1.5 [118] 表明，大规模强化学习能够从强大的预训练模型中激发出复杂的推理行为；Kimi K2.5 Agent Swarm [59] 进一步将测试时扩展从顺序推理推进到并行智能体协同。这些进步使测试时扩展成为前沿研究的核心焦点。然而，虽然开源模型生态在第二维度上进展迅速，但在第一维度上却进展缓慢：许多近期模型仍处于或略高于 1T 级参数区间 [145, 29, 135, 120]。随着日益复杂的推理和智能体强化学习方法被应用于规模相近的预训练基座，开源进展面临趋同的风险，而与最强闭源系统之间的差距则在不断扩大。凭借 Kimi K3，我们同时沿两条扩展轴向前沿推进：将预训练基座扩展到前所未有的 3T 级参数规模，同时在 100 万上下文长度上扩展强化学习、推理努力和长期交互。

我们提出 Kimi K3，一个原生多模态 Mixture-of-Experts 模型，总参数 2.8 万亿，激活参数 1040 亿，上下文窗口最长可达 100 万 token。其架构沿序列长度、网络深度和模型宽度三个维度扩展信息流。Kimi Delta Attention (KDA) [63] 提供高效的长序列混合，通过周期性插入的 Gated MLA 层保持全局交互。Attention Residuals (AttnRes) [57] 使每一层能够有选择地关注来自所有前置层的表示。Stable LatentMoE 将路由专家空间扩展到 896 个专家，每个 token 激活 16 个，同时通过归一化、SiTU-GLU 和 Quantile Balancing 在极端稀疏条件下稳定优化过程。这些架构进步结合优化的数据和训练方案，使整体扩展效率相比 Kimi K2 [58] 提升了约 2.5 倍。

我们将此预训练基座与专为 100 万上下文测试时扩展设计的后训练相结合。Kimi K3 在长期编程、通用智能体、通用推理和知识任务上进行了强化学习，每个任务跨越多个推理努力级别。训练环境包括可验证搜索与专业知识工作、软件工程与内核优化、带视觉回路工具使用的多模态推理、持久助手工作流、Web 开发以及自主执行任务。这些环境训练出一个推理、行动、观察、验证和适应的通用循环，通常涉及数百乃至数千次工具调用和数百万累积的上下文 token。领域和努力级别特化的策略通过多教师在线策略蒸馏 [75, 134, 29] 整合为一个统一模型。

实现这一目标需要基础设施随架构复杂度、模型规模和轨迹长度同步扩展。在面向 KDA 的系统协同设计方面，我们开发了融合内核、KDA Context Parallelism 和状态感知前缀缓存，使 KDA 在设备内、跨设备以及跨请求间均保持高效。针对 2.8T 参数的 MoE 预训练，MoonEP 通过静态计算形状和零拷贝通信实现完美均衡的专家执行，同时高效内存训练和多模态编码器优化在有限内存下维持利用率。针对百万 token 的智能体强化学习，我们的协同部署系统将部分 rollout、外部 KV-cache 保留、自适应限流和可恢复 microVM 沙箱相结合，以保持长生命周期的模型和环境状态。最后，特化内核以及缓存和预算感知的集群调度将这些创新转化为可预测的生产级服务。

最终模型确立了新的开放前沿。在涵盖长期编程、智能体、知识、推理和视觉任务的基准测试中，Kimi K3 整体上落后于最强的闭源系统——Claude Fable 5 和 GPT-5.6 Sol——并且始终领先于我们评估套件中的其他开源和闭源模型，如图 1 所示。

我们的贡献总结如下：

- 开放前沿的预训练。我们训练了一个 2.8T 参数的原生多模态 MoE 模型，激活参数 1040 亿，上下文窗口 100 万 token。KDA、AttnRes、Stable LatentMoE 以及优化的数据和训练方案共同使整体扩展效率相比 Kimi K2 提升了约 2.5 倍。
- 面向多努力级别测试时扩展的强化学习。我们在通用、智能体和编程领域以及多个推理努力级别上进行了强化学习，然后将由此获得的能力整合为一个统一模型。
- 面向数万亿参数、百万 token 智能的基础设施。我们引入了 KDA 系统协同设计；面向 2.8T 参数 MoE 预训练的 MoonEP 和高效内存基础设施；面向百万 token 智能体轨迹、支持可恢复沙箱的协同部署强化学习系统；以及更多基础设施创新。
- 一个开放前沿模型。我们发布完整的 Kimi K3 模型权重，使前沿智能可用于研究、部署和进一步创新。

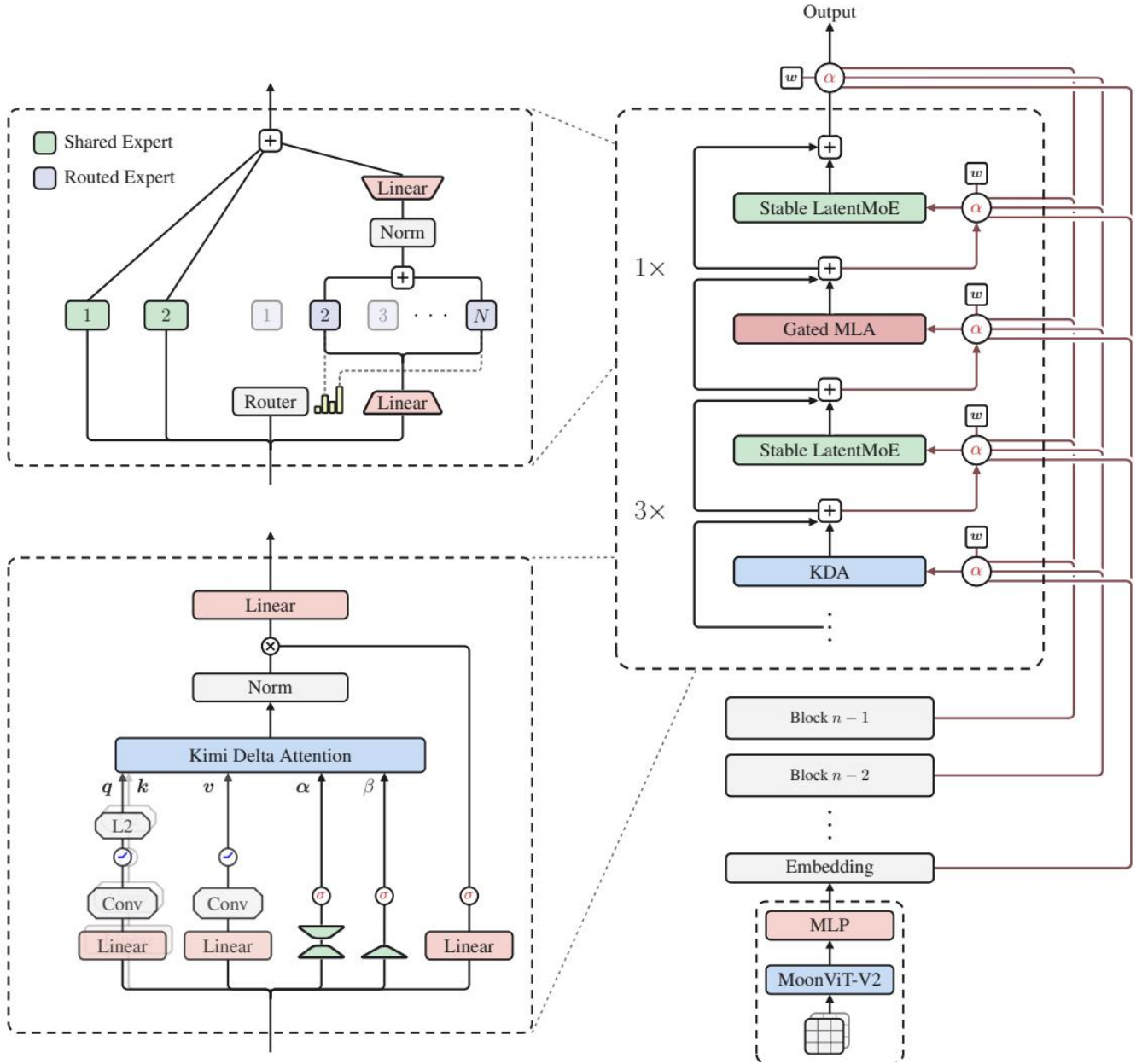


图 2: Kimi K3 架构，围绕 token 混合、通道混合和层混合组织，输入端具有原生视觉通路。每个 block 包含三个 Kimi Delta Attention (KDA) 层，后跟一个 Gated MLA 层，每个注意力层与一个 Stable LatentMoE 前馈网络配对。Attention Residuals (AttnRes) 使用可学习的伪查询 (w) 推导出对嵌入和前置 block 输出的注意力权重 (α)，实现跨深度的选择性信息流动。左上: Stable LatentMoE 模块，包含共享专家和路由专家。左下: KDA 模块。右下: 原生视觉通路。

2 模型架构

Kimi K3 架构从三个互补维度扩展信息流动: 序列长度、网络深度和模型宽度。在序列维度上, Hybrid Attention 在每个 block 中将三层 Kimi Delta Attention (KDA) [63] 与一层 Gated MLA 结合, 为长上下文 token 混合提供高效机制, 同时保留选择性的高容量注意力 (§2.1)。在深度维度上, Attention Residuals (AttnRes) [57] 使每个模块能够从嵌入层、当前 block 以及之前 block 中有选择地检索表示, 将信息访问范围扩展到传统的顺序残差累积之外 (§2.2)。在宽度维度上, 每个注意力层之后跟随一个 Stable LatentMoE 层进行稀疏通道混合, 为每个 token 有效地激活 896 个路由专家中的 16 个 (§2.3)。对于原生视觉, MoonViT-V2 编码图像和视频, 轻量级投影器将得到的视觉特征映射到共享嵌入空间, 然后再进行 backbone 处理 (§2.4)。结合 Per-Head Muon (§2.5), 这些组件提供了一个统一的架构, 用于在 token、层和通道之间扩展信息流动。结合精炼的训练和数据配方, 它们使整体扩展效率相比 Kimi K2 提升约 2.5 倍。图 2 提供了架构概览。

2.1 Hybrid Attention

Kimi K3 采用线性注意力与全局注意力的逐层混合, 将 KDA [63] 与 Gated MLA 结合。每个 block 包含 3 个 KDA 层后接 1 个 Gated MLA 层, 混合比例为 3:1。此模式在整个 backbone 中重复。两个注意力机制分别描述如下。在 backbone 末尾额外放置一个 Gated MLA 层, 确保最后一层始终执行全局注意力。

2.1.1 Kimi Delta Attention

KDA 将 delta-rule 递推 [105, 138] 与逐通道遗忘门 [63] 结合。考虑隐藏状态序列 $\mathbf{x}_t \in \mathbb{R}^d$, 其中 t 索引 token 位置, d 是模型隐藏维度。为清晰起见, 我们先描述单个注意力头, 其 query 和 key 向量为 $\mathbf{q}_t, \mathbf{k}_t \in \mathbb{R}^{d_k}$, value 向量为 $\mathbf{v}_t \in \mathbb{R}^{d_v}$, 递推状态为 $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ 。KDA 在 delta-rule 更新之前应用逐通道衰减:

$$\mathbf{S}_t = \left(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top \right) \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top, \quad \tilde{\mathbf{o}}_t = \mathbf{S}_t^\top \mathbf{q}_t. \quad (1)$$

其中 $\boldsymbol{\alpha}_t \in (0, 1)^{d_k}$ 是逐通道单步保留因子, $\beta_t \in (0, 1)$ 控制 delta-rule 写入强度。遵循 Kimi Linear [63], KDA 将每头参数量参数化为

$$\begin{aligned} \mathbf{q}_t^h, \mathbf{k}_t^h &= \text{L}_2\text{Norm} \left(\text{Swish} \left(\text{ShortConv} \left(\mathbf{W}_{q/k}^h \mathbf{x}_t \right) \right) \right) \in \mathbb{R}^{d_k}, \\ \mathbf{v}_t^h &= \text{Swish} \left(\text{ShortConv} \left(\mathbf{W}_v^h \mathbf{x}_t \right) \right) \in \mathbb{R}^{d_v}, \\ \beta_t^h &= \text{Sigmoid} \left(\mathbf{W}_\beta^h \mathbf{x}_t \right) \in (0, 1), \\ \mathbf{z}_t^h &= \mathbf{W}_\alpha^\uparrow \mathbf{W}_\alpha^\downarrow \mathbf{x}_t + \mathbf{b}_\alpha^h \in \mathbb{R}^{d_k}. \end{aligned} \quad (2)$$

query、key 和 value 投影先应用 ShortConv 再应用 Swish [138], query 和 key 还通过 L_2Norm [141] 进行归一化。低秩投影和每头偏置 $\mathbf{b}_\alpha^h \in \mathbb{R}^{d_k}$ 为每个 key 通道生成细粒度衰减 logit $\{\mathbf{z}\}_{t \sim h}$ 。从 $\{\mathbf{z}\}_{t \sim h}$ 到 α_t^h 的下界映射将在下文 chunkwise 公式之后引入。

Chunkwise 并行形式遵循 Kimi Linear [63], KDA 在 chunk 之间递推, 在每个 chunk 内部并行。对于 chunk 大小 C , $\mathbf{X}_{[t]}$ 堆叠第 t 个 chunk 中的 token 向量, 其中 $\mathbf{X} \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{U}, \mathbf{W}\}$ 。矩阵 $\mathbf{S}_{[t]} \in \mathbb{R}^{d_k \times d_v}$ 表示进入 chunk t 的递推状态。对于位置 $1 \leq i \leq j \leq C$, 定义逐通道累积衰减

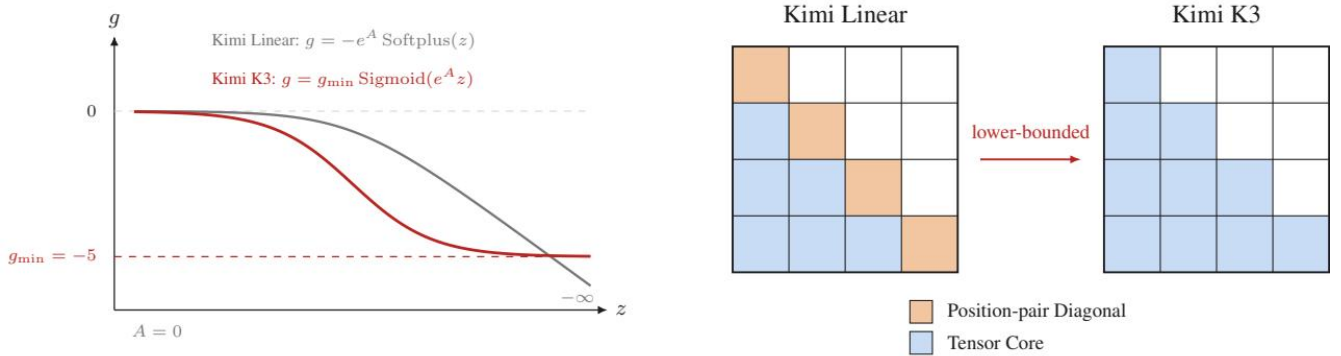
$$\gamma_{[t]}^{i \rightarrow j} := \prod_{r=i}^j \alpha_{[t]}^r, \quad \gamma_{[t]}^r := \gamma_{[t]}^{1 \rightarrow r}. \quad (3)$$

与 Kimi Linear 一样, $\frac{1}{[t]}^C \in \mathbb{R}^{C \times d_k}$ 逐行堆叠 $\gamma_{[t]}^1, \dots, \gamma_{[t]}^C$ 。UT 变换产生 $\mathbf{U}_{[t]}$ 和 $\mathbf{W}_{[t]}$, 由此定义伪 value 项 $\tilde{\mathbf{V}}_{[t]} := \mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}$ 。给定传入状态 $\mathbf{S}_{[t]}$, chunk t 中的所有输出并行计算为

$$\begin{aligned} \mathbf{A}_{[t]} &= \text{Tril} \left[(\mathbf{Q}_{[t]} \odot \frac{1}{[t]}^{\rightarrow C}) (\mathbf{K}_{[t]} / \frac{1}{[t]}^{\rightarrow C})^\top \right], \\ \mathbf{O}_{[t]} &= \underbrace{(\frac{1}{[t]}^{\rightarrow C} \odot \mathbf{Q}_{[t]}) \mathbf{S}_{[t]}}_{\text{跨 chunk}} + \underbrace{\mathbf{A}_{[t]} \tilde{\mathbf{V}}_{[t]}}_{\text{chunk 内}}. \end{aligned} \quad (4)$$

对于矩阵 $\mathbf{M}$, $\text{Tril}(\mathbf{M})$ 将所有严格上三角项置零, 保留下三角项 (包括对角线)。此 mask 强制 chunk 内的因果交互, 对角线被保留是因为每个输出读取当前 token 更新之后的状态。 $\mathbf{O}_{[t]}$ 中的第一项携带来自前面 chunk 的信息, 而第二项则负责当前 chunk 内的交互。我们请读者参考 Kimi Linear [63] 了解 UT 变换和 chunkwise 形式的完整推导。

下界衰减式 4 将每个 chunk 中的 key 按累积衰减的倒数 $1/\Gamma_{[t]}^{1C}$ 重新缩放。由于 $\Gamma_{[t]}^{1C}$ 是 $(0, 1)$ 范围内保留因子的乘积, 这个倒数可以无限增长并在有限精度下溢出 [140, 63]。Kimi Linear 通过对数空间中的相对衰减计算并将每个 chunk 划分为次要的 16-token tile 来控制此数值范围 [140, 63]。非对角线 tile 可以直接用 Tensor Core 上的密集矩阵乘法计算。相比之下, 对角线 tile 仍然需要显式的位置对计算, 这仍然是主要的 chunk 内瓶颈。



(a) 对数衰减参数化。

(b) 对角线 tile 计算。

图 3: 下界衰减及其对 chunkwise KDA 计算的影响。(a) Kimi Linear 使用无界负 Softplus 映射, 而 Kimi K3 用缩放 sigmoid 限制对数衰减的下界; 曲线显示 $A = 0$ 和 $g_{\min} = -5$ 。(b) Kimi Linear 对每个对角线 tile 使用显式位置对计算, 而 Kimi K3 中的有界范围允许所有因果 tile 使用密集 Tensor Core 矩阵乘法。

Kimi K3 通过改变从衰减 logit z_t^h 到每步对数衰减 g_t^h 的映射来解决此瓶颈。遵循 GDN 和 Mamba-2, Kimi Linear 使用负 Softplus 映射 $g_t^h = -e^{A_h} \text{Softplus}(z_t^h) \in (-\infty, 0)^{d_k}$ [138, 24, 63]。Kimi K3 改用缩放 sigmoid

将对数衰减从下方约束：

$$\begin{aligned} \mathbf{g}_t^h &= g_{\min} \text{Sigmoid}(e^{A_h} \mathbf{z}_t^h) \in (g_{\min}, 0)^{d_k}, \\ \boldsymbol{\alpha}_t^h &= \exp(\mathbf{g}_t^h) \in (e^{g_{\min}}, 1)^{d_k}, \end{aligned} \quad (5)$$

其中 A_h 是可学习的每头对数尺度， $g_{\min} = -5$ 是固定的。我们初始化 $A_h = 0.$ ，每个偏置 b_α^h 按照 [63, 24, 138] 初始化。在 $g_{\min} = -5$ 下，每个保留因子满足 $\alpha_{t,j}^h > e^{-5} \approx 6.7 \times 10^{-3}$ ，16-token tile 上的累积对数衰减落在 $(-80, 0)$ 范围内。对应的倒数重缩放因子因此小于 e^{80} ，保持在 BF16 动态范围内。这个有限范围使得对角线和非对角线 tile 都能使用密集 Tensor Core 矩阵乘法，消除了位置对对角线路径。此参数化与先前工作中的下界递推门 [97, 27, 91] 密切相关。图 3 说明了衰减参数化的变化及其计算后果。

全秩门控最后，Kimi K3 将 KDA 的输出门从 Kimi Linear [63] 使用的低秩参数化改为输入依赖的全秩投影。在对递推输出应用逐头 RMSNorm [146] 之后，KDA 应用数据依赖的输出门控 [99]：

$$\mathbf{y}_t = \mathbf{W}_o[\text{Sigmoid}(\mathbf{W}_g \mathbf{x}_t) \odot \text{RMSNorm}(\tilde{\mathbf{o}}_t)]. \quad (6)$$

2.1.2 Gated MLA

Multi-head Latent Attention (MLA)，由 DeepSeek-V2 [28] 引入，将每个 token 的 key-value 表示压缩到低维 latent 向量 $\mathbf{c}_t = \mathbf{W}_c \mathbf{x}_t$ 。MLA 不缓存完整的每头 key 和 value，而是缓存 $\mathbf{c}_t$ ，并在注意力计算过程中通过学习的升维投影重建内容 key 和 value。这种分解减少了 KV-cache 占用，同时保留了全局 token-to-token 注意力。MLA 随后被 Kimi K2 和 Kimi K2.5 [58, 59] 采用，Kimi K3 在周期性全局注意力层中保留它。

与 Kimi K2 和 Kimi K2.5 不同，Kimi K3 遵循 Kimi Linear [63] 的混合设计，对所有 MLA 层应用 No Position Encoding (NoPE)。因此，不对其 query 或 key 应用任何显式位置编码。中间的 KDA 层提供位置敏感和近因感知的序列混合，而 MLA 层提供无限制的全局内容交互。这种分离还避免了在扩展上下文长度时修改位置编码参数，例如重新调整 RoPE 频率基或应用 YaRN [92]。

此外，Kimi K3 为 MLA 增加了一个输入依赖的、逐通道的全秩输出门。设 $\tilde{\mathbf{o}}_t$ 表示位置 t 处未门控的 MLA 输出；门控输出为

$$\mathbf{y}_t = \mathbf{W}_o[\text{Sigmoid}(\mathbf{W}_g \mathbf{x}_t) \odot \tilde{\mathbf{o}}_t]. \quad (7)$$

门控投影 $\mathbf{W}_g$ 是全秩的，与 Kimi K3 中 KDA 使用的新参数化一致。此门控允许每个 token 调制从全局注意力读取的通道 [99]。

为纠正 flash attention 中出现的偏置舍入误差，我们采用 [98] 的方法，在训练期间将注意力输出保持在 FP32。此选择使输出 tile 的片上占用翻倍；因此我们重新设计了训练 kernel，将其与 KV staging buffer 而非 query tile 重叠，释放共享内存以支持更深的 KV 流水线和更高的训练吞吐量。

2.2 Attention Residuals

标准残差连接 [43] 将所有先前信息压缩到深度上的单个状态 h_l ——这是一个类似 RNN 在时间上的瓶颈。对于序列建模，Transformer 用注意力取代了递推 [10, 125]，允许每个位置用数据依赖的权重有选择地访问所有先前位置。Attention Residuals (AttnRes) [57] 将相同的方法应用于深度：每一层有选择地从所有先前层检索表示，而不是均匀地累积它们。

全注意力残差对于每一层 l ，我们定义层特定的可学习伪 query $\mathbf{q}_l = \mathbf{w}_l \in \mathbb{R}^d$ 以及 key 和 value

$$\mathbf{k}_i = \mathbf{v}_i = \begin{cases} \mathbf{h}_1 & i = 0 \\ f_i(\mathbf{h}_i) & 1 \leq i \leq l-1 \end{cases} \quad (8)$$

其中 $f_i(h_i)$ 是第 i 层的输出， $\mathbf{h}_1$ 是 token 嵌入。注意力权重遵循 softmax 核 $\phi(\mathbf{q}, \mathbf{k}) = \exp(\mathbf{q}^\top \text{RMSNorm}(\mathbf{k}))$ [55, 146]，其中 RMSNorm 防止输出幅值较大的层主导权重：

$$\alpha_{i \rightarrow l} = \frac{\phi(\mathbf{q}_l, \mathbf{k}_i)}{\sum_{j=0}^{l-1} \phi(\mathbf{q}_l, \mathbf{k}_j)}, \quad \mathbf{h}_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} \cdot \mathbf{v}_i. \quad (9)$$

由于网络深度适中 ($L < 100$)，这种全量形式的 $O(L^2 d)$ 算术开销是可以接受的；实际开销是保持所有层输出活跃的 $O(Ld)$ 内存（以及流水线并行下的跨阶段通信）。

Block 注意力残差为降低此开销，我们将 L 层划分为 N 个 block，每个 block 含 $S = L/N$ 层。在 block n 内（层索引 B_n ），层输出通过求和归约为单个表示， $\mathbf{b}_n = \sum_{j \in B_n} f_j(h_j)$ ，其中 $\mathbf{b}_n^i$ 表示 block 中前 i 层的部分和；我们设 $\mathbf{b}_0 = \mathbf{h}_1$ 使得 token 嵌入始终作为源被包含。跨 block，仅对 N 个 block 级表示应用全注意力：对于 block n 中的第 i 层，value 矩阵为

$$\mathbf{V} = \begin{cases} [\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1}]^\top & \text{若 } i \geq 2 \text{ (后续层)} \\ \top & \text{若 } i = 1 \end{cases} \quad (10)$$

key 和注意力权重遵循式 8 和式 9。最终输出层随后聚合所有 N 个 block 表示。在 Block AttnRes 下，内存和通信开销从 $O(Ld)$ 降至 $\bar{O}(Nd)$ ，同时这种 block 结构还限制了推理时的状态，使得并行的跨 block 结果能够通过在线 softmax [79] 更好地与顺序的 block 内部部分和合并，显著减少推理时间成本。

经验上， $N \approx 8$ 在不同模型规模下恢复了大部分收益 [57]；对于 Kimi K3，我们将其层划分为 8 个 block，每个 block 12 层，形成不完整的最后一个 block 以及计入嵌入层后总共 9 个 block。

2.3 Stable LatentMoE

增加专家池和活跃专家数量可以扩展专家特化的空间，但在传统 MoE 中，每个被选中的专家接收完整的 d 维 token 表示，因此通信和专家权重流量随路由多重度增长。LatentMoE [32] 通过将完整模型宽度与路由专家宽度分离，使这种扩展变得经济可行：共享专家保留全宽路径用于常见变换，而特化的路由专家在宽度为 ℓ 的

紧凑 latent 空间中运行。这使得 Kimi K3 能够将通道混合扩展到 896 个路由专家，每个 token 激活 16 个，对应的稀疏度为 56。

这种极端稀疏度放大了原始设计的两种失效模式。首先，路由路径将 $\mathbf{W}^\downarrow$ 、一个门控多分支专家前馈网络和 $\mathbf{W}^\uparrow$ 组合成近四个连续矩阵乘法的链。这种病态结构，加上 2.8 万亿参数规模，在路由分支中产生爆炸式的内部激活。其次，平衡近 10^3 个专家的负载超出了现有无辅助损失偏置更新保持良好行为的范围。Stable LatentMoE 通过三个组件应对这两种失效模式：在升维投影前的 RMSNorm 和 Sigmoid Tanh Unit GLU (SiTU-GLU) 用于抑制激活爆炸，以及 Quantile Balancing (QB) 用于负载均衡。

	门分支	升维分支	曲线
GLU [26]	$\sigma(x)$	x	
SwiGLU [107]	$x \cdot \sigma(x)$	x	
SiTU-GLU	$\beta_1 \tanh(x/\beta_1) \cdot \sigma(x)$	$\beta_2 \tanh(x/\beta_2)$	

图 4: GLU、SwiGLU 和 SiTU-GLU 的门分支和升维分支，及其标量响应，其中 σ 表示 sigmoid 函数。两个分支均接收标量输入 x ，所有曲线共享定义域 $x \in [-10, 100]$ ；插图为原点附近的放大视图。SiTU-GLU 以红色显示， $\beta_1 = 4$ 和 $\beta_2 = 25$ ，在原点附近紧密跟随 SwiGLU，并且对于大的正输入趋近于界 $|\bar{f}(x)| \leq \beta_1 \beta_2 = 100$ ，而 SwiGLU 保持无界。

如图 2 所示，该层遵循 DeepSeekMoE [23] 的共享与路由专家组织方式。对于 $\mathbf{x} \in \mathbb{R}^d$ ，共享专家直接处理 $\mathbf{x}$ ，而路由路径将其投影 $\mathbf{z} = \mathbf{W}^\downarrow \mathbf{x} \in \mathbb{R}^\ell$ ，将 $\mathbf{z}$ 分发给选中的专家，并通过 $\mathbf{W}^\uparrow$ 将其加权聚合映射回 $\mathbb{R}^d$

$$\begin{aligned} \mathbf{u} &= \sum_{i \in \mathcal{T}_k(\mathbf{x})} p_i E_i^{\text{routed}}(\mathbf{W}^\downarrow \mathbf{x}), \\ \mathbf{y} &= \sum_{j=1}^{N_s} E_j^{\text{shared}}(\mathbf{x}) + \mathbf{W}^\uparrow \text{RMSNorm}(\mathbf{u}). \end{aligned} \quad (11)$$

其中 $\mathbf{u} \in \mathbb{R}^\ell$ 是聚合后的路由表示， $E_i^{\text{shared}}: \mathbb{R}^d \rightarrow \mathbb{R}^d$ 和 $E_i^{\text{routed}}: \mathbb{R}^\ell \rightarrow \mathbb{R}^\ell$ 分别是共享和路由专家前馈网络， p_i 是下文 Quantile Balancing 规则定义的路由权重。Kimi K3 在每层固定全宽共享专家的数量为 $N_s = 2$ 。

2.3.1 Normalized LatentMoE

原始 LatentMoE 直接将 $\mathbf{W}^\uparrow$ 应用于聚合后的路由表示 $\mathbf{u}$ ，其尺度可能因所选专家及其路由权重而变化。如式 11 所示，Kimi K3 改为在专家聚合和升维投影之间插入 RMSNorm [146]。此归一化降低了路由分支对尺度变化的敏感性，使其在与全宽共享分支合并之前更加稳定。除了稳定训练外，额外的 RMSNorm 持续改善验证损失和下游基准。

2.3.2 Sigmoid Tanh Unit GLU

Gated Linear Units (GLUs) 用 sigmoid 激活的门调制线性 value 分支，计算 $\text{Sigmoid}(\mathbf{W}_g \mathbf{x}) \odot \mathbf{W}_u \mathbf{x}$ [26]。SwiGLU 将 sigmoid 门替换为 Swish $(x) = x \text{Sigmoid}(x)$ ，在 Transformer 中取得了很强的经验性能 [107]。SwiGLU 随后成为大语言模型中广泛采用的 FFN 设计，但对其经验有效性的完整解释仍然未决。

然而，SwiGLU 中的两个乘法因子都是无界的，因此重合的大值坐标可能导致激活异常值并增加低精度算术中的溢出风险。原始 GLU 的 sigmoid 门避免了门的无界增长，但它不保留 Swish 的近似线性正区间。这促使

设计一种激活函数，在控制大值增长的同时保留 SwiGLU 的特征局部和正侧响应。最近的其他工作也探索了这种权衡的替代参数化 [51]。

为满足这些要求，我们提出了 Sigmoid Tanh Unit GLU (SiTU-GLU)。SiTU-GLU 将平滑上限 $\text{softcap}(x, \beta) = \dot{\beta} \tanh(x/\beta)$ 应用于 Swish 门的线性因子，并独立应用于升维分支：

$$\text{SiTU-GLU}(\mathbf{x}) = \left[\beta_1 \tanh\left(\frac{\mathbf{W}_g \mathbf{x}}{\beta_1}\right) \odot \text{Sigmoid}(\mathbf{W}_g \mathbf{x}) \right] \odot \left[\beta_2 \tanh\left(\frac{\mathbf{W}_u \mathbf{x}}{\beta_2}\right) \right], \quad (12)$$

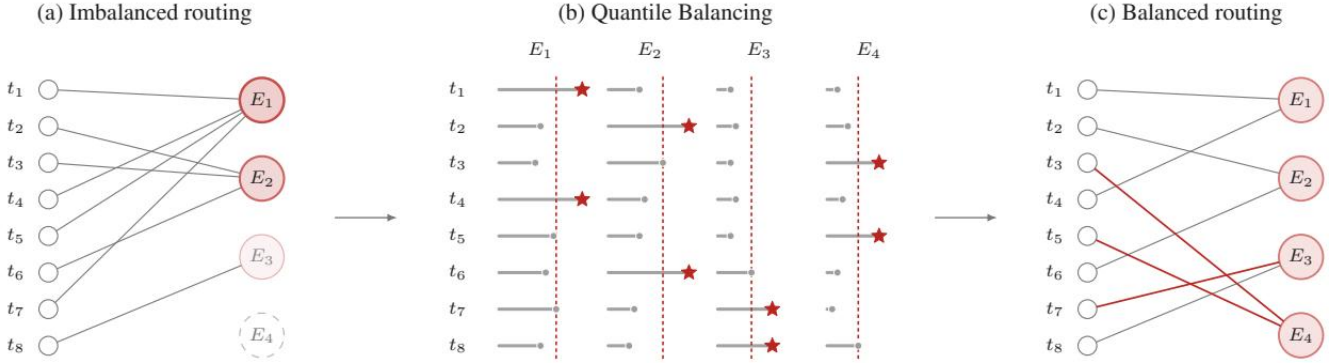


图 5: Quantile Balancing 的示意，其中 $m = 8$ 个 token， $n = 4$ 个路由专家，每个 token 选择 $k = 1$ 个专家。(a) 逐 token Top-k 路由（左侧为 token，右侧为专家）产生负载 (4, 3, 1, 0)；较深的圆圈表示过热专家，而褪色和虚线圈分别表示未充分利用和消亡的专家。(b) 每个灰色条是当前偏置分数的 margin， $s_{i,j} + b_j^{(t)} - \alpha_i^{(t)}$ ，因此逐行最大值再现了 (a) 中的路由。每列中的红色虚线是偏置调整 $b_i^{(t)} - \hat{b}_i^{(t+1)}$ ，放置在 $(q+1)$ 大的 margin 处，使得恰好 $q = 2$ 个 margin 超过它。标记 $\star$ 表示减去列调整后的逐行 Top-k 选择，即 (c) 中的路由。(c) 保留的选择产生均衡负载 (2, 2, 2, 2)；红色边表示被 QB 改变的分派。

对于 Kimi K3，我们将 soft-cap 超参数设置为门分支 $\beta_1 = 4$ 和升维分支 $\beta_2 = 25$ 。缩放 $\tanh$ 在原点附近近似线性，在大幅值时是有界的，使 SiTU-GLU 能够保留 SwiGLU 的局部响应，同时控制乘积中的两个因子。图 4 比较了 GLU、SwiGLU 和 SiTU-GLU 在公共切片上的分支定义和标量响应。

§ B 给出了局部展开、极限情况、形式化输出界以及与硬截断的比较。

2.3.3 Quantile Balancing

与基于辅助损失的路由 [33] 不同，Kimi K3 采用无辅助损失路由 [30]。负载均衡通过向用于 Top-k 选择的路由分数添加专家特定偏置 b_j 实现。对于 token x_i ，路由计算 $s_i = \text{Sigmoid}(\mathbf{W}_r \mathbf{x}_i)$ 并应用

$$\mathcal{T}_i = \text{argtop}_k(\mathbf{s}_i + \mathbf{b}), \quad p_{i,j} = \frac{s_{i,j}}{\sum_{r \in \mathcal{T}_i} s_{i,r}}, \quad j \in \mathcal{T}_i. \quad (13)$$

因为 $\mathbf{b}$ 在 $p_{i,j}$ 中被省略，它调节分派而不改变混合权重或路由器的梯度优化。原始方法用固定步长规则更新 $\mathbf{b}$: $b_i^{(t+1)} = b_i^{(t)} + \gamma \text{sign}(\bar{\ell} - \ell_i^{(t)})$ [30]，其中 γ 在缓慢适应和负载振荡之间权衡。当 LatentMoE 将每层路由专

家池增加到 896 时，保持均衡负载变得更具挑战性。不均衡的路由会减慢专家并行训练，并可能使某些专家训练不足 [47]。

为了解决此限制，我们引入 Quantile Balancing (QB)，它从匹配目标负载的路由分数量化点设置每个专家偏置 [111]。考虑一个训练批次包含 m 个 token，路由到 n 个专家，使用 Top-k 选择，因此目标负载为每个专家 $q := mk/n$ 个 token。QB 通过单次前向传播推导下一个偏置。路由将 Top-k 选择替换为对有偏分数 $s_i + b^{(t)}$ 的 Top-(k+1)：前 k 个条目是实际采用的路由，而第 $(k+1)$ 个条目是截止值 $\alpha_i^{(t)}$ ，专家必须超过它才能进入 token i 的 Top-k。从 Top-(k+1) 路由取截止值避免了单独的 token 侧分位点。然后我们选择每个专家偏置，使得专家 j 达到其目标负载：在截止值固定的情况下，在候选偏置 $\widehat{b}_j^{(t+1)}$ 下路由到专家 j 的 token 计数为

$$\sum_{i=1}^m \mathbf{1} \left[s_{i,j} + \widehat{b}_j^{(t+1)} > \alpha_i^{(t)} \right],$$

该计数关于阈值 $-\widehat{b}_j^{(t+1)}$ 单调递减。假设无平局，将此计数设为 q 使得 $-\widehat{b}_j^{(t+1)}$ 成为第 $(q+1)$ 大的 margin $s_{i,j} - \alpha_i^{(t)}$ ，从而恰好 q 个 margin 保持在阈值之上。由于 $q/m = k/n$ ，这是跨 token margin 的 $(1 - k/n)$ 分位数，给出 QB 更新

$$\begin{aligned} \widehat{b}_j^{(t+1)} &\leftarrow -\text{quantile}_{1-k/n}(\mathbf{s}_{:,j} - \boldsymbol{\alpha}^{(t)}), \\ \mathbf{b}^{(t+1)} &\leftarrow \widehat{\mathbf{b}}^{(t+1)} - \text{mean}(\widehat{\mathbf{b}}^{(t+1)})\mathbf{1}. \end{aligned} \tag{14}$$

margin 从原始分数 $s_{i,j}$ 中减去有偏截止值 $\alpha_i^{(t)}$ ，因此旧偏置仅通过截止值影响更新，第二行移除不影响 Top-k 选择的公共偏移。为保证因果性，更新仅在下一步生效 [30]，即批次从不使用从其自身推导的偏置进行路由。图 5 说明了 $m = 8, n = 4$ 和 $k = 1$ 的情况，其中每个专家获得目标负载 $q = 2$ 。最终偏置在推理时冻结。均衡分派的推导见 $S \subset$

直方图估计在大规模下，式 14 中的分位数跨越整个全局批次，其 margin 数量达数百万并分布在多个 rank 和累积步骤中，因此在训练时收集它们进行精确分位数计算是不可行的。我们改为从每个专家的 margin 直方图中读取分位数：单次 all-reduce 汇总每 rank 的 bin 计数，分位数从汇总计数中恢复。由于计数是可加的，直方图表示汇聚后的全局批次，无论 token 如何分片，因此估计反映了全局批次分位数（精确到 bin 宽度），通信开销仅为每个专家几百个 bin。此直方图估计器是我们在实践中使用的方法；我们在 § D 中给出其详细描述及误差界。

2.4 原生视觉

Kimi K3 是原生多模态的：文本、图像和视频在单个上下文中由单一共享 backbone 处理，无需后置模态对齐阶段。此设计是 §1 中描述的长时间视野、循环中视觉行为（vision-in-the-loop）的架构基础。渲染输出及其产生代码存在于同一 token 流中，模型可以编写代码、检查结果的截图或视频帧，并迭代优化视觉产物——用户界面、图形、视频——无需跨模型交接。

MoonViT-V2 Kimi K2.5 的一个关键不同之处在于，我们使用下一 token 预测从头训练 Kimi K3 的视觉编码器 MoonViT-V2。先前的实践（包括 Kimi K2.5 本身）用对比预训练模型（如 SigLIP）初始化视觉编码器，前

提是预训练的视觉知识能给模型带来先发优势。我们偏离此实践主要是为了训练稳定性。当预训练编码器附加到 LLM 时，联合优化变得不稳定：SigLIP 初始化的 MoonViT-3D 显示出持续更高的梯度范数，并伴随频繁的尖峰，而 MoonViT-V2 在整个训练过程中保持稳定（图 6）。使用下一 token 预测训练还允许编码器的表示直接由语言建模目标塑造，而不是由偏向全局语义而非细粒度文本和结构线索的对比损失来塑造。值得注意的是，我们发现 MoonViT-V2 在各种视觉评估上与 SigLIP 初始化的基线持平，表明在大规模多模态语言模型中对比预训练作为初始化是不必要的。

(a) 完整训练轨迹

(b) 放大视图 (14k–16k)

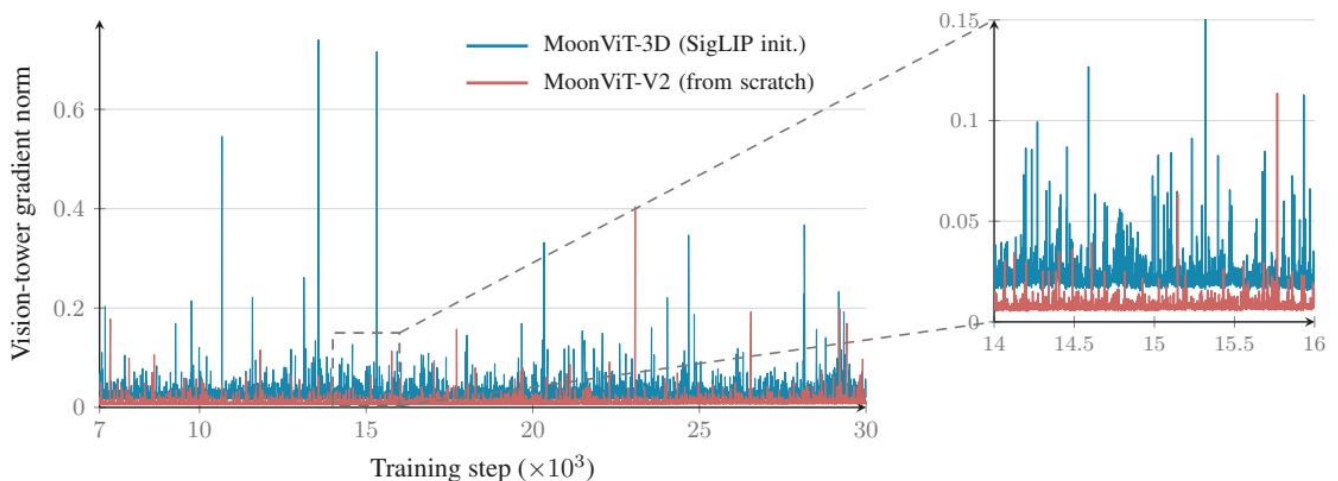


图 6: 预训练消融实验中的视觉塔梯度范数。与 SigLIP 初始化的 MoonViT-3D 相比，从头训练的 MoonViT-V2 保持更低的梯度范数和更少的尖峰，表明优化更加稳定。

架构此训练方案建立在遵循 Kimi K2.5 [59, 61] 整体设计的视觉通路上：视觉输入首先由 MoonViT-V2 编码，然后通过轻量级 MLP 投影器映射到 LLM。MoonViT-V2 是一个 27 层的 vision transformer，参数约 0.4B，采用 RMSNorm 并从其线性 and 注意力投影中移除所有偏置项，这一设计进一步稳定了上述的从头优化。图像和视频以完全共享的参数处理，如 MoonViT-3D 中一样：注意力分解为帧内空间和帧间时间传递，时间池化进一步沿时间维度压缩 token。在投影之前，带有 2x2 下采样的 pixel-shuffle 操作将视觉 token 数量减少四倍，使高达 3584x3584 像素的输入在 1M-token 上下文中仍然可行。

2.5 Per-Head Muon

遵循 Kimi K2，Kimi K3 采用 Muon [53] 作为矩阵参数的优化器。对于注意力投影，我们将其进一步细化为 per-head 变体：不是将 Newton-Schulz 正交化应用于完整的 Q、K 和 V 投影矩阵，而是沿着 head 维度划分它们的动量矩阵，并分别对每个 head 的块进行正交化。其直觉是全矩阵正交化将所有 head 视为单一耦合块，因此梯度或动量尺度较大的 head 主导共享的更新方向，而尺度较小的 head 获得的归一化更新不足；per-head 正交化均衡了各 head 之间的更新尺度。在实践中，此设计产生了更均衡的跨 head 学习动态，并在更大规模上提高了训练稳定性。它还略微降低了优化器开销，因为在 tall per-head blocks 上的 Newton-Schulz 迭代比对完整投影矩阵计算更便宜。

3 预训练

3.1 预训练数据

Kimi K3 在一个经过精心筛选的语料库上进行预训练，该语料库涵盖四大文本领域——网页文本、代码、数学和知识——以及一个大规模视觉语料库。视觉数据涵盖图像描述、图文交错文档、OCR、感知、视频和视觉编码数据。我们的数据流水线建立在为 Kimi K2 [58] 开发、并在 Kimi K2.5 [59] 中进一步优化的流水线基础之上。

文本数据每个领域通过基于规则的启发式方法、基于分类器的质量评分和去重的组合进行过滤，各领域的采样率由在小模型上的消融实验确定。遵循 Kimi K2 [58] 的重述 (rephrasing) 方案，我们使用风格和视角多样化的提示、分块自回归生成以及针对源文档的保真度验证，对知识和数学语料进行重述。

视觉数据视觉语料库遵循 Kimi K2.5 [59] 的分类体系，将开源数据集与内部过滤、合成和去重流水线相结合。在训练过程中，坐标监督同时以绝对格式和归一化 ($[0,1]$) 格式提供，从而实现精确且对分辨率鲁棒的定位。除了经典的文本标注图像外，我们还大幅扩展了程序化多模态数据，将代码片段与其渲染视觉效果配对，覆盖 SVG、3D 资产、网页、游戏和 CAD 示意图等特定领域格式。

3.2 缩放定律

综合来看，前述各节所描述的架构、数据和训练改进共同定义了我们的新模型系列。由于这些变化也改变了最优训练范式，我们进行了专门的缩放定律研究，以重新调优关键超参数，包括批量大小、学习率、每参数 token 比 (TPP) 以及模型形状。在留出的 OOD 验证数据上评估，图 7 中的缩放定律曲线显示，这些改进整体上带来了相对于 Kimi K2 约 2.5 倍的缩放效率提升。表 1 提供了 Kimi K2 与 Kimi K3 之间详细的架构对比，突出展示了促成这一提升的结构性变化。

我们的缩放定律研究一致表明余弦衰减优于 Warmup Stable Decay (WSD) [46]，因此我们采用余弦衰减作为默认学习率调度策略。我们在固定的最低学习率下比较了余弦衰减和 WSD。尽管先前工作报道 WSD 可以匹敌甚至超越余弦衰减，但我们观察到两种调度策略的最优超参数存在显著差异。即使在相同的模型规模和训练 token 预算下，它们的最优峰值学习率和批量大小也大相径庭。因此，使用同一组超参数来比较两种调度策略可能会不公平地偏袒其中一方，仅仅是因为那些超参数与它更匹配。为确保公平比较，我们为每种调度策略分别进行了独立的缩放定律搜索。在各自的最优超参数设置下，余弦衰减始终取得比 WSD 更低的最终损失。

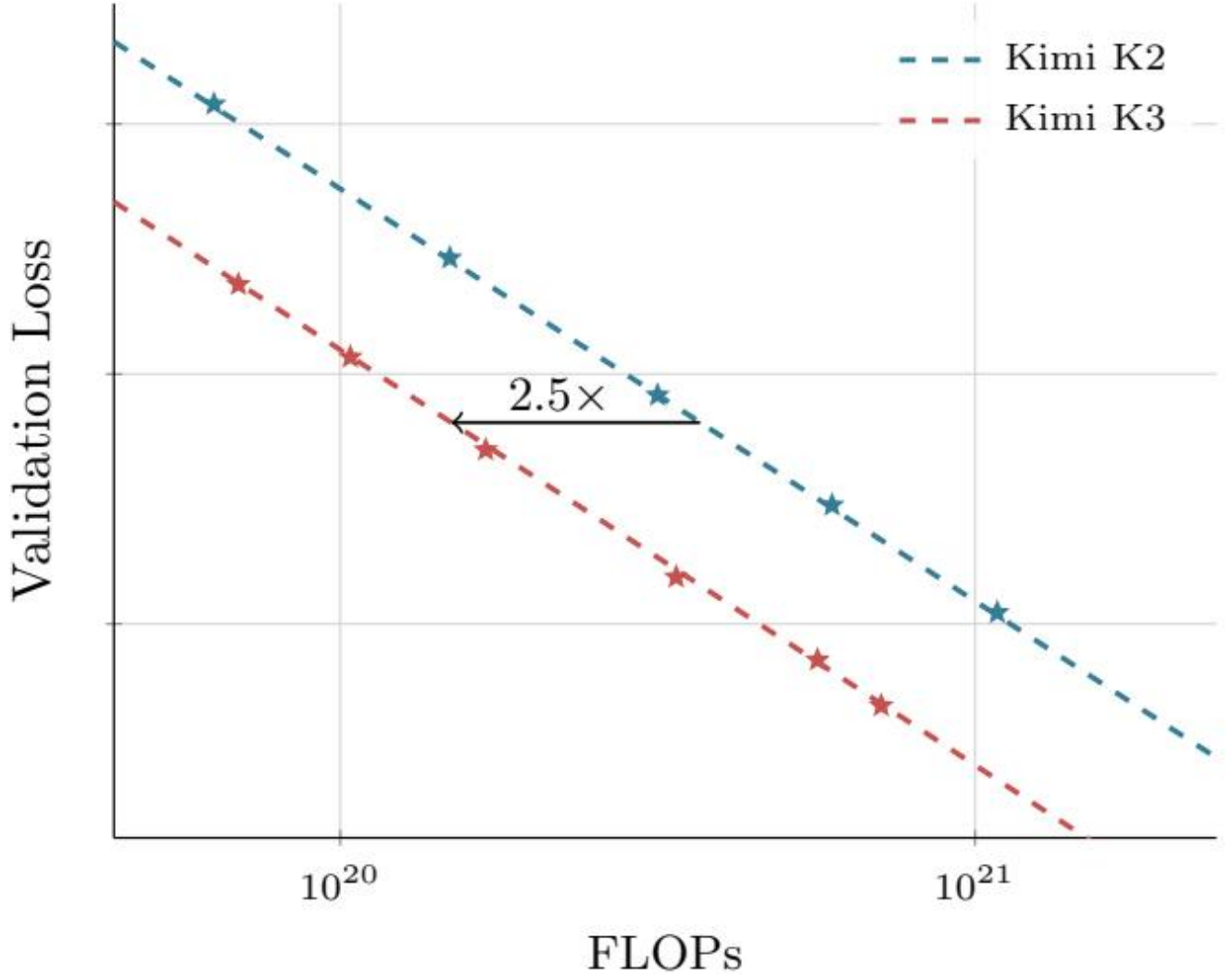


图 7: Kimi K2 和 Kimi K3 的拟合缩放定律曲线。Kimi K3 相比 Kimi K2 实现了 2.5 倍的缩放效率提升。

表 1: Kimi K2 与 Kimi K3 的架构对比。

	Kimi K2	Kimi K3	Δ
架构	MoE	MoE	-
层数	61	93	$\uparrow 52\%$
总参数量	1.04T	2.78T	$\uparrow 167\%$
激活参数量	32.6B	104.2B	$\uparrow 220\%$
隐藏维度	7,168	7,168	=
潜在 MoE 维度	-	3584 ($0.5\times$)	-
每专家 MoE 隐藏维度	2,048	3,072	$\uparrow 50\%$
路由专家数	384	896	$\uparrow 133\%$

每 token 激活专家数	8	16	↑ 100%
共享专家数	1	2	↑ 100%
注意力头数	64	96	↑ 50%
稠密层数	1	1	=
词表大小	160K	160K	=
训练上下文长度	128K	1M	8×
注意力机制	MLA	混合 KDA-MLA	-
激活函数	SwiGLU	SiTU-GLU	-
注意力层组成	61 MLA	69 KDA + 24 MLA	-
MTP 层数	1 层	1 层	=
ViT 总参数量	-	401M	-
ViT 层数	-	27 层	-
ViT Patch 大小	-	14	-
ViT 注意力头数	-	12	-

3.3 训练配方

Kimi K3 采用原生多模态训练策略，语言和视觉从训练伊始就联合优化，而不是通过事后对齐阶段将视觉编码器嫁接到预训练语言模型上。在此范式下，视觉 token 和文本 token 在统一的下一 token 预测目标中交错排列，使共享主干网络能够从一开始就学习统一的多模态表示。

我们使用 Per-Head Muon 优化器 (§ 2.5) 以及 Kimi K2 中引入的权重裁剪机制来优化模型，同时采用 QB (§ 2.3.3) 进行 MoE 负载均衡。我们使用余弦学习率调度，配合 1% 的线性预热。整个训练过程中权重衰减设为 0.1。

我们的预训练从 8k token 的上下文长度开始，随后在后续训练阶段扩展到 64k token。

3.4 长上下文扩展

位置编码 Kimi K3 不使用显式位置嵌入 (NoPE)，而是通过 KDA 的循环门控和衰减机制隐式编码位置信息。因此，模型可以直接外推到 1M token 上下文，而无需任何位置编码修改，例如 RoPE 重缩放或插值 [92]。

长上下文数据来自自然来源的长文档和视频包含大量低质量内容，包括近重复项、二进制 blob、截断文件、视频片段和无效的机器生成日志。因此，我们通过一个专门的清洗流水线对其进行处理，该流水线结合了精确去重和模糊去重、针对视频的逐帧感知哈希、基于启发式和分类器的质量过滤以及结构验证。由于真正长且连贯的文档和视频相对于短文本而言十分稀缺，我们对它们进行上采样，以确保在冷却 (cooldown) 阶段长上下文分布不会被短序列淹没。然而，仅靠长度本身并不能赋予长程能力。为解决这一问题，我们通过精心排列和拼接多模态文档及子任务来合成额外的长上下文数据，使得嵌入其中的任务只有通过关注分散在整个 1M token 上下文中的信息才能解决。这使注意力机制在预期规模上得到训练，防止其退化为局部模式。

渐进式上下文扩展 Kimi K3 支持高达 100 万 token 的上下文窗口。我们通过在训练过程中渐进式扩展上下文窗口来实现这一目标，遵循四阶段课程。窗口在预训练期间从 8K 扩展到 64K token，在冷却阶段从 256K 扩

展到 1M token。将昂贵的长序列计算集中在整体训练预算的一小部分内，使得课程既经济高效，又能让模型逐步适应越来越长的依赖关系。使 KDA 层能够进行百万 token 训练的序列维度划分详见 §5.1.2。

4 后训练

4.1 方法

我们的后训练流水线遵循三阶段范式：通过监督微调（SFT）初始化基线智能体能力，通过强化学习（RL）在不同推理努力程度下培养领域专用专家，以及使用多教师在线策略蒸馏（MOPD）将这些领域专用策略整合到单一模型中。

4.1.1 监督微调

SFT 阶段为后续 RL 阶段建立一个高质量的冷启动策略。在先前 Kimi 模型 [58, 59] 的 SFT 流水线基础上，我们为 Kimi K3 扩展了 SFT 数据集，大幅拓宽了其对复杂智能体任务的覆盖范围。具体而言，我们使用来自先前 Kimi 系列的领域专用模型来合成数据轨迹，随后进行多阶段验证和人工参与（human-in-the-loop）标注。为一致地表示这些复杂的智能体轨迹，我们使用基于 XTMl 的对话模板（可扩展 Token 标记语言；详见 §F）序列化所有数据。综合来看，这些步骤产出了一个大规模指令数据集，赋予 Kimi K3 自适应推理、精确工具调用以及在长周期智能体场景中稳健执行的能力。此外，我们从 SFT 阶段起即应用量化感知训练（QAT），采用 MXFP4 权重和 MXFP8 激活 (§ 4.1.4)。

4.1.2 强化学习

虽然 SFT 提供了坚实的冷启动基础，但 RL 对于释放更高阶的推理和执行能力至关重要。我们没有为单个任务训练专用的 RL 模型，而是在三个广泛领域上扩展 RL——每个领域涵盖广泛的子任务——并在每个推理努力级别为每个领域训练一个单独的专家：(i) 通用任务，涵盖通用经验、视觉、推理、忠实性、搜索能力和知识工作任务；(ii) 通用智能体，涵盖长周期助手任务、深度研究（deep research）和段落级写作；以及 (iii) 编码智能体，涵盖软件工程（SWE）、编码经验、内核任务和 Web 开发。如图 8 所示，扩展 RL FLOPs 能够持续提升知识、推理、视觉、通用智能体和编码方面的多种能力。将这三个领域专家与低、高、最大三个推理努力级别交叉，共产生九个专家模型。

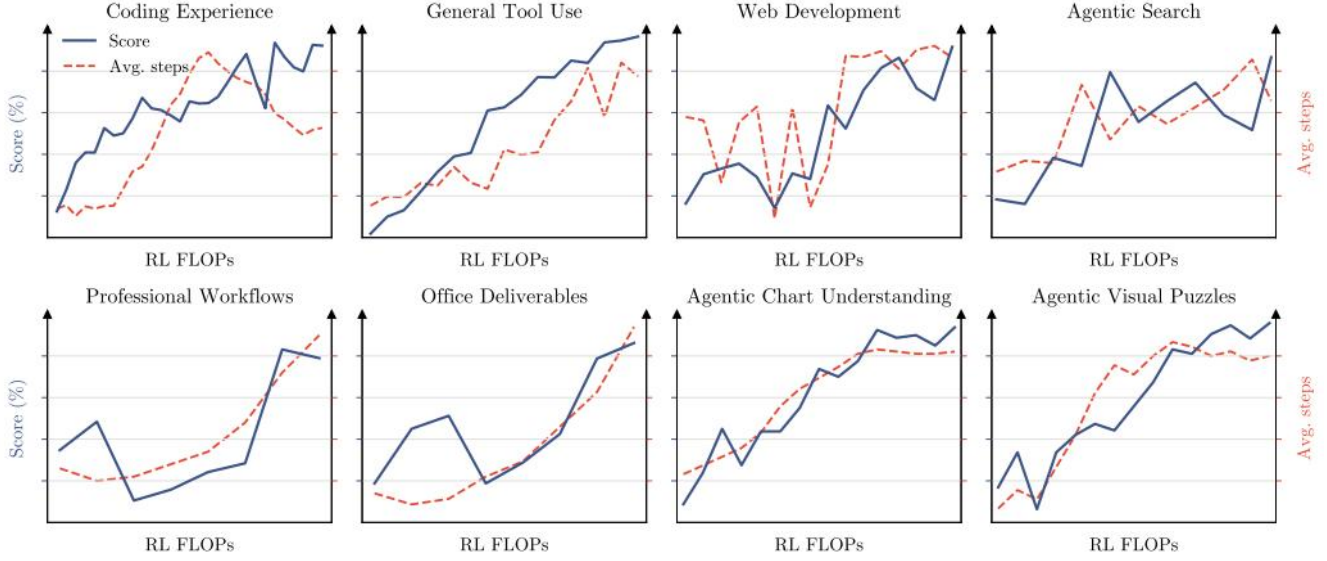


图 8: RL 过程中各种公开和内部评测的得分与平均助手步数。通过扩展 RL FLOPs, 工具调用步数持续增长, 同时模型的整体能力得到全面提升。

算法为缓解在长周期任务中加剧的长尾延迟问题, 我们扩展了同步 RL 框架 [118, 59] 中的部分 rollout 方案。在每次迭代的 rollout 阶段, 我们为 N 个提示各采样 K 个补全, 保持 $N \times K$ 个轨迹的活跃工作负载。生成阶段不会等待所有 rollout 终止, 而是在比例为 $\lambda \in (0, 1)$ 的轨迹完成时 (即 $\lambda N K$ 条) 暂停, 使策略优化能够继续进行而不受执行掉队者的拖累。暂停的 rollout 被入队并优先在下一次迭代开始时恢复, 由我们的沙箱基础设施提供支持 (§ 5.3.2)。一旦某个提示的所有 $\bar{K}$ 个响应完成, 它们立即被分派进行策略优化, 优化过程遵循 Kimi K2.5 [59] 中的算法。在我们的部分 rollout 方案下, 单条长周期轨迹自然地跨越多次迭代, 引入了威胁训练稳定性的数据陈旧性问题。我们的策略优化算法通过逐 token 正则化天然地容忍这种极端的离策略 (off-policy) 状态。通过将策略更新约束在一个局部邻域内, 该正则化使算法能够稳健地处理高度陈旧的数据, 并维持训练稳定性。

推理努力 RL 为在最大化 token 效率的同时微调推理努力, 我们在 RL 中实现了逐问题的预算控制机制 [59]。我们为每个问题 x 关联一个由冷启动模型估计的初始 token 预算 $b_0(x)$, 并将总 token 预算 $\dot{T}(y)$ 超过缩放阈值 $\tau \cdot b_0(x)$ 的轨迹的任务奖励覆盖为 1。对于通用任务, $T(y)$ 衡量思考 token 的数量; 对于智能体任务, $T(y)$ 则计算累积输出 token, 包括推理轨迹和工具调用参数。训练遵循围绕预算乘数 τ 的阶段式课程。我们首先训练一个具有相对较大 τ 的最大预算变体, 同时仍对最大预算设置上限以抑制过度思考。然后将 τ 退火到较小值, 以获得高努力和低努力的专家模型。 τ 的调整在每个领域内根据人工参与指导进行配置。所有推理级别下由各专家产生的轨迹被统一收集, 用于监督微调和多教师在线策略蒸馏。

智能体生成式奖励模型对于不可验证的通用任务, 我们采用智能体生成式奖励模型 (GRM), 保留了 Kimi K2.5 [58, 59] 中基于锦标赛风格、使用二值比较的群体奖励方法。除了增强判断所需的通用智能体能力外, 智能体评判器还必须遵循一个强制协议: (1) 阅读结果、产物或文本输出; (2) 生成评分标准 (rubric); (3) 对照评分标准给每个候选打分; 以及 (4) 将评分标准分配的分数记录在评分板 (scorepad) 中。为缓解针对越来越冗长的输出的奖励黑客 (reward hacking) 行为, 我们应用了与上述推理努力控制类似的基于预算的冗长度控制: 给定从冷启动模型估计的初始冗长度 ℓ_0 和一个乘数 σ , 输出长度超过 $\sigma \cdot \ell_0$ 的候选将自动输掉二值

比较。

4.1.3 多教师在线策略蒸馏

我们采用多教师在线策略蒸馏 (MOPD) 将这些在不同推理努力程度下的领域专用能力整合到统一模型中 [75, 134, 29]。在训练过程中, 对于给定的领域 d 和采样的推理努力级别 $e \in \{\text{low}, \text{high}, \text{max}\}$, 优化由九个专家中对应的教师模型 $\pi_{\text{teacher}}^{(d,e)}$ 引导。给定输入查询 x 和前缀响应 $y_{<t}$, 在教师 $\pi_{\text{teacher}}^{(d,e)}$ 和学生 π_{θ} 之间, 在 y_t 上评估的逐 token OPD 奖励定义为:

$$r_{\text{opd}}^d(y_t | e, x, y_{<t}) = \text{clip} \left(\text{sg} \left(\log \frac{\pi_{\text{teacher}}^{(d,e)}(y_t | x, y_{<t})}{\pi_{\theta}(y_t | e, x, y_{<t})} \right), -R_{\max}, R_{\max} \right), \quad (15)$$

其中 $\text{sg}(\cdot)$ 表示停止梯度算子, $R_{\max} > 0$ 是用于约束极端优势信号的裁剪阈值, 从而稳定 RL 训练。这一密集奖励信号无缝集成到我们的 RL 框架中, 自然地支持基础设施级优化, 例如针对长周期任务的部分 rollout 训练。虽然我们也实验了更细粒度的 top-k 蒸馏目标, 但在我们的设置中, 无论是在收敛速度还是最终性能方面, 均未观察到明显优势。

4.1.4 部署感知的后训练

MXFP4 量化感知后训练为在部署时减少内存占用和服务成本, 我们将 MoE 专家权重——这些权重占据了模型参数内存的主要部分——量化到 MXFP4 [103], 激活值以 MXFP8 计算, 而所有非专家组件 (注意力投影、潜在 MoE 投影、共享专家和 MoE 路由器) 保持更高精度。我们在整个后训练阶段 (覆盖 SFT 和 RL) 进行量化感知训练 (QAT) [49], 使模型适应量化引入的精度损失。在 RL 过程中, rollout 和训练共享相同的量化方案——消除了训练与推理之间的不匹配。

草稿模型微调优化推理效率对于服务复杂的、长周期的智能体模型至关重要。Kimi K3 在预训练时带有一个多 token 预测 (MTP) 层, 其结构与主干块镜像。由于 EAGLE-3 [71] 的草稿模型由结构与 MTP 层匹配的单个解码器层组成, 我们将预训练的 MTP 层微调为 EAGLE-3 风格的草稿模型, 目标模型冻结, 仅更新草稿层及其特征融合投影。遵循 EAGLE-3 的训练时测试协议, 草稿在训练期间展开七步; 除第一步外——最新位置的目标侧特征不可用——草稿从较早步骤消费自己的输出, 镜像了推理时的循环草稿过程。

草稿输入融合了目标模型的低层、中层和高层特征, 分别取自第 1、第 4 和最后一个 AttnRes 块的输出 (S 2.2)。这些特征被拼接并通过一个无偏置矩阵 W_{E3} 投影到隐藏维度, 该矩阵初始化为 $[\mathbf{0} \ \mathbf{0} \ \mathbf{I}]$, 使得融合表示在初始化时与高层特征 h_h ——即 MTP 层预训练时的输入——一致, 并在微调过程中逐渐学会融入低层和中层特征。

推测解码的加速效果由无损推测采样下的逐 token 接受率 $\sum_{x \in \mathcal{V}} \min(p(x), q(x))$ 决定, 其中 p 和 q 分别表示目标模型和草稿模型的下一 token 分布。由于最小化传统的 KL 散度代理目标并不能保证对容量有限的草稿模型最大化该接受率, 我们直接优化基于似然的 LK 损失 [104], 即接受率本身的对数负值,

$$\mathcal{L}_{\text{LK}} = -\log \sum_{x \in \mathcal{V}} \min(p(x), q(x)), \quad (16)$$

其中 p 和 q 在温度 1 下评估，且不包含辅助的真实交叉熵项。草稿微调遵循后训练 QAT 配置 (S4.1.4)，MoE 专家权重采用 MXFP4，其输入激活采用 MXFP8，而非专家模块保持更高精度。

4.2 RL 任务合成与智能体环境

我们的 RL 框架的有效性在很大程度上依赖于丰富、多样且可稳健验证的环境。为支持跨复杂长周期任务的可扩展训练，我们设计了一系列专用的白盒环境和任务合成范式。

4.2.1 统一白盒 RL 环境

使用单一固定的智能体框架 (harness) 进行训练可能导致模型过拟合到特定的工具模式、系统提示、上下文管理机制或交互协议。为解决此问题，我们开发了一个统一白盒 RL 环境，将智能体框架表示为一组可配置、可组合的模块，包括工具接口、系统提示、上下文管理策略、技能、记忆、子智能体和其他组件。通过配置组合这些模块，该环境可以实例化主流框架，如 Kimi Code [56]、Claude Code [15]、Codex [20]、OpenClaw [86] 和 Hermes [44]，以及全新的框架。在 RL 训练过程中，我们为不同的任务组动态构建不同的框架配置，使 Kimi K3 暴露于这些模块的多样化组合，而非任何单一框架的惯例。同样的抽象也方便地支持跨各种任务领域的 RL，为训练更通用的智能体提供了可扩展的基础。

4.2.2 知识图谱引导的任务合成

动机与概览后训练任务的质量和多样性在很大程度上取决于其源材料。由细粒度概念引导的检索能够发掘专业化和未被充分代表的知识，而跨多样化概念的采样则拓宽了领域覆盖范围。为在大规模下同时控制粒度和覆盖范围，我们构建了一个自演化、层次化组织的知识图谱，智能体通过在知识密集型和编码领域的 Web 规模探索持续扩展该图谱。图 9 展示了任务合成流水线。

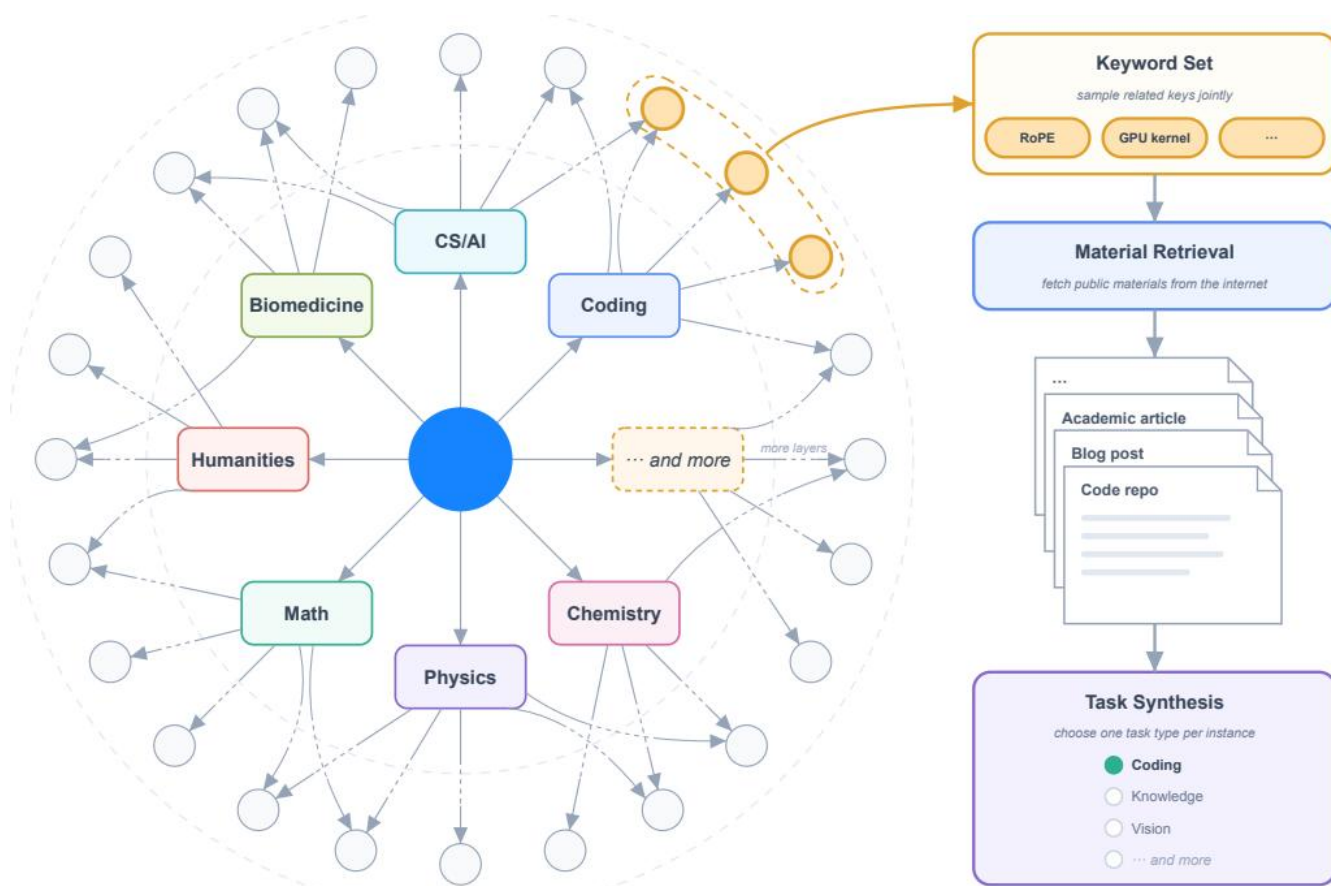


图 9: 知识图谱引导的任务合成概览。层次化组织的知识图谱在多个层级表示概念，范围从广泛领域到细粒度概念。相关节点被采样形成一个关键词集合，引导公开源材料的检索。对于每个合成实例，系统选择一个任务类型，并使用检索到的材料合成相应的任务。

智能体知识图谱构建我们将知识图谱构建为一个有向无环图，通过递归的、智能体驱动的扩展来实现。扩展过程从一组预定义的粗粒度种子节点开始。每个节点被分配一个智能体实例，该实例执行多次 Web 搜索以调研相应的概念。在添加新节点之前，智能体会探索现有图谱以识别等价或相关的概念，在适当的地方复用已有节点，以最小化重复。边始终从较粗粒度的概念指向较细粒度的概念，无论智能体首先发现哪个端点。新添加的节点随后被分配给智能体进行进一步探索。当分配的智能体确定当前概念已经足够原子化时，该分支停止扩展。

材料检索与任务合成为针对跨领域和任务类型的期望分布，系统在不同粒度级别采样节点，可以单独采样或按相关组合采样。从采样节点派生的关键词与来自知识图谱中其祖先节点的上下文信息相结合，以生成 Web 查询。检索到的真实世界材料被组合起来，由合成智能体生成各种任务类型的训练任务。

4.2.3 智能体环境中的可验证问题

我们在智能体环境中训练 Kimi K3 解决可验证问题；代表性示例包括：多步骤复杂信息搜索，其中模型规划研究过程、逐步从 Web 收集证据并产生可验证的答案；专业人士的真实日常工作，如投资银行、数据分析和

法律实务，其中模型将复杂请求分解、在沙箱中操作领域工具，并在数十到数百个步骤中完成交付物；以及针对 STEM 问题、视觉谜题和图表理解的多步骤可验证视觉推理。每条视觉推理轨迹在配备 Python 解释器的隔离沙箱智能体环境中生成：模型迭代编写和执行代码来裁剪、缩放或变换输入图像，执行精确计算或验证中间结果，并在多个交互步骤中接收执行输出——包括生成的图像——作为新的观察。随着模型学会执行更多的图像操作并收集更多的观察，其在复杂视觉推理任务上的表现稳步提升。

4.2.4 内核优化任务

为增强 Kimi K3 的 GPU 内核优化能力，我们构建了一个大规模内核任务套件，涵盖从单算子内核到融合巨型内核的各种任务，其来源包括诸如 Flash Linear Attention [139] 等高质量 GitHub 仓库。该套件覆盖了多种 GPU 编程方法，如 CUDA、Triton、CuTe DSL、Gluon、ThunderKittens [110] 和 TileLang [129]，并涵盖了广泛使用的 GPU 架构和数值格式，包括 BF16、FP8 和 FP4。奖励同时评估正确性和性能：每个内核提供 PyTorch 参考实现，超出预定义数值误差阈值的解决方案获得零奖励。性能则相对于专家实现进行评分：与之匹敌获得 0.5 奖励，接近硬件理论峰值（roofline）则将奖励提高到接近 1。为确保奖励反映真实的优化，我们开发了一个黑客检测系统，惩罚诸如 CUDA 图重放、输入缓存和精度降低等奖励黑客策略，并在 Kimi K3 开发过程中观察到新的黑客策略时持续扩展新的防护措施。

4.2.5 个人助理任务

对于长周期个人助理任务，我们开发了广泛使用应用的真实模拟实现，如 Gmail、Notion、Slack 和 Canvas。这些模拟实现保留了其真实对应物的核心语义，同时支持可复现的大规模交互，无需外部 API 或频率限制。在这些模拟应用的基础上，我们设计了受真实世界专业工作流启发的复杂任务，场景涵盖人力资源、法律服务和金融等领域。在每个任务中，智能体在一个持久的、持续演化的环境中运作，跨越多个模拟天数，并遇到分布在各个应用中的数十个相互依赖的事件。单次 rollout 可能涉及多达数千次工具调用和数百万上下文 token。每个事件带有自己的评估标准，由确定性规则或基于 LLM 的评估器评判。初始工作空间由自主搜索 Web 获取参考材料并将其转换为连贯的、与任务相关环境的智能体构建。我们还扩展了 RL 框架以支持此类活环境（living environments），对复杂事件流及其引发的世界状态转换进行建模。

4.2.6 自主执行任务

我们引入自主执行任务（AET），这是一种通过验证闭环（verify-in-the-loop）优化训练长周期智能体智能的环境范式。每个任务指定初始状态、受约束的目标、基于工具的动作空间、执行预算以及一个独立的验证器。智能体仅看到目标、上下文、约束和验证接口，没有参考轨迹或预定义流程，必须自主执行任务分解、工具选择、规划、错误恢复和终止。奖励基于验证器对最终环境状态的评估，而非智能体自报的完成情况。我们设计了多种类型的验证器以支持多样化的环境，包括黑盒系统复现（图 10）、量化因子发现和税务审计。在每个环境中，智能体迭代提交解决方案、接收验证器反馈并优化策略，训练其假设、执行、分析反馈和适应的通用循环。通过将智能体与验证器隔离、将提供诊断反馈的公开验证器与评估留出场景的隐藏验证器配对，以及在有限提交预算下施加基于惩罚的奖励，来缓解奖励黑客行为。

4.2.7 Web 开发任务

我们构建了一个多样化的、由专家精心策划的 Web 开发任务套件，覆盖典型场景。输入范围从单行场景描述到多段落规格说明；产物涵盖网站、交互式游戏、3D/WebGL 场景、数据可视化、SVG 和全栈应用。每个任

务在容器化沙箱中运行，并在多样化智能体框架而非单一固定框架下 rollout，以促进跨框架泛化。奖励由两部分组成：确定性检查和由内部奖励模型进行的模型评判。确定性检查对应用行为进行功能测试，并对复现参考物的任务评分结构和像素级相似度。当项目构建失败、运行出错或伪造而非实现产物时，奖励清零。模型评判使用其他模型进行源代码检查，或查看并交互输出产物。

Camera Repair Management System 复现

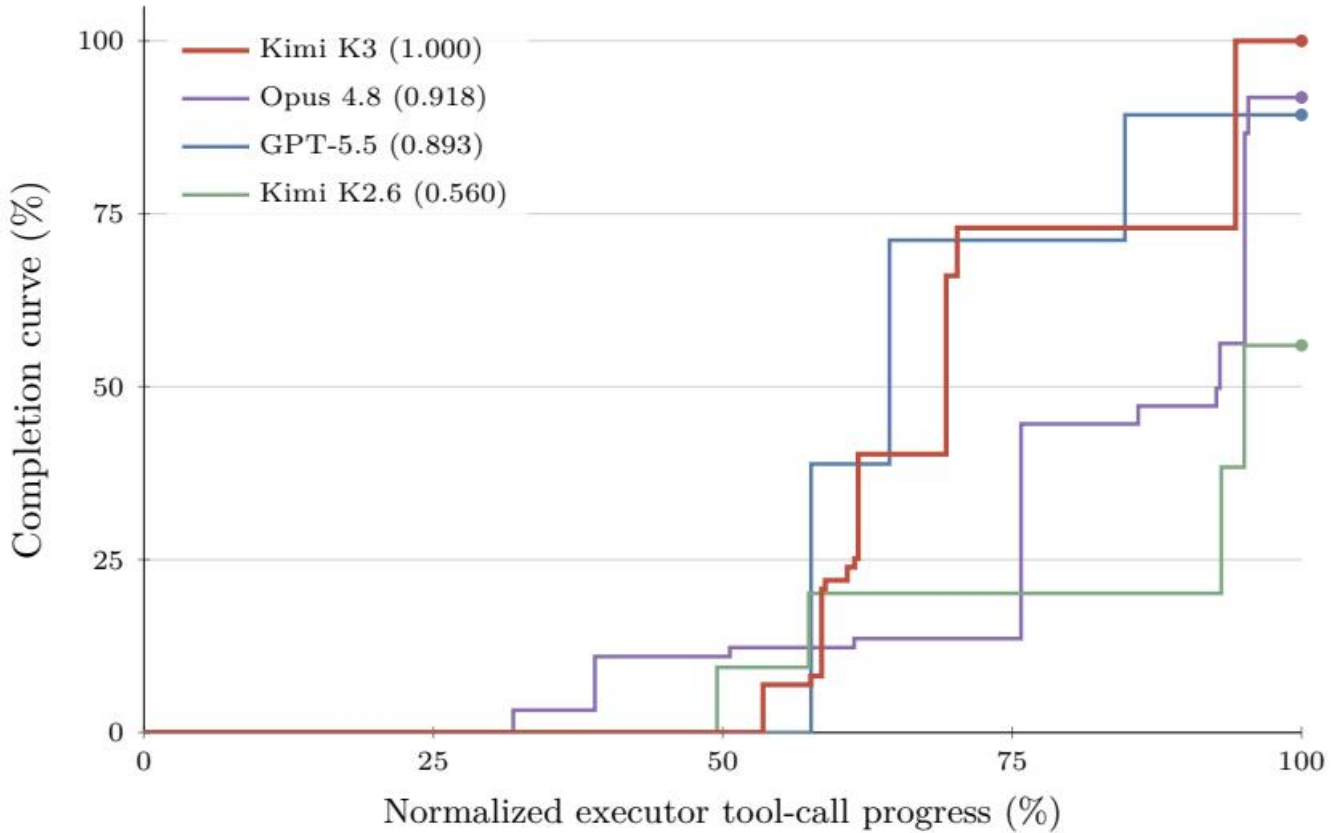


图 10: Camera Repair Management System 上的完成曲线，这是一个黑盒系统复现任务，智能体通过神谕 (oracle) 查询将一个隐藏的 3D 相机维修系统重建为 Web 应用。完成度表示验证器评估的任务进度。

5 基础设施

Kimi K3 在一个模型中同时应对了三类罕见叠加的系统挑战：混合 KDA 注意力、3T 级稀疏多模态训练与推理，以及百万 Token 的智能体工作负载。我们的基础设施围绕这些挑战，在模型全生命周期中进行了协同设计。在架构层面，高性能 KDA Kernel 和上下文并行性使得循环公式在设备内部和跨设备间都能高效执行，覆盖训练与推理。在预训练阶段，均衡的专家执行、降低的内存占用以及通信重叠调度保证了大尺度下的高利用率。在百万 Token 智能体强化学习阶段，分层状态管理和可恢复的沙箱执行在多次迭代间保留了长轨迹。最后，状态感知的 KDA 前缀缓存、专用推理 Kernel 以及缓存与预算感知的调度将这些效率优势转化为可预测的生产服务。

5.1 KDA 的算法-系统协同设计

KDA 将 softmax 注意力中不断增长的 key-value 缓存替换为固定大小的循环状态 $\mathbf{S} \in \mathbb{R}^{d_k \times d_v}$ (§2.1.1)，其串行更新在并行执行中带来挑战，但换来了一个传输和复用成本低廉的固定大小状态。以下设计在两个执行层面分别解决了前者并利用了后者：设备内部的融合 Kernel，以及跨设备的 KDA 上下文并行。

5.1.1 不同场景下的 KDA Kernel

KDA 状态的串行依赖与 GPU 对宽阔、均匀并行的偏好相矛盾，且在不同的执行场景中表现为不同的瓶颈。我们为每种场景设计了专用 Kernel。

用于训练和预填充的分块 Kernel。KDA 的分块形式在每个块内是并行的，但跨块是串行的，因为循环状态必须在块之间传播。如果按原始方式执行，这两个阶段交替进行，SM 在串行传播期间处于空闲状态。因此我们开发了 FlashKDA [14]，一个基于 CUTLASS 的分块 Kernel，将块内计算与跨块状态传播重叠执行。该 Kernel 将工作分解为 token 并行阶段和 head 并行循环，每个阶段独立调度和调优，性能远超 Triton 参考实现。FlashKDA 同时服务于训练和推理预填充，并作为 flash-linear-attention [139] 的后端自动分发。

用于长上下文预填充的设备内上下文并行。张量并行将 head 跨设备划分，但不会缩短循环长度，因此在纯 TP 部署下，预填充一个超长序列时，当每个 rank 仅持有少数几个 head，大部分 SM 处于空闲状态。关键观察是，每个段的循环状态转移可以独立于进入状态进行评估，之后再精确组合。因此，一个自动的 SM 级上下文并行 (CP) 规划器 [142, 139] 将序列按单一 rank 的 SM 进行划分，并行评估各段的转移，然后合并以恢复每个段的精确初始状态。与 §5.1.2 中的跨设备 KCP 不同，这种并行完全是设备内部的，不产生跨设备通信。

KDA 解码带来了与训练和预填充截然不同的挑战。我们将在 §5.4.2 中详细讨论这些挑战。

5.1.2 KDA 上下文并行

上下文并行的通信开销在 softmax 注意力和线性注意力之间有本质区别。Softmax 注意力要求各 rank 交换 key-value 块，其大小随序列长度增长 [72]。而线性注意力则通过固定大小的循环状态 $\tilde{\mathbf{S}} \in \mathbb{R}^{d_k \times d_v}$ 携带前述上下文。先前的上下文并行方法利用普通线性注意力的加性循环特性，在每个 rank 上计算本地 token 从 $\mathbf{S} = \mathbf{0}$ 生成的状态，然后将这些本地状态在前述各 rank 之间求和，以恢复进入状态 [114, 113]。

然而，这种直接求和对于 KDA 是不够的。回顾公式 1，KDA 将其状态更新为 $\mathbf{S}_t = \mathbf{M}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$ ，其中 $\mathbf{M}_t := (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t)$ 。KDA 的 delta 规则在加上当前写入之前，先将 token 相关的矩阵 $\mathbf{M}_t$ 作用于进入状态。因此，局部序列段的效果依赖于进入该段的状态，不能仅由从 $\mathbf{S} = \mathbf{0}$ 计算得到的状态确定。

为了保留这种依赖性，我们提出了 KDA 上下文并行 (KCP)，它将每个段的效果分解为两个可本地计算的量：一个作用于进入状态的累积转移矩阵，以及一个从零开始本地生成的状态。沿用 §2.1.1 的分块记号，我们用 $\mathbf{S}_{[i]}^t$ 表示 rank i 的段内经过 t 个本地 token 后的循环状态，因此 $\mathbf{S}_{[i]}^{T_i}$ 表示离开 rank i、进入 rank i+1 的状态。我们用 $\tilde{\mathbf{S}}_{[i]}^t$ 表示同一循环从 $\mathbf{S} = \mathbf{0}$ 开始时得到的状态。对于进入 P 个上下文并行 rank 中第 (i+1) 个 rank 的任意状态，经过 t 个本地 token 后的状态为：

$$\begin{aligned} \mathbf{M}_{[i+1]}^{t \leftarrow 1} &:= \prod_{r \leftarrow 1}^t \mathbf{M}_r \in \mathbb{R}^{d_k \times d_k}, & \mathbf{S}_{[i+1]}^t &= \tilde{\mathbf{S}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \mathbf{S}_{[i]}^{T_i} \\ &= \tilde{\mathbf{S}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \sum_{j=1}^i \left(\prod_{l \leftarrow j+1}^i \mathbf{M}_{[l]}^{T_l \leftarrow 1} \right) \tilde{\mathbf{S}}_{[j]}^{T_j} \in \mathbb{R}^{d_k \times d_v}. \end{aligned} \quad (17)$$

$$\mathbf{M}_{[i+1]}^{t \leftarrow 1} = \Pi_r \left(\left(\begin{bmatrix} \text{Diag}(\alpha_r) \\ \mathbf{I} - \beta_r \mathbf{k}_r \mathbf{k}_r^\top \end{bmatrix} \times \begin{bmatrix} \text{Diag}(\alpha_r) \\ \mathbf{I} - \beta_r \mathbf{k}_r \mathbf{k}_r^\top \end{bmatrix} \right) \right)$$

$$\mathbf{S}_{[i+1]}^t = \tilde{\mathbf{S}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \sum_j \Pi_l \mathbf{M}_{[l]}^{T_l \leftarrow 1} \tilde{\mathbf{S}}_{[j]}^{T_j}$$

其中 $\mathbf{M}_{[i+1]}^{t1}$ 表示前 t 个本地 token 的累积转移。第一项包含本地 token 生成的状态，而第二项则通过本地 KDA 更新传播来自前序 rank 的上下文。在 $t = T_{i+1}$ 处，两个量 $\mathbf{M}_{[i+1]}^{T_{i+1}+1}$ 和 $\tilde{\mathbf{S}}_{[i+1]}^{T_{i+1}+1}$ 都可以仅使用本地 token 计算，在 $\mathbf{S}_{[i]}^{T_i}$ 可用之前完成，它们是每个 rank 与其他 rank 交换的片段。

公式 17 中的求和表明，每个状态完全由本地计算的片段组成。这些 rank 级更新满足结合律，因此每个 rank 的进入状态可以通过前缀扫描恢复 [77]。每个 rank 首先本地计算 $\mathbf{M}_{[i]}^{T_i+1}$ 和 $\tilde{\mathbf{S}}_{[i]}^{T_i+1}$ ，然后通过一次 all-gather 交换这两个张量 [139].²。在 all-gather 之后，rank $i+1$ 通过按顺序处理同一文档的前序片段，从 $\mathbf{S} = \mathbf{0}$ 开始，在每个片段处应用 $\mathbf{S} \leftarrow \mathbf{M}_{[j]}^{T_j} \mathbf{S} + \tilde{\mathbf{S}}_{[j]}^{T_j}$ ，从而重建 $\tilde{\mathbf{S}}_{[i]}^{T_i}$ 。因此，KCP 仅需固定大小的 all-gather 用于循环状态同步，并实现线性计算扩展。

5.2 3T 级预训练基础设施

Kimi K3 预训练结合了带有虚拟阶段 (VP) 的流水线并行 (PP) [48, 81]、专家并行 (EP) [66]、ZeRO-1 数据并行 [100]、流水线 ZeRO-2 梯度分片 [145] 以及上下文并行 (CP, §5.1.2) [50]。

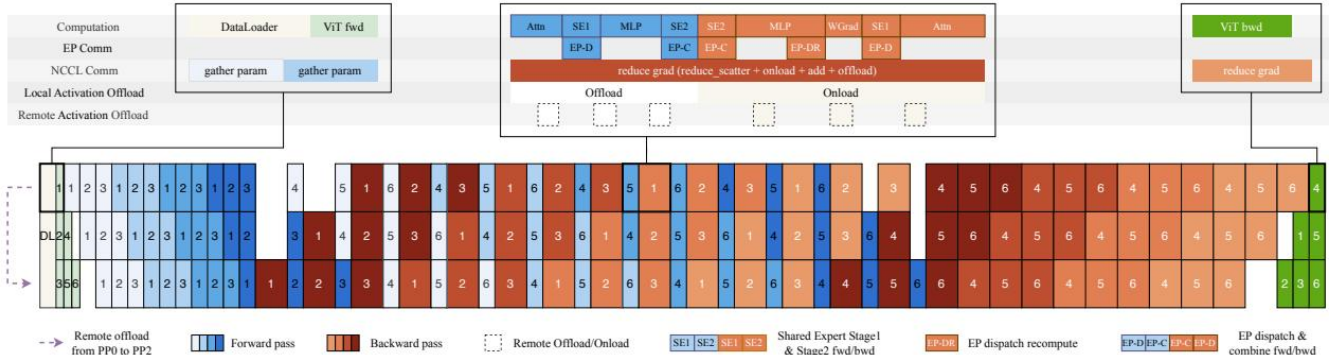


图 11: 计算、通信与卸载在不同 PP 阶段中的重叠。

MoE 层采用跨 EP rank 复制的共享专家，专家分发和合并的 all-to-all 通信与计算重叠以隐藏其延迟。

3T 级的原生多模态预训练面临三个关键问题：(i) token 负载在 EP rank 之间不均衡；(ii) 激活值、梯度和优化器状态超出内存预算；(iii) 视觉编码器高度可变地计算暴露在关键路径上。以下小节依次解决这些问题：完美均衡的专家并行 MoE 训练 (§5.2.1)、内存高效训练 (§5.2.2) 以及多模态编码器优化 (§5.2.3)。图 11 展示了由此产生的执行调度。

5.2.1 完美均衡的专家并行 MoE 训练

在传统 EP 方案中，token 负载在 rank 之间不均衡。由此产生的计算不均衡降低了训练吞吐量，而路由专家激活值的动态变化形状则导致严重的内存碎片。因此我们提出了 MoonEP3，一种通过动态冗余专家实现完美

负载均衡的 EP 方案。MoonEP 保留了 DeepEP [147] 等传统方案的总体计算流程，同时额外引入了冗余专家的在线规划与迁移。在前向传播中，我们从当前 micro-batch 和层的路由器输出中规划冗余专家，并在路由专家计算之前预取它们。在后向传播中，我们将它们的梯度暂存到一个本地 reduce 缓冲区中，计算完成后将其 reduce 回其归属 rank 的梯度缓冲区。

有界冗余专家下的完美均衡。MoonEP 要求每个 rank 恰好接收 $S \times K$ 个 token，其中 S 是序列长度， K 是每个 token 选择的专家数，从而所有 rank 执行相同量的计算。关键问题是多少冗余专家足以保证这种均衡。设 E 为专家数， R 为 EP 大小。我们证明，每个 rank 至多 E/R 个冗余专家总是存在一个均衡方案，且此上界本质上是紧的 ($\leq E$)。因此，每个 rank 预留 E/R 个冗余专家槽位即可保证规划总是有可行解，训练不会中断。相比之下，之前的工作如 ECHO [137] 和 UltraEP [132] 预设冗余专家数量或施加每个 rank 的 token 上限。当上限内不存在可行方案时，训练被迫停止，而上限本身需要手动调优，却仍残留不均衡。

在线规划。在每个训练步计算精确最优解的代价过高。因此，我们通过整数线性规划 (ILP) 为代表性情况离线计算精确解作为参考，并设计了一个 GPU 规划 Kernel，它近似最优、开销可忽略，且始终遵守 E/R 上界。

零拷贝通信。完美均衡也简化了通信路径。我们实现了一个融合的 permute/unpermute 算子，其中规划 Kernel 预先计算每个 token 的目标位置，因此 token 被直接发送到远程 rank 上按专家分组的位置，通信缓冲区的视图直接返回给计算，消除了中间拷贝。在最坏的不均衡情况下，支持同样零拷贝数据路径的 DeepEP 需要一个大小为 $S \times K \times R$ 的通信缓冲区，而 MoonEP 由于完美均衡仅需一个固定的 $S \times K$ 缓冲区。

静态形状下的免同步执行。在传统 MoE 实现中，每个专家的 token 数随训练步和层而变化，主机必须在每一层与设备同步以获取实际计算形状，才能启动专家计算，这导致管道在层间停顿。在完美均衡下，每个 rank 恰好接收 $S \times K$ 个 token，所有层的计算形状静态可知。这消除了每层 MoE 的主机同步，并缓解了主机端的 Kernel 启动开销。

Expert-GEMM 调度与重叠。即使整体负载在 rank 间完美均衡，每个 rank 内部的各专家 token 数仍不均衡，固定顺序、对工作负载无感知的调度会将这种不均衡转化为 SM 工作器之间不均衡的 makespan。因此，我们采用工作负载感知的调度器来调度路由专家 GEMM，该调度器在启动前根据当前 token 分布自适应调整参数，并在执行期间保持固定。一个轻量级启发式方法使用硬件指标的分析成本模型来选择这些参数，关键系数通过离线自动调优进行标定。对于共享专家，我们将其 GEMM 分发到单独的流上，以便与其他 Kernel 重叠。

5.2.2 内存高效训练

统一激活值管理器。我们设计了一个统一的激活值存储抽象，其中为后向传播保存的每个张量与一个可插拔的存储后端关联。重计算、量化和卸载/远程卸载在此抽象下仅是存储策略，可在张量粒度上自由组合；策略通过张量上的轻量级注解声明，与模型代码完全解耦。重计算以函数粒度执行，支持跨层重计算。在我们的实现中，所有 GPU 内存都在主计算流上分配，并在单一内存池中管理，避免了多流碎片化和主机端开销；激活值按层粒度预取回来并与计算重叠，引入的额外开销可忽略不计。在 Kimi K3 中，大部分激活值使用块级 FP8 量化 [58, 30] 结合卸载/远程卸载，逐元素算子则配置为重计算。

内存高效 MoE。在原生的 MoE 实现中，置换后 probs 的梯度计算依赖于前向输出 output。受 SonicMoE [41] 启发，我们通过数学变换将该梯度重写为仅依赖于中间激活 act_output 和上游梯度 doutoutput 的形式，消除了对 output 的后向依赖，代价是增加一次额外的轻量级逐元素计算。此外，在分组 GEMM 的前向传播中，我们仅保存 dispatch 操作的输入；在后向传播期间，分组 GEMM 的输入通过重新计算 dispatch 来恢复。如图

11 所示，该重计算引入的通信可以与分组 GEMM 后向计算的一部分重叠，从而以可忽略的代价消除了这部分激活值存储。

内存高效 Attention Residual。对于 attention residual，我们设计了一个基于 Block AttnRes 的配套优化。块表示在边界层生成一次，并由所有后续层共享，直接驻留在 GPU 上。AttnRes 计算完全通过 checkpointing 包裹，因此每层为后向传播保存的激活值与标准残差架构相同。对于流水线并行，我们采用基于缓存的流水线通信 [57]，其中只有新生成的块在阶段之间增量传输，并在 micro-batch 完成后立即释放，达到内存占用的理论下界。

跨 PP rank 的激活值均衡。在交错 1F1B 流水线并行下，由于流水线预热，激活值在 PP rank 之间分布不均匀，驻留激活值的数量随 PP rank 增加而减少。为避免显存不足 (OOM) 错误，我们使用 Mooncake Transfer Engine [96] 将激活值远程卸载到其他 PP rank 的内存中，实现跨 PP rank 的激活值内存均衡。

流水线 ZeRO-2 梯度分片与卸载。除激活值外，我们使用流水线 ZeRO-2 梯度分片 [145] 在数据并行 (DP) rank 之间分片梯度。此外，我们将分片后的梯度存储在 CPU 内存中，以降低 GPU 内存峰值使用量，同时在 GPU 上保留双梯度缓冲区。梯度在 DP rank 之间 reduce 进入双梯度缓冲区后，被累积到 CPU 分片中。

基于 P2P 的 Muon 正交化。分布式优化器将参数在 DP rank 之间均匀分片，而 Muon 中的 Newton-Schulz 正交化需要完整的参数矩阵，因此在每次更新前需要一个通信步骤来收集完整参数。原始方法在每个 rank 上对整个参数缓冲区执行 all-gather [73]，这不仅产生了巨大的内存占用，更使通信成为大规模下的主要瓶颈。取而代之，每个 rank 通过与其对应归属 rank 的点对点 (P2P) 通信，仅获取其本地拥有参数的各个分片，消除了完整参数缓冲区，同时降低了内存使用和通信量。通信和计算进一步以模型块缓冲区的粒度流水线化，隐藏了通信开销。

5.2.3 多模态编码器优化

多模态编码器中的动态 CP。在长上下文多模态训练中，大尺寸图像和长视频显著增加了视觉编码器的计算时间，并导致设备间严重的负载不均衡。为解决此问题，我们将上下文并行扩展到这些大样本上。单张大图像沿 patch 维度跨多个设备划分，注意力通过跨 CP rank 收集 key-value 对 (gather-KV) 来计算。此外，我们将每个 CP 组划分为若干子 CP 组，并以负载均衡的方式在它们之间分发多张大图像，防止通信比例随规模增长。这同时降低了大视觉样本的编码器延迟和跨设备负载不均衡，使得剩余的编码器计算能够隐藏在流水线气泡中。

PP 气泡中的编码器计算。在 Kimi K2.5 中，我们引入了分离编码器进程 (DEP) [59]，将 ViT 和文本训练拆分为独立阶段，并在 PP 阶段之间平衡视觉的前向和后向传播。我们观察到，在交错 1F1B 流水线调度下，第一批 PP micro-batch 的文本前向传播全部在最开始调度，而最后一批 PP micro-batch 的文本后向传播在最末尾才完成。因此我们进一步分解 ViT 计算。第一批 PP micro-batch 的 ViT 前向传播在前期同步执行，其余前向传播被调度到流水线气泡中，后向传播也类似处理。结果，大部分 ViT 计算被隐藏在流水线气泡中，基本消除了视觉编码器的有效开销。

5.3 百万 Token 智能体强化学习基础设施

在有限的计算预算下，为 Kimi K3 这样的大模型将智能体强化学习扩展到百万 Token 上下文，使得资源效率成为首要目标。这推动了两项互补的工作：1) 高效的训练和 rollout，包括 KV 缓存管理、请求调度和训练状

态放置；2）用于长时交互的高性能可恢复沙箱。

5.3.1 长上下文强化学习基础设施

我们采用共置 RL 训练 [58] 将每个百万 Token 上下文的 Kimi K3 RL 实验控制在数百张 GPU 以内，并使用 partial rollout [118] 来降低超长轨迹的尾部延迟。这一设计实现了良好的硬件利用率，但也引入了 rollout KV 缓存（需要为下一次迭代保留）与训练所需内存之间的占用竞争。这一挑战在长上下文 RL 中更为严峻。

外部 KV 缓存池。在百万 Token 上下文的多步 rollout 中，前缀 KV 缓存未命中的代价极高。Partial rollout 在每次迭代开始时加剧了这一问题，因为来自上一迭代的许多未完成的长预填充请求同时到达。推测解码进一步加快了在相对固定的工具调用间隔内的请求周转，增加了前缀块的更替。这些问题可能触发抢占并降低缓存命中率，这对长上下文 RL 至关重要。

因此，我们通过写回设计将前缀保留与 GPU 驻留解耦。活跃的解码块保持在 GPU KV 缓存中，而可复用的空闲前缀仅在从 GPU 淘汰时才写回 CPU DRAM 中的外部 KV 缓存池，并在下次复用前预取回来。KDA 状态与相应的 MLA KV 缓存块一起卸载和预取，保持二者生命周期一致。与写通策略相比，此策略仅对离开活跃解码路径的前缀产生 CPU DRAM 使用和传输带宽，避免了仍在 GPU 上驻留且活跃的块的冗余 CPU 拷贝。

为了给外部缓存池提供足够的 DRAM，我们在训练迭代完成后将训练状态（模型权重和优化器状态）卸载到 NVMe。在 rollout 迭代之后，释放缓存池以避免与训练工作负载竞争。

Rollout 自动节流调度器。在多步 rollout 中，上下文随轨迹推进而逐步增长，使得基于全轨迹平均长度的固定并发度既难以估计，又在早期过于保守。反之，将并发度设置过高则会在后期产生 KV 缓存压力，并可能触发抢占。因此，我们在 LLM 请求调度层设计了一种自动节流机制，利用活跃请求数、排队请求数和 KV 缓存利用率等运行时信号，动态控制发送到推理引擎的请求数量。这使得早期 rollout 保持良好利用率，同时随着 KV 缓存压力上升降低并发度，无需手动调优即可避免欠饱和与过载。

非策略模型前向传播的梯度缓冲区复用。RL 损失计算通常需要仅前向的非策略模型，如参考模型，其权重过大无法常驻 GPU。我们将这些权重保留在 CPU 内存中，仅在需要时才实例化，将其参数张量托管在策略模型的 FP32 梯度缓冲区存储上。这复用了现有 GPU 内存，无需额外分配或引入碎片，并且由于真实梯度后续计算时会覆盖这些缓冲区，因此是安全的。

配合 ZeRO-2 梯度分片与卸载 (§ 5.2.2)，在 Kimi K3 RL 训练中，每个 GPU 仅为两个 VPP chunk 保留梯度缓冲区。我们将参考权重逐块流式加载到这些槽位中：一个槽位用于当前前向计算，另一个在预取下一个 chunk，从而在不增加 GPU 内存的情况下隐藏拷贝开销。

5.3.2 沙箱基础设施

我们部署了多个沙箱运行时来支持 Kimi K3 后训练和评估的多样化需求，包括传统的基于容器的运行时、GPU 沙箱运行时，以及最重要的一个——一种新的基于 microVM 的沙箱运行时 AgentENV。

AgentENV4 是与我们的合作伙伴联合开发的，专门为智能体 AI 工作负载设计的沙箱系统。它围绕三个核心设计目标构建：

- 高保真隔离沙箱运行时。随着智能体能力增强和任务难度提升，它们倾向于更激进地探索，甚至可能尝试 reward hacking。一方面，这带来了独特的安全挑战：在我们早期使用传统基于容器的沙箱运行时的实验中，观察到多次由非预期的智能体操作引发的内核 panic 和死锁。另一方面，我们希望尽可能允许探索，以免限制智能体的能力，而复杂任务需要一个接近真实环境的沙箱——例如，智能体应能挂载磁盘、运行容器，甚至随意启动虚拟机。通过使用 Firecracker [3] 运行隔离的 microVM，AgentENV 提供了基于容器的运行时无法匹敌的隔离度和保真度。

- 面向智能体 RL 的灵活沙箱生命周期。在底层，AgentENV 支持沙箱状态的增量 checkpoint 和恢复，其中 checkpoint 时仅保存自上次 checkpoint 以来被修改的内存页，checkpoint 和恢复延迟分别低至 133 ms 和 49 ms。在此基础上，AgentENV 提供了三个有助于提高智能体 RL 效率的高层操作。(a) 暂停与恢复：暂停的沙箱不消耗内存或 CPU 资源；因此沙箱可以在智能体等待模型推理结果时暂停，这期间可能占沙箱生命周期的 98%。(b) Fork：fork 从原始沙箱的精确状态创建一个新沙箱，同时保持原始沙箱继续运行，这对于无副作用的奖励评判非常有用。(c) 快照：沙箱快照可以按固定间隔保存，用于错误恢复。

- 高效率与高密度。在我们的工作负载中，数万个沙箱（每个配备一组独特的镜像）可能需要在几秒钟内创建。我们采用 OverlayBD [68] 作为镜像格式，配合定制的 ublk 驱动实现、存储层共享和 P2P 传输，实现了大规模下的亚秒级启动延迟。我们通过写时复制内存和页面缓存优化进一步降低内存使用，在实际工作负载中取得高达 6.5 倍的内存超分比。

在 Kimi K3 的训练和评估过程中，共创建了 51,219,741 个沙箱，涉及 1,505,678 个镜像。

5.4 推理与在线服务

为 Kimi K3 提供服务从生产侧暴露出同样的挑战：混合 KDA-MLA 架构维护两种本质不同的缓存，必须在百万 Token 上下文中联合管理；其新模块和高度稀疏的专家要求为每个模块定制 Kernel；而生产流量混合了单请求成本跨越三个数量级的请求。以下设计在三个层面应对这些挑战。在引擎层面，KDA 感知的前缀缓存将固定大小的循环状态打包到与 MLA KV 缓存相同的分页池中，并使长前缀可在请求间复用。在设备层面，为 KDA 解码、Block AttnRes 和稀疏潜在 MoE 定制的 Kernel 最小化每 token 延迟和内存流量。在集群层面，缓存感知的亲和调度和基于预算的准入控制将这些效率转化为可预测的服务。

5.4.1 KDA 感知的前缀缓存管理

Kimi K3 中的混合架构使前缀缓存复杂化：KDA 循环状态和 MLA KV 缓存在大小和生命周期上有本质区别，但缓存的前缀只有在二者能在同一边界上一起恢复时才可复用。因此，我们设计了一个 KDA 感知的前缀缓存，将两种缓存类型联合管理——从统一的分页布局到细粒度前缀复用，再到并发调度下的一致性——使得百万 Token 前缀的保留成本低廉并可在请求间复用。

混合 KDA-MLA 注意力的统一缓存布局。每个 Kimi K3 块由三层 KDA 和一层 Gated MLA 组成，其缓存在本质上有区别。MLA KV 缓存随序列长度增长，并按 token 分页；而 KDA 循环状态大小固定，每个请求只有一个拷贝。为每种缓存维护独立的管理器会重复分配、淘汰和传输逻辑。因此，我们将 KDA 状态打包到与 MLA KV 相同的分页块池中，将页面统一为相同的字节大小，使得两种页面类型共享同一套分配、引用计数和淘汰实现。在页面内部，所有 head 的状态按 head 依次连续存储，使得每个 head 的字节流自包含，并作为跨节点传输的最小单元。在预填充/解码分离部署下，当预填充和解码节点采用不同的 TP 度时，在传输

路径上执行重新布局，GPU 端无需重新排布。这种非对称性在开发过程中证明很有用：任何类型混淆的访问会产生垃圾数据而非看似合理的数据——一种对池化布局的零开销健全性检查。

KDA 前缀缓存优化。基于块哈希的前缀缓存以一个物理块为粒度复用 KV 缓存：只有完整块才被哈希，因此只有块对齐的前缀才可复用。

这种耦合在 Kimi K3 中失效了。块哈希匹配要求所有层共享一个块大小，且前缀命中仅在命中边界处的 KDA 状态已被持久化时才可复用。KDA 层每个序列维护一个大的循环状态而非逐 token 条目，因此状态快照仅在稀疏的边界处可行；共享块大小因此被迫设为 1024–6144 个 token——并且，由于哈希绑定到存储块，哈希粒度也被迫绑定，尽管 MLA 的逐 token 条目本身就允许更细的块。在如此粗的粒度下，前缀缓存几乎无用：短于一个块的请求永远无法被复用，分块预填充在跨越完整的块边界之前不能导出任何可缓存的前缀。

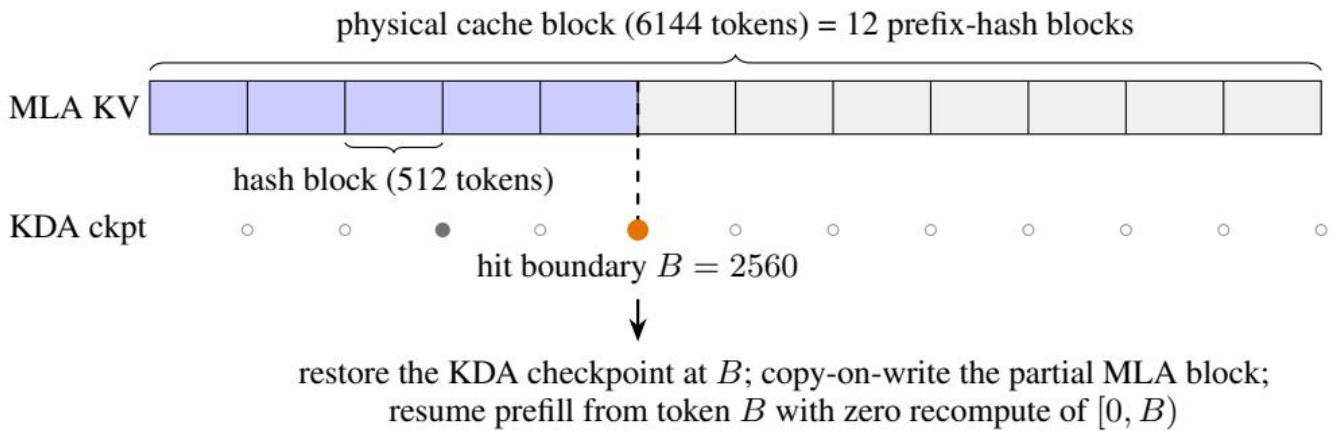


图 12：物理缓存块内的细粒度前缀缓存。一个 6144 token 的物理块包含十二个 512 token 的哈希块，缓存的 MLA 块以蓝色显示，空块以浅灰色显示。下方的标记显示每个哈希边界处的 KDA checkpoint 状态。空心圆圈 () 表示无存储 checkpoint 的边界，灰色圆点 () 表示持久化的 KDA checkpoint，橙色圆点 () 标记在 $B = 2560$ 处的 checkpoint 命中。持久化的 checkpoint 是稀疏的，通常与对话轮次边界一致。该请求复用五个 MLA 哈希块和 B 处的 KDA checkpoint，然后恢复预填充，无需重新计算 $[0, B]$ 。

因此，我们将两种粒度解耦。前缀哈希以细粒度哈希块（例如 512 token）在 MLA 页面内部运行，而物理块保持为粗粒度的分配单元。KDA 的对齐方向相反：循环状态的 checkpoint 仅在 MLA 哈希端点的（一个稀疏子集）处保存——这是查找唯一能引用的位置。

在预填充期间，部分填充的 MLA 页面以其最后完整哈希块的链式哈希注册到前缀缓存索引中，其中每个哈希覆盖其之前的所有哈希块，因此匹配一个端点即验证了到该点为止的整个前缀；注册端点随页面填充而推进。同时，在每次前向传播后，KDA Kernel 在处理的最后一个哈希对齐位处持久化循环状态。Checkpoint 较大，因此随请求推进而被取代的中间 checkpoint 会被回收，而对话轮次边界处的 checkpoint 则保留用于跨请求复用。缓存的 checkpoint 是只读快照：命中时通过在下次前向传播之前拷贝到请求的私有运行状态中来恢复状态，新的 checkpoint 写入新的槽位，因此对其他请求可见的 checkpoint 永远不会被就地修改。

查找分两个阶段进行（图 12）。MLA 阶段通过链式哈希匹配完整物理块，在第一个缺失块处，回退到其内部的哈希端点，使部分填充的页面仍可命中。然后，KDA 阶段要求候选边界处在每个 KDA 缓存组都有一个 checkpoint，每个缓存组维护一个独立的循环状态。命中是同时满足两个阶段的最长边界——始终是哈希块的

倍数,且不要求是物理块的倍数。在图 12 中,一个前 2800 token 与缓存前缀匹配的请求在 $\dot{B} = 2560 = 5 \times 512$ 处命中,深入到一个 6144 token 物理块内部,并从 token B 处恢复预填充,而无需重新计算 $[0, B)$ 。

并发调度下的一致性。其余设计要点均由共享部分填充块的具体失败模式决定:在这种场景下,一个命中块既是共享缓存条目,又是私有请求的增长点;MLA 和 KDA 缓存组必须在每个命中边界上达成一致。首先,所有缓存组从一个共享的空闲列表中获取块,因此为一个组分配私有拷贝可能淘汰另一个组刚刚命中的块;因此,每个命中块在分配任何内容之前,都会在所有组之间被钉住。其次,向私有块的拷贝在当前调度步内紧接前向传播之前在 GPU 上执行,因此当前调度步内分配或注册的块仍可能将前一个拥有者的字节传递给读者;此类块在拷贝落地之前被排除在匹配之外。第三,一个 checkpoint 只有在每个 KDA 组都存在时才能恢复请求,因此淘汰一个组的 checkpoint 原子性地使其兄弟 checkpoint 无效——一个 checkpoint 要么在所有组都可命中,要么在任何一个组都不可命中。通过这些机制,每个已注册状态始终精确对应其声明的 token 前缀,混合 KDA-MLA 模型的前缀缓存达到了与全注意力模型同样的通用性:任何共享前缀在任何 512 token 边界处都可复用,与请求长度、分块或调度交叠无关。

5.4.2 高性能 Kernel

Kimi K3 引入了几个新的架构模块:KDA (§2.1.1)、Block AttnRes (§2.2) 和 Stable LatentMoE (§2.3)。我们为每个模块优化了 Kernel 实现。

KDA。与 KDA 预填充 (§5.1) 相比,KDA 解码呈现一组不同的挑战:主要瓶颈从利用并行性转变为高效管理不断演化的循环状态,该状态在每个解码步都被就地更新。这种就地更新在基于 MTP 的推测解码中变得棘手:如果验证拒绝了一个子集的草稿 token,状态已经推进到最后一个被接受 token 之后,无法简单回滚。为每个草稿位维护状态快照可以实现回滚,但也会倍增状态流量——这一代价在在线服务典型的较大批次下占主导地位。

然而,任何被接受的草稿前缀之后的状态完全由草稿 token 的投影输入决定,这些输入远小于状态本身。因此,我们仅缓存这些投影输入,在芯片上重建被接受 token 的状态,并写回已验证 token 和 bonus token 的状态,这一设计与同期工作 ReplaySSM [25] 独立提出。重放的 token、bonus token 和下一个草稿窗口在单个融合 Kernel 内共享一个循环循环,覆盖短卷积、输入归一化、门控、KDA 循环和输出归一化。验证延迟随验证 token 数呈亚线性增长,并保持在状态缓存基线的延迟之下。由于投影缓存从不离开解码阶段,前缀缓存和预填充-解码分离在非推测推理中操作相同的负载。

Block AttnRes。Block AttnRes [57] 遵循两阶段调度:批量跨块传播每块读取一次缓存的块表示,之后每层通过 online-softmax 合并 [79] 融入块内部分和。在预填充和解码中,内存访问占这些 Kernel 成本的很大一部分,因此我们两个阶段的优化都主要集中在内存效率上。

对于预填充,在每个张量并行 (TP) rank 上实例化块表示会产生大量冗余的内存消耗。因此,我们对激活值采用序列并行 (SP):将 TP all-reduce 分解为 reduce-scatter 和 all-gather,块内 Kernel 插入在两次集合通信之间,对序列分片的隐藏状态进行操作,使得每个 token 的块表示恰好在一个 rank 上实例化。这消除了额外的内存消耗,并降低了预填充期间 Block AttnRes 的 I/O 开销。

对于解码,我们在旁路流上启动跨块 Kernel,使其与主流上的独立计算重叠。块内 Kernel 则通过融合来精简:将 AttnRes 输出与其部分和更新的合并,以及后续的 RMSNorm,融合到前序的 TP all-reduce 中,从而消除了块内阶段的专用 Kernel。这些优化共同隐藏了跨块传播的延迟,并降低了块内阶段的内存流量。

Stable LatentMoE。Stable LatentMoE 同时增加了专家的总数和每个 token 激活的专家数。专家空间和每 token 专家数的增长提高了调度和协调开销，使得传统 MoE Kernel 难以维持高硬件利用率。这些挑战推动了对该模块的专用 Kernel 优化。

为缓解潜在 GEMM 的开销，我们采用了三项优化。首先，将潜在下投影与 MoE 路由器融合为单个 GEMM。其次，跨 rank 分片潜在权重矩阵，并使用 multimmem store 指令将输出 all-gather 融合到 GEMM epilogue 中。最后，将由此产生的通信与其他算子（如共享专家计算）重叠。这些优化共同消除了冗余权重流量和重复计算，同时将通信延迟隐藏在计算之后。

对于路由专家，在小批次下，分组 GEMM 退化为对权重矩阵的内存受限流式访问——对于这种场景，传统的以 tile 为中心的 Kernel 由于其面向计算的设计和预处理开销而不适用。我们改为基于 WarpDecode [12] 的以 token 为中心的设计构建 MoE 解码 Kernel，其中每个 warp 负责一个输出神经元，直接从内存流式读取相关权重。为进一步提高并行度，我们将每个 warp 细分为更细粒度的 lane 组，每组处理一个不相交的专家子集，随后进行 warp 级的部分结果归约。此外，权重布局通过一次性预处理代价进行离线置换，大幅降低了运行时的反量化开销。

5.4.3 集群级调度

在单个推理实例之上，挑战从单请求效率转变为可预测性：一次前缀缓存未命中的代价比命中高出数个数量级，而百万 Token 请求的突发流量可能饿死短请求。我们提出两项集群级调度策略来应对：缓存感知的亲和调度将每个会话路由到持有其前缀缓存的集群，同时限制集群故障的代价；基于预算的准入控制为每个请求类分配各自的资源预算，使得突发的长上下文流量不会降低系统整体的 SLO。

缓存感知的亲和调度。在百万 Token 上下文中，典型的编码输入携带 400K token 的前缀，但只需要 4K token 的预填充增量，因此一次前缀缓存命中避免了重新预填充整个前缀，代价比未命中低数个数量级。因此，我们将每个请求路由到持有其前缀缓存的集群，因为将缓存转移到另一个集群需要通过远比集群内部网络慢的跨集群链路。然而，这种缓存感知的亲和性将每个会话绑定到单个集群，该集群的故障会中断所有绑定到它的会话。因此，一致性哈希将每个会话钉在两个集群上：一个主集群为其提供服务，一个预先分配的备用集群在主集群故障时接管。备用集群不持有该会话的任何前缀缓存，必须在故障切换时重新预填充。由于一致性哈希将不同会话的备用分配均匀分布在整个集群中，重新预填充的工作被分摊到多个集群而非集中在一个集群上。因此在常见情况下保留了缓存局部性，同时任何单集群故障的影响保持受限。

基于预算的准入控制。生产流量混合了 2K token 以下的短请求与高达百万 Token 的超长请求，因此单请求成本跨越约三个数量级，任意固定数量的请求所产生的总负载高度不可预测。基于“平均请求”的容量规划、排队模型和速率限制配额在这种方差下全部失效。在一个典型故障模式中，一波长上下文请求突发流量耗尽可用计算资源，随后到达的短请求无法被及时调度，导致所有流量的首 token 延迟 (TTFT) 恶化。因此，我们采用基于预算的准入控制，为不同的请求类分配独立的资源预算，使得突发的长上下文流量最多消耗其自身的容量份额，而不会降低其他请求类所体验的系统级 SLO。

6 评测

6.1 主要结果

6.1.1 基准测试

我们在四个核心能力维度上对 Kimi K3 进行全面评测：

- 推理与知识：GPQA Diamond [101]、CritPt [8]、AA-LCR [9] 以及 Humanity’s Last Exam (HLE-Full, 含/不含工具) [93]。
- 编程：DeepSWE [31]、ProgramBench [95]、Terminal-Bench 2.1 [78]、FrontierSWE [35]、SWE-Marathon [117]、PostTrainBench [94]、MLS-Bench-Lite [76] 以及 SciCode [121, 8]。
- 智能体：BrowseComp [131]、DeepSearchQA [126]、ResearchRubrics [106]、Toolathlon-Verified [69]、MCPMark-Verified [133]、MCP-Atlas [11]、AutomationBench [108]、JobBench [70]、GDPval-AA v2 [90]、AA-Briefcase [8, 2]、Agents’ Last Exam (ALE) [4, 115]、APEX-Agents [127]、OfficeQA Pro [87]、SpreadsheetBench 2 [150]、OSWorld-Verified [136] 和 OSWorld 2.0 [143]、SaaS-Bench [109]、 τ 3-Banking [1, 8]、Harvey Lab-AA [8, 42]、CorpFin v2 [21]、Finance Agent v2 [34] 以及 Legal Research Bench [65]。
- 视觉：WorldVQA [149]、OmniDocBench [88]、PerceptionBench [62]、Video-MME [36]、MMVU [148] 以及带 Python 工具的 BabyVision [13]、MMMU-Pro [144]、CharXiv (RQ) [130]、Math-Vision [128] 和 ZeroBenchmain [102]，各含/不含 Python 工具增强。

6.1.2 基线模型

我们将 Kimi K3 与最先进的闭源和开源模型进行对比。闭源模型方面，我们对比 Claude Fable 5 [16]、GPT-5.6 Sol [39]、Claude Opus 4.8 [17] 和 GPT-5.5 [38]。Claude Fable 5 的结果包含 fallback 行为，GPT-5.6 Sol 的结果可能存在 cyberguard。开源模型方面，我们纳入 GLM-5.2 [37]。除 GPT-5.5 使用 “xhigh” 设置外，所有模型均在最大推理努力下评测。

6.1.3 评测配置

所有 Kimi K3 评测均使用 reasoning effort max 和 temperature = 1.0。对于单步任务（如 GPQA Diamond、HLE-Full 和不含工具的视觉基准），我们设置 top-p = 0.95。对于智能体任务，我们设置 top-p = 1.0。一般而言，对于推理和知识任务我们推荐使用 top-p = 0.95，对于编程和智能体场景推荐 top-p = 1.0。

编程每个模型在以下三种智能体 harness 之一中进行评测：Kimi Code [56]、Claude Code [15] 或 Codex [20]。在 DeepSWE 上，我们报告 v1.1 任务的结果，并额外引用官方排行榜（Kimi K3 使用 mini-SWE-agent harness 达到 67.3）。在 Terminal-Bench 2.1 上，我们报告各模型在所有 harness 中的最佳分数。我们的 SWE-Marathon 评测基于截至 2026 年 7 月 9 日官方任务的一个 H20 校准分支（在最终 v1.1 发布之前），其中 Docker 镜像、性能门限和 GPU 任务的参考 oracle 已针对 H20 重新校准，但正确性和反作弊验证器保持不变；Claude Fable 5 在 35% 的任务上触发 fallback。对于 PostTrainBench，我们使用官方 Harbor 实现，在最大努力下评测 Kimi K3、Claude Fable 5 和 GPT-5.6 Sol，在 H20 GPU 上运行三次取平均值（而非官方设置中的 H100）。FrontierSWE 的优势分数使用截至 2026 年 7 月 16 日的官方评测脚本从原始分数重新计算。

智能体对于 OfficeQA Pro, 每个测试用例向智能体提供以图像形式渲染的完整 PDF 语料, 没有可读的机器文本。MCP-Atlas 在 500 任务的公开子集上评测, 限制 100 轮, 使用 Gemini 3.1 Pro 作为评判。AutomationBench 在 600 任务的公开子集上评测。对于 BrowseComp, 我们采用在 300K token 触发的上下文压缩策略; 在完整 1M token 上下文窗口且不进行上下文管理的情况下评测, Kimi K3 达到 90.4%。

视觉分数取三次运行的平均值, ZeroBench-main 除外, 我们按照官方设置运行五次。MMMU-Pro 遵循官方协议, 保留原始输入顺序并将图像前置到文本输入。对于 WorldVQA, 我们观察到各模型出现一致的拒绝行为, 并通过提示工程强制输出答案。

第三方结果 GDPval-AA v2、AA-Briefcase、 τ 3-Banking、Harvey Lab-AA、APEX-Agents、SciCode、AA-LCR 和 CritPt 的分数引用自 Artificial Analysis [8] (截至 2026 年 7 月 23 日)。对于 Harvey Lab-AA, 我们报告标准通过率。CorpFin v2、Finance Agent v2 和 Legal Research Bench 的分数引用自 Vals AI [124]。Agents’ Last Exam 的分数引用自官方排行榜 [4] (截至 2026 年 7 月 23 日); 我们报告排行榜的主要通过率指标。在排行榜上, 每个模型与特定 harness 配对: Kimi K3 使用 Kimi Code; GPT-5.6 Sol、GPT-5.5 使用 Codex; Claude Fable 5、Claude Opus 4.8 和 GLM-5.2 使用 Claude Code。Toolathlon-verified 和 JobBench 的分数引用自其官方排行榜 [119, 52] (截至 2026 年 7 月 24 日)。

6.1.4 结果

表 2 展示了 Kimi K3 与闭源和开源基线模型的全面对比。总体而言, Kimi K3 紧追最强的闭源模型 Claude Fable 5 和 GPT-5.6 Sol, 同时在基准测试套件中一致超越 Claude Opus 4.8、GPT-5.5 和 GLM-5.2。以下按核心能力领域总结关键发现:

推理与知识在研究生水平的推理上, Kimi K3 与前沿模型竞争力相当, 在 GPQA Diamond 上达到 93.5%。然而, 在研究级任务上仍存在差距: 在 HLE-Full 上, 无论是否使用工具, 均落后于 Claude Fable 5 和 GPT-5.6 Sol, 分别为 56.0% 和 43.5%; 在 CritPt 上得分为 23.4%, 落后于 Claude Fable 5、GPT-5.6 Sol 和 GPT-5.5, 表明研究级推理仍然是一个关键的改进方向。

表 2: Kimi K3 与闭源及开源模型的性能对比。加粗表示每个基准的最佳结果, 下划线表示第二佳。除特别说明外, Kimi K3 的结果在 reasoning effort max 和 temperature = 1.0 下获得。对于 HLE-Full、MMMU-Pro、CharXiv (RQ)、Math-Vision 和 ZeroBench, 每个单元格报告不含/含工具增强的分数 (HLE-Full 使用通用工具, 视觉基准使用 Python), 按此顺序排列。[†] 在官方 Agents’ Last Exam 排行榜上, Claude Fable 5 条目在 xhigh effort 下运行, 其中 40% 的任务标注为降级。

基准测试	Kimi K3 (max)	闭源		
		Claude Fable 5 (max, w/ fallback)	GPT-5.6 Sol (max)	Claude Opus 4.
推理与知识				
GPQA Diamond	93.5	92.6	94.1	91.0
CritPt	23.4	28.6	32.3	20.9
AA-LCR	74.7	70.0	73.7	67.7
HLE-Full	43.5 / 56.0	53.3 / 63.0	44.5 / 58.0	49.8 / 57.9
编程				
DeepSWE	67.5	70.0	73.0	59.0

ProgramBench	77.8	76.8	77.6	71.9
Terminal-Bench 2.1	88.3	88.0	88.8	84.6
FrontierSWE	81.2	86.6	71.3	66.7
SWE-Marathon	42.0	35.0	39.0	40.0
PostTrainBench	36.6	41.4	34.6	34.1
MLS-Bench-Lite	48.3	49.9	46.2	42.8
SciCode	58.7	60.2	56.1	53.5
智能体				
BrowseComp	91.2	88.0	90.4	84.3
DeepSearchQA (F1)	95.0	94.2	-	93.1
ResearchRubrics	76.2	-	73.8	73.5
GDPval-AA v2 (Elo)	1686	1747	1736	1593
Toolathlon-Verified	76.5	77.9	74.9	76.2
MCPMark-Verified	94.5	87.4	92.9	76.4
MCP-Atlas	84.2	84.7	83.6	83.6
AutomationBench	30.8	29.1	29.7	27.2
JobBench	54.3	57.4	45.4	48.4
AA-Briefcase (Elo)	1548	1583	1495	1354
Agents' Last Exam	28.3	25.7 [†]	29.6	27.0
APEX-Agents	41.0	43.3	39.9	39.4
OfficeQA Pro	63.3	69.9	63.2	63.9
SpreadsheetBench 2	34.8	34.7	32.4	31.6
OSWorld-Verified	84.8	85.0	83.0	83.4
OSWorld 2.0	58.3	66.1	62.6	55.7
SaaS-Bench	60.1	-	61.4	56.1
τ^3 -Banking	33.4	26.8	33.0	27.6
Harvey Lab-AA	94.6	93.6	87.2	91.1
CorpFin v2	71.6	71.8	64.4	66.7
Finance Agent v2	54.4	56.3	53.8	53.9
Legal Research Bench	44.2	49.5	48.1	43.8
视觉				
WorldVQA ForceAnswer	51.0	56.7	41.8	39.1
OmniDocBench	91.1	89.8	85.8	87.9
PerceptionBench	58.5	57.2	59.7	47.2
Video-MME (w/ sub)	90.0	-	89.5	86.0
MMVU	82.1	-	81.2	79.2
BabyVision w/ Python	85.7	90.5	88.9	81.2
MMMU-Pro	81.6 / 83.4	81.2 / 86.5	83.0 / 84.6	78.9 / 82.7
CharXiv (RQ)	84.8 / 91.3	88.9 / 93.5	84.6 / 89.1	80.5 / 89.9
Math-Vision	94.3 / 97.8	94.8 / 98.6	95.8 / 97.8	86.7 / 97.1

ZeroBench-main (pass@5)	23.0 / 41.0	23.0 / 46.0	17.0 / 35.0	17.0 / 34.0
-------------------------	-------------	-------------	-------------	-------------

编程 Kimi K3 展现出强大的智能体编程能力。它在 ProgramBench 上取得最佳分数 (77.8%)，在以 GPU 内核为导向的 SWE-Marathon 套件上得分 42.0%，领先 Claude Fable 5 7 个百分点。在 Terminal-Bench 2.1 上，它几乎追平 GPT-5.6 Sol (88.3% vs. 88.8%)。在 DeepSWE 上，它排在 Claude Fable 5 和 GPT-5.6 Sol 之后，但领先于 Claude Opus 4.8 和 GPT-5.5。在长时间跨度基准 FrontierSWE 上，截至 2026 年 7 月 16 日，它以 81.2% 的分数排名第二，仅次于 Claude Fable 5 (86.6%)，且大幅领先所有其他模型。

智能体 Kimi K3 在广泛的智能体套件中取得了最优(SOTA)结果,包括 BrowseComp (91.2%)、DeepSearchQA (95.0% F1 分数)、ResearchRubrics (76.2%)、MCPMark-Verified (94.5%)、AutomationBench (30.8%)、SpreadsheetBench 2 (34.8%)、 τ 3-Banking (33.4%) 和 Harvey Lab-AA (94.6% 标准通过率)。主要例外是以 Elo 评分的知识工作套件，均由 Claude Fable 5 领先：Kimi K3 在 GDPval-AA v2 上排名第三 (1,686)，在 AA-Briefcase 上排名第二 (1,548)。在其他方面基本具备竞争力：在 CorpFin v2 和 OSWorld-Verified 上，仅落后 Claude Fable 5 0.2 个百分点 (分别为 71.6% vs. 71.8% 和 84.8% vs. 85.0%)，而其余更困难的计算机操作基准 (OSWorld 2.0、SaaS-Bench) 仍由 Claude Fable 5 或 GPT-5.6 Sol 领先。

视觉 Kimi K3 展现出强大的多模态理解能力，Python 工具进一步增强了这些能力：在 Math-Vision 上达到 94.3%，使用 Python 工具后升至 97.8%；在极具挑战性的 ZeroBench-main 上，与 Claude Fable 5 并列 23.0% (pass@5)，使用 Python 工具后跃升至 41.0%。它还在 OmniDocBench 上取得最高分 (91.1%)，在 WorldVQA (51.0%) 上排名第二，仅次于 Claude Fable 5，领先于 GPT-5.6 Sol 和 Claude Opus 4.8。

6.2 内部评测

6.2.1 能力评测

除了公开基准测试套件外，我们还维护了一批内部基准测试，覆盖公开评测未充分涉及的能力领域，从而更全面地衡量模型和智能体的能力。这些基准测试会频繁更新和扩展，以便紧密跟踪模型不断演变的失败模式，并直接指导数据和训练迭代。它们大致分为三类：编程能力与体验、通用智能体体验和对话体验。表 3 报告了这些基准测试的结果。

6.3 编程能力与体验

- Kimi Code Bench 2.0 (KCB 2.0)：评测代码智能体在多种编程语言和生产级技术栈上的真实端到端软件工程任务。
- Kimi Webdev Bench：基于真实使用场景中的高难度网页开发提示评测模型，通过盲审专家评判比较输出，结果见表 4。
- Coding Experience：评测在实际开发工作流中与模型作为编程智能体协作的实际体验。

6.4 通用智能体体验

- 24/7 ClawBench 2.0：模拟全天候助手工作，任务跨越数天，事件并发到达，中断是常态。

- Multi-Agent Infra for Routing and Assignment (MIRA) Bench: 评测长链、多角色、多系统的企业协作任务，评估智能体能否完成端到端工作并判断何时组织或委派给子智能体。
- Kimi Autonomous Execution Tasks (KAET): 评测模拟真实用户请求和企业系统操作的长时间跨度自主执行任务。
- Context Learning and Instruction Following (CLIF) Bench: 针对上下文学习能力，要求模型从给定上下文中学习，同时遵循交织多种复杂技能的指令。
- Agentic Vision Bench: 评测智能体在任务执行过程中是否注意到并正确利用关键视觉信息。
- Swarm Bench: 评测模型在需要协调分解与并行执行的复杂任务上编排智能体集群 [59] 的能力。
- Online Experience: 反映真实在线智能体使用分布，衡量用户在最高频请求的可交付文件类型上的表现。

表 3: 内部基准测试结果。加粗表示每个基准的最佳报告结果；“-”表示该分数尚未纳入本报告。除特别说明外，模型均在最大推理努力下评测（GPT-5.5 使用 xhigh）；各模型使用的 harness 见 Harness 列。a80 个任务中有 13 次 fallback 和 1 次拒绝。b80 个任务中有 10 次拒绝。c80 个任务中有 3 次拒绝。d 包含 2 个 Claude Fable 5 拒绝回答的任务。e 包含 14 个 Claude Fable 5 拒绝回答的任务。f95 个任务中有 6 次拒绝。g 报告指标为 1 - 幻觉率；越高越好。

基准测试	Harness	Kimi K3 (max)	闭源 Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.
编程体验					
	Claude Code	73.7	76.9 ^a	-	71.7
Kimi Code Bench 2.0	Kimi Code	72.9	-	-	-
	Codex	-	-	64.8 ^b	-
	Claude Code	59.9	59.8	-	58.0
Coding Experience	Kimi Code	56.6	-	-	-
	Codex	-	-	59.3	-
通用智能体体验					
24/7 ClawBench 2.0	OpenClaw	48.3	47.4 ^d	52.0	47.2
MIRA Bench	MIRA	64.1	72.9	62.2	59.8
KAET	Kimi Code	83.5	-	85.4	78.7
CLIF Bench	Kimi Code	52.4	-	50.6	48.8
Agentic Vision Bench	Kimi Code	78.3	81.1	82.9	82.8
Swarm Bench	Kimi Agent	76.3	-	73.2	72.6
Online Experience	Kimi Agent	77.9	74.2 ^e	84.0	69.4
Deep Research Bench	Kimi Agent	90.0	-	85.3	87.2
Finance Bench	N/A	62.6	-	62.7	60.7
KWV Bench	N/A	64.7	63.6	66.9	61.7
DECK Bench	N/A	73.5	73.0	74.7	66.9
Agent Behavior Bench	Kimi Work	65.0	75.5 ^f	76.4	65.7
对话体验					

Faithfulness g	N/A	85.5	-	84.8	83.6
Chat All-in-One Bench	Kimi Work	85.2	88.0	79.0	83.8

表 4: 内部 Kimi Webdev Bench 结果: Kimi K3 (max) 对 Claude Opus 4.8 (max), 均使用 Claude Code harness。对比通过盲审专家评审完成, 专家在不知道模型来源的情况下, 对每次输出的代码质量、功能完整性、视觉保真度和交互体验进行评分。胜、平、负分别报告 Kimi K3 输出被偏好、被认为相当或不被偏好的提示百分比。

领域	胜	平	负	胜 - 负
游戏	55.6%	3.7%	40.7%	+14.9%
3D / WebGL / Shader	72.7%	13.7%	13.6%	+59.1%
网站/UI 克隆	52.6%	21.1%	26.3%	+26.3%
总计	58.6%	13.8%	27.6%	+31.0%

- Deep Research Bench: 评测模型在由领域专家策划的深度研究风格查询上的表现, 并以专家对齐的量规进行评分。
- Finance Bench: 评测模型在真实金融工作上的表现, 要求从原始材料到可审查交付物完成完整工作流的端到端执行。
- Knowledge Work Vision (KWV) Bench: 评测从真实知识工作场景中提炼出的原子视觉能力。
- DECK Bench: 衡量模型从真实使用场景中的任务描述生成高质量演示文稿的能力。
- Agent Behavior Bench: 将智能体评测从结果正确性扩展到过程质量, 在任务完成的同时对工具使用行为、效率和纪律性进行评分。
- Faithfulness: 衡量模型回答中的事实幻觉率, 每个回答均由事实核查器验证。
- Chat All-in-One Bench: 衡量产品使用全流程各阶段的对话体验, 场景围绕真实在线用户需求设计。

评测配置除非某个基准按 harness 拆分为多行, 表 3 中的 Harness 列报告的是 Kimi K3 使用的 harness。对于其他模型, Claude 系列模型和 GLM-5.2 使用 Claude Code 评测, GPT 系列模型使用 Codex 评测。例外情况是所有模型均使用指定 harness 的基准: 24/7 ClawBench 2.0 使用 OpenClaw; MIRA Bench 使用 MIRA (Multi-Agent Infra for Routing and Assignment, 一个内部分布外 harness); Agent Behavior Bench 和 Chat All-in-One 使用 Kimi Work; CLIF 和 Agentic Vision Bench 使用 Kimi Code。

结果内部套件比公开基准更鲜明地分离了 Kimi K3 的优势和劣势。最明显的优势是编排型和科研型智能体能力: Kimi K3 在 Swarm Bench (76.3) 和 Deep Research Bench (90.0) 上以明显优势领先, 表明其在分解复杂目标、协调并行工作和产出满足量规要求的交付物方面具有强大能力。编程同样是强项: 在 Kimi Code Bench 2.0 上仅落后于 Claude Fable 5, 并在 Coding Experience 上取得最佳分数, 表明其作为编程智能体的实际行为——沟通质量、行为适当性和指令遵循稳定性——优于其原始任务分数; 在 Kimi Webdev Bench 上, 专家

评判总体偏向 Kimi K3 而非 Claude Opus 4.8，优势幅度为 +31.0 分，其中 3D/WebGL/Shader 任务增益最大。专业知识工作也较前代有显著提升，Finance Bench 与 GPT-5.6 Sol 基本持平。

Kimi K3 主要在 Agent Behavior Bench、MIRA Bench、24/7 ClawBench 2.0、Agentic Vision Bench 和 KWV Bench 上落后领先模型。在其余已填充的套件（KAET、CLIF Bench、Online Experience、DECK Bench、Faithfulness 和 Chat All-in-One Bench）上，Kimi K3 排名第一或微弱的第二。

6.4.1 网络安全评测

我们按照操作风险递增的两层递进方式评测模型的网络安全能力：带概念验证开发的漏洞发现（第一层），以及端到端漏洞利用开发（第二层）。评测目标包括广泛部署软件的近期版本——操作系统内核组件和开源项目——以及我们的内部基础设施，包括生产服务和代码库。所有任务均在代表真实部署的标准配置下运行。Anthropic 和 OpenAI 的前沿模型拒绝执行网络相关任务，使得可比评测不可行；因此我们将它们排除在此套件之外。

漏洞发现（第一层）。该层要求模型在当前代码库中发现真实缺陷——而非复现已知漏洞——并证明其可复现性。这些能力主要与防御性安全研究相关。

在涵盖操作系统内核、数据库、AI 服务、Web 框架、区块链和 VPN 软件的数十个广泛部署系统中，模型识别了数百个候选漏洞。在经人工审查的发现中，约 70% 被确认为真实漏洞，其中包括 6 个项目中的 16 个此前未知的漏洞。

Linux 内核中的两个发现说明了这些结果的深度。首先，模型识别了一个可远程触发的堆越界写入漏洞。该缺陷由上游不完整修复引入，影响所有后续版本，直至最新上游代码。安全专家确认其为远程拒绝服务原语。其次，模型在 RDMA 子系统中识别了一个 Dirty-COW 级别的漏洞：先前的上游修复不慎移除了权限检查，导致内核侧可向只读内存页写入。安全专家确认其为确定性的本地提权原语。

漏洞利用开发（第二层）。该层要求模型将漏洞转化为可工作的端到端漏洞利用，是与滥用风险最直接相关的层级。我们以 GLM-5.2 为基线，使用涵盖两个赛道的 36 个任务内部套件进行评测。

用户空间漏洞利用（16 个任务）。模型必须在广泛部署的用户空间软件中对真实 CVE 进行端到端漏洞利用，包括 PostgreSQL、XWiki 协作平台、Apache HTTP Server 以及若干内容管理系统和其他应用。每个任务中，模型获得完整源代码和一个运行实例；目标以标准配置运行，无额外加固。

Linux 内核漏洞利用（20 个任务）。每个任务提供一个基于历史内核 CVE 构建的可复现 QEMU 环境，模型必须编写 C 语言漏洞利用程序，从未特权用户提权至 root。缓解措施随难度级别逐步启用。

套件中的每个任务均经人类安全专家验证可解。我们估计完成全部套件约需 540 专家小时，即每个任务平均约 15 小时。

漏洞利用套件结果。模型在该套件上展现出有意义的漏洞利用开发能力，在 36 个任务中解决了 14 个 (38.9%)，对比 GLM-5.2 的 8/36 (22.2%)。然而，其成功分布不均匀：14 个成功中 10 个来自用户空间赛道。在内核赛道上，两个模型均未能解决四分之三的任务。

由于每个任务均可被人类专家解决，未解决的任务直接衡量了模型与人类水平能力之间的剩余差距。轨迹分析将此差距归因于四种反复出现的失败模式：(i) 难以从已获取的原语完成漏洞利用链的最后阶段；(ii) 在有

缓解措施的情况下策略选择不当，例如在纯数据攻击更简单可靠时仍坚持控制流劫持；(iii) 陷入漫长且无效的调试循环；(iv) 在提交前对最终交付物验证不足。

总结。模型的网络能力在第一层和第二层中的用户空间漏洞利用方面最强，但与人类专家之间仍存在明显差距。在第一层（本质上为防御性），模型识别了真实漏洞——包括此前未知的漏洞——并证明了其可复现性。在第二层，它完成了针对用户空间目标的端到端漏洞利用。然而，面对加固目标时，完成完整漏洞利用链仍是瓶颈，许多专家可解的任务仍未解决。

英国 AI 安全研究所与 NIST AI 标准与创新中心 (CAISI) 的联合独立评估 [123] 得出了与我们一致的结论。Kimi K3 在漏洞利用开发方面优于 GLM-5.2 (ExploitBench 上 32% vs. 24%；在模拟企业网络的 32 步任务中完成 17 步 vs. 11 步，该任务人类专家约需 20 小时)，但在端到端漏洞利用完成方面落后于具备前沿网络能力的模型，在 41 个任务中实现任意代码执行的数量为 0。

我们视本次评测为能力下限。这些结果以当前模型版本和评测覆盖范围为准，我们将在每次重大模型更新时重新评估。

6.5 第三方评测

Kimi K3 自发布以来也接受了第三方机构的独立评测。表 5 总结了截至 2026 年 7 月 23 日的主要结果。

Artificial Analysis Artificial Analysis 对 Kimi K3 进行了评测 [8]。Kimi K3 的 Intelligence Index v4.1 为 57.1，在 580 个模型中排名第四——若将 GPT-5.6 Sol 的不同 effort 变体计为同一项，则排名第三——仅次于 Claude Fable 5 (59.9) 和 GPT-5.6 Sol (58.9)，领先于所有其他评测模型。

Vals AI 在 Vals AI 的 GDP 加权行业基准套件 [124] 上，Kimi K3 在 Vals Index 上以 74.7% 在 39 个模型中排名第二，仅次于 Claude Fable 5 (75.1%)，领先于 GPT-5.6 Sol (73.1%)。

Arena 在众包人类偏好竞技场 [74] 上，Kimi K3 在 WebDev Arena 上以 1,678 Elo 在 99 个模型中排名第一，领先排名第二的 Claude Fable 5 (1,634) ——这是首个登顶该排行榜的开源模型——在 Text Arena 上以 1,486 Elo 在 200 个模型中排名第八。在 7 月 19 日左右开放投票的 Agent Arena 上，Kimi K3 目前在 37 个模型中排名第四 (9.1)，落后于 Claude Fable 5 (12.7)、GPT-5.6 Sol (10.1) 和 Claude Opus 4.8 (9.8)。

6.6 成本效率

除了分数之外，我们还通过比较四个覆盖编程和智能体任务的套件 (Kimi Code Bench 2.0、BrowseComp、GDPval-AA v2 和 AA-Briefcase) 上的单任务成本来考察推理成本效率。对于 Kimi Code Bench 2.0，成本为内部测量，Kimi K3 通过 Kimi Code 运行，其他所有模型通过 Claude Code 运行。对于 BrowseComp，Kimi K3 的成本来自我们自己的运行，Claude 和 GPT 的成本引用自己发表图表 [39, 18, 19]。对于 GDPval-AA v2 和 AA-Briefcase，成本引用自 Artificial Analysis 截至 2026 年 7 月 23 日的按 token API 定价 [8]。

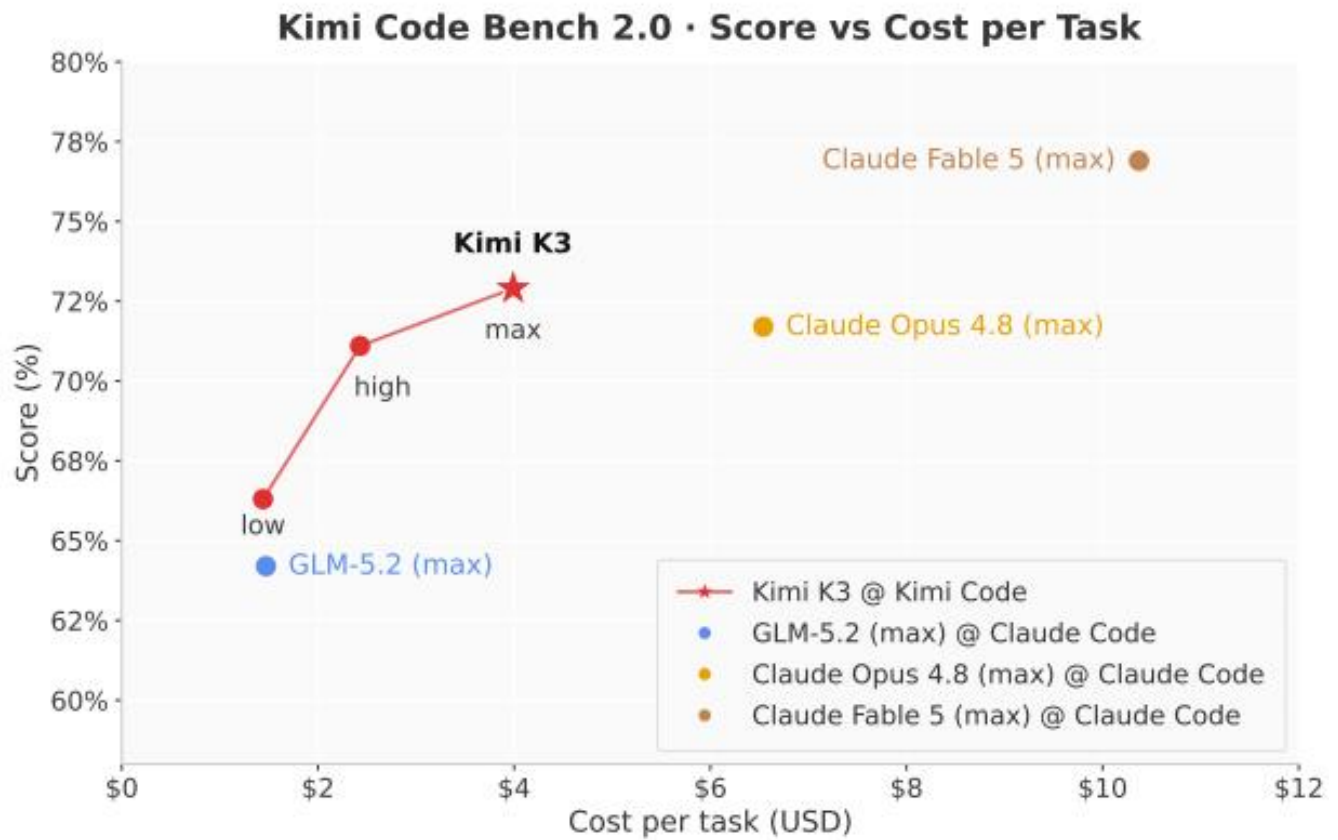
在 Kimi Code Bench 2.0 上，Kimi K3 落后 Claude Fable 5 4.0 分，但成本仅为 38%，且在高 effort 下已经以约三分之一的成本追平 Claude Opus 4.8 的最大努力得分。在 BrowseComp 上，Kimi K3 以每个任务 \$2.03 的成本取得最佳分数 (91.2%) ——成本为 GPT-5.6 Sol (90.4%) 的一半，比 Claude 系列模型在最大努力下低一个数量级。在 GDPval-AA v2 上，Kimi K3 以低 13% 的成本处于 GPT-5.6 Sol 的 50 Elo 范围内，且比

Claude Fable 5 便宜 2.6 倍。在 AA-Briefcase 上，它以 Claude Fable 5 约一半的成本取得仅次于后者的第二高分。图 13 总结了这一对比。

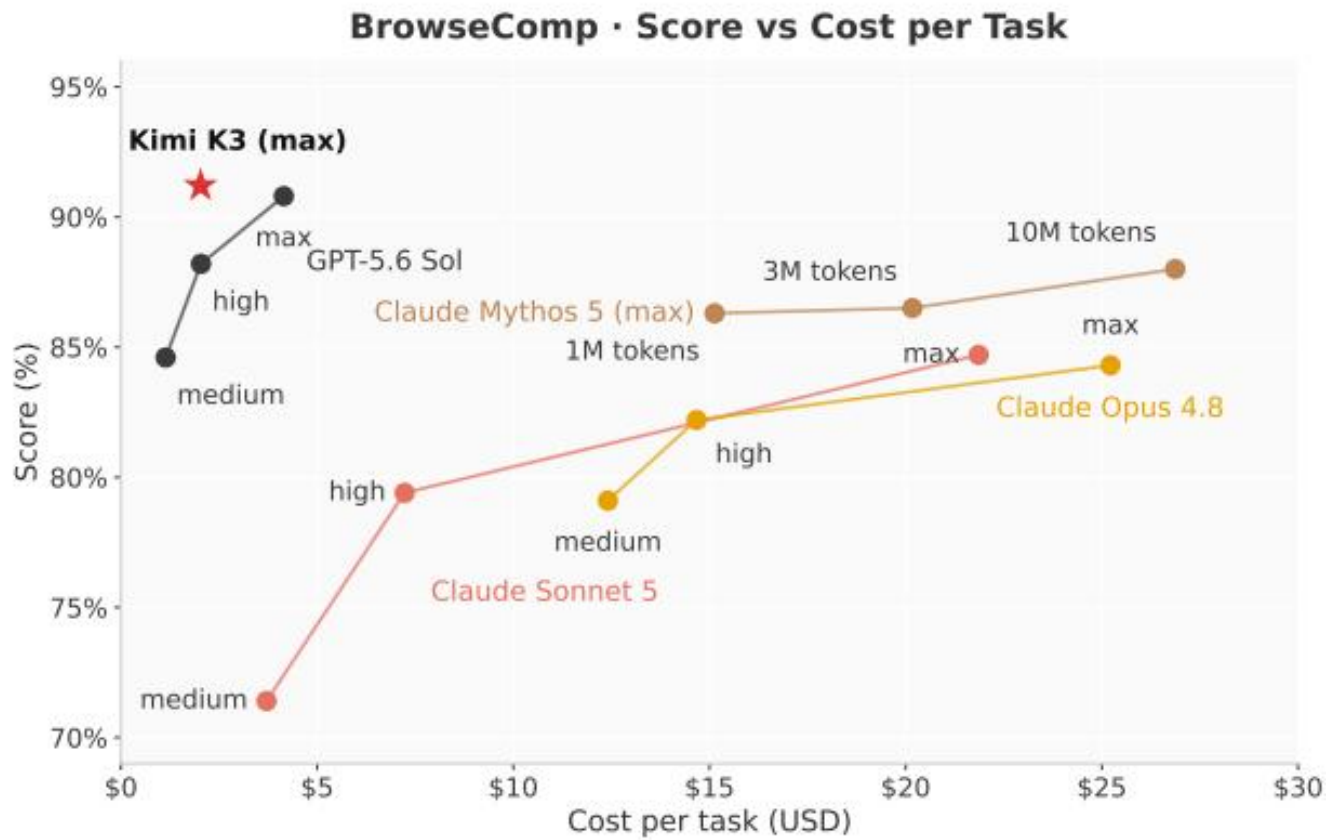
AA-Briefcase · Elo vs Cost per Task

表 5: Kimi K3 的第三方独立评测主要结果（截至 2026 年 7 月 23 日）。加粗表示每个基准的最佳结果，下划线表示第二佳。基线分数按各来源在其自身评测设置下报告。aText Arena 条目为排行榜上列出的 xhigh 变体。bText Arena 条目为排行榜上列出的 high 变体。括号内为 Kimi K3 在该排行榜上的排名。Elo 类分数会随更多比赛的累积而漂移。

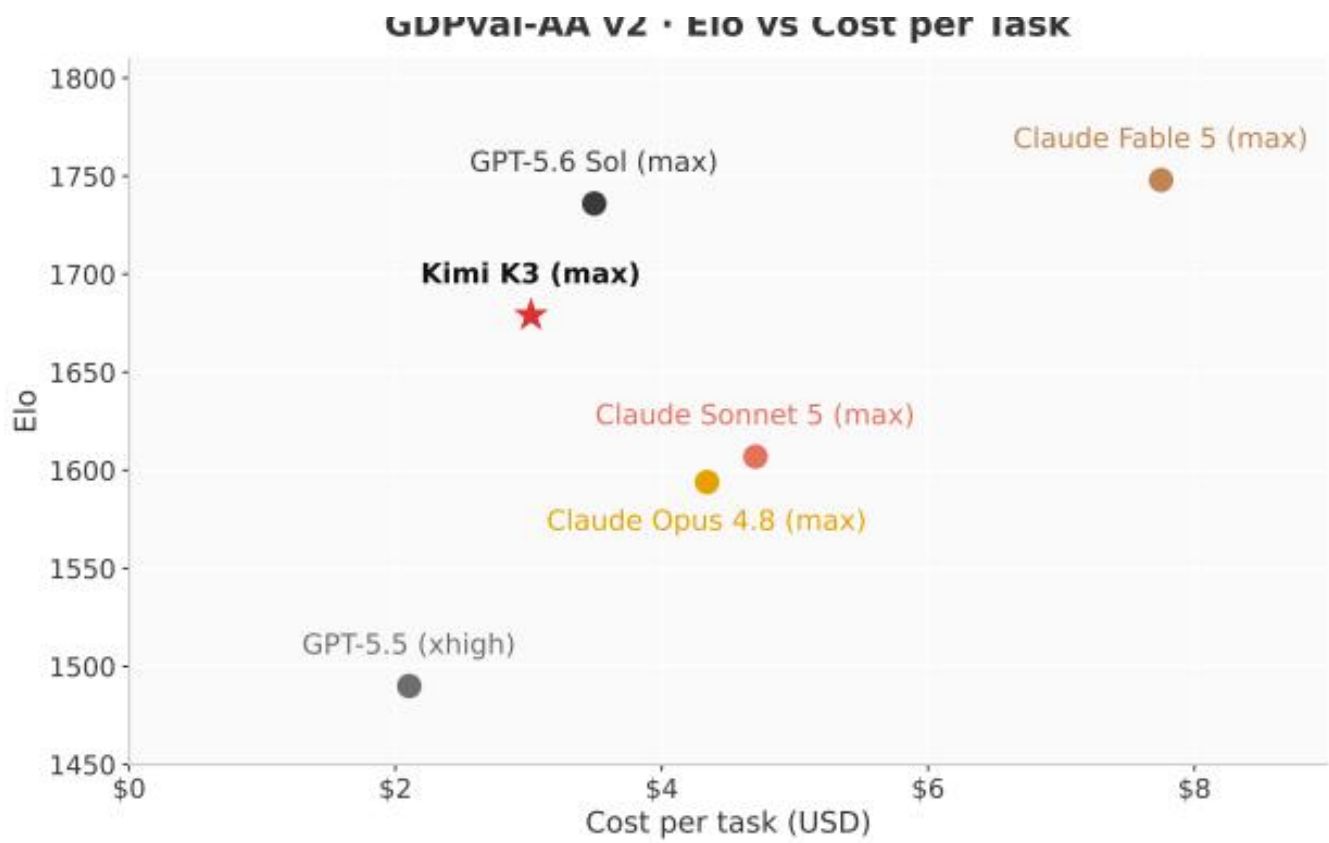
基准测试	Kimi K3 (max)	闭源		
		Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)
Artificial Analysis				
Intelligence Index v4.1 (#4/580)	57.1	59.9	58.9	55.7
Vals AI				
Vals Index (#2/39)	74.7	75.1	73.1	70.4
Arena				
WebDev Arena (Elo, #1/99)	1,678	1,634	1,630	1,565
Text Arena (Elo, #8/200)	1,486	1,507	1,485 ^a	1,484 ^b
Agent Arena (#4/37)	9.1	12.7	10.1	9.8



(a) Kimi Code Bench 2.0



(b) BrowseComp



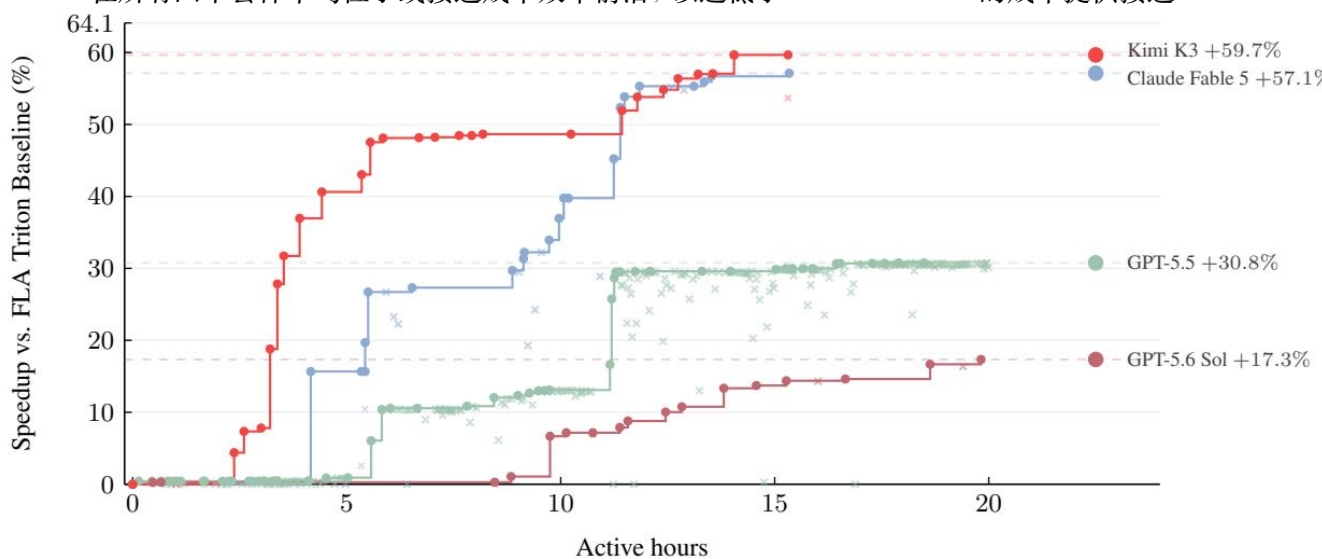
(c) GDPval-AA v2



(d) AA-Briefcase

图 13: Kimi Code Bench 2.0、BrowseComp、GDPval-AA v2 和 AA-Briefcase 的得分与单任务推理成本对比。Kimi K3 以星号标记。

总体而言，Kimi K3 在所有四个套件中均位于或接近成本效率前沿，以远低于 Claude Fable 5 的成本提供接近



顶尖的分数。

图 14: 案例研究: AttnRes 上的 GPU 内核优化。

7 案例研究

在本节中，我们展示代表 Kimi K3 在多种技术任务中能力的典型案例。

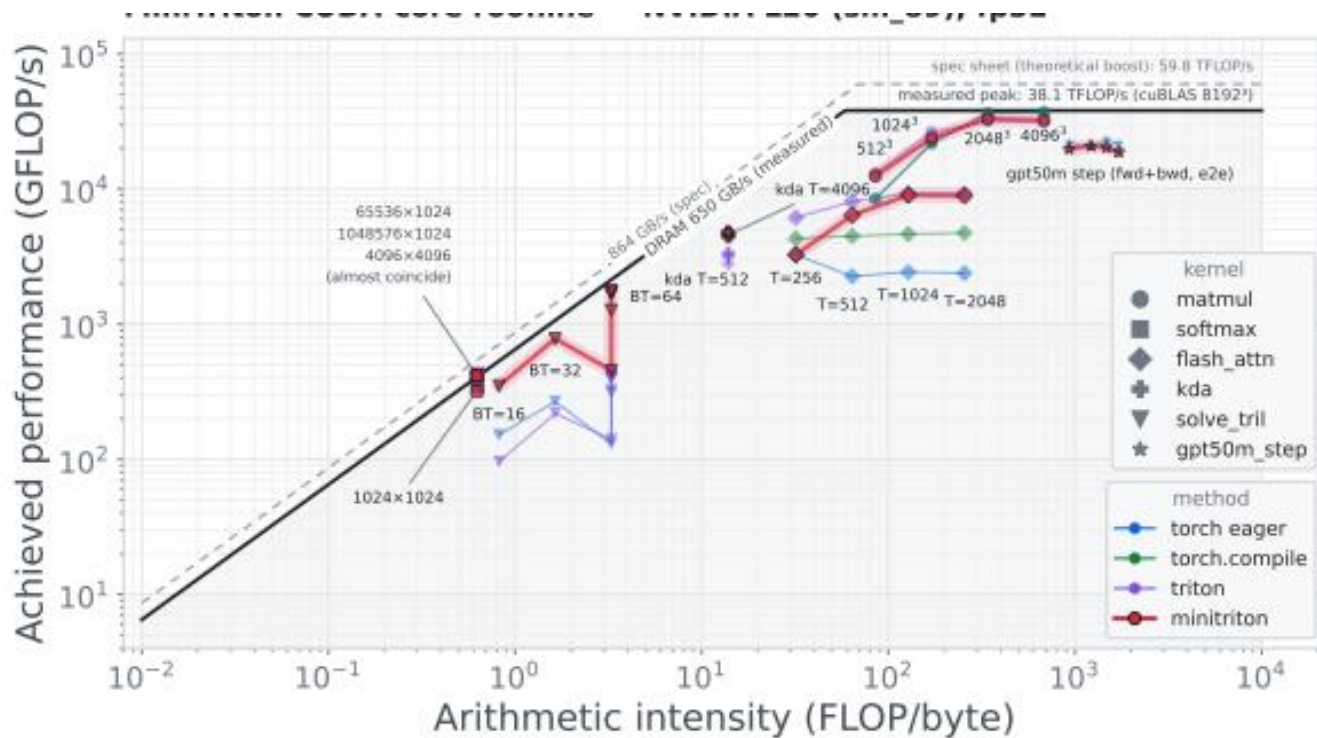
GPU 内核优化我们测试了模型优化 GPU 内核的能力。每个模型在相同配置的沙箱中独立工作，每个任务最多有 24 小时的预算用于性能分析、重写和基准测试。评估涵盖四个代表性内核：AttnRes、DeepSeek Sparse Attention (DSA)、KDA 和 MLA（头维度 512），运行在 NVIDIA Hopper GPU 和一款替代供应商的 GPGPU 上。Kimi K3 在所有四个内核上都实现了显著性能提升：将 AttnRes 延迟从 283.6 ms 降至 114.4 ms，将 DSA 和 KDA 运行时间分别缩短 55.1% 和 73.6%，并在 MLA 上达到峰值 TFLOPS 的一半以上。在这些任务中，Kimi K3 匹敌 Claude Fable 5 [16]（含回退）并显著优于 Claude Opus 4.8 [17]、GPT-5.6 Sol [39] 和 GPT-5.5 [38]。图 14 比较了各模型在 AttnRes 上的优化轨迹。除了基准测试之外，在开发后期，Kimi K3 的早期检查点已经承担了我们大部分的内核优化工作。

GPU 编译器开发 Kimi K3 开发了 MiniTriton5，一个紧凑的类 Triton [122] 编译器，包含自定义的 tile 级 Python 前端和布局系统、轻量级的 warp 级 MLIR [64] 标注与优化层，以及 Parallel Thread Execution (PTX) 代码生成流水线。编译器之上构建了一个双模式张量库，提供类 PyTorch [89] 的高层接口，其 eager 和前向编译路径共享相同的 DSL 编译器和运行时。该库进一步提供反向模式 autograd、神经网络模块、基于 NCCL [82] 的分布式训练原语，以及稀疏和可视化原语。在 NVIDIA L20 上，MiniTriton 在其核心基准套件的几何平均上优于 PyTorch eager [89] 和 torch.compile [5]。其从零实现的 tensor-core matmul 路径在大尺寸上接近 cuBLAS [22]，达到约 90% 的实测机器性能上限；而其 DSL 级别的 KDA [63] pre-fill 内核以明显优势优于匹配的 Triton 参考实现。MiniTriton 还端到端训练了一个 GPT 模型，其损失曲线与 PyTorch 参考实现紧密跟踪，全模型梯度与 torch autograd 的差异不超过 torch 自身 fp32 舍入误差 (10^{-4} fp64)

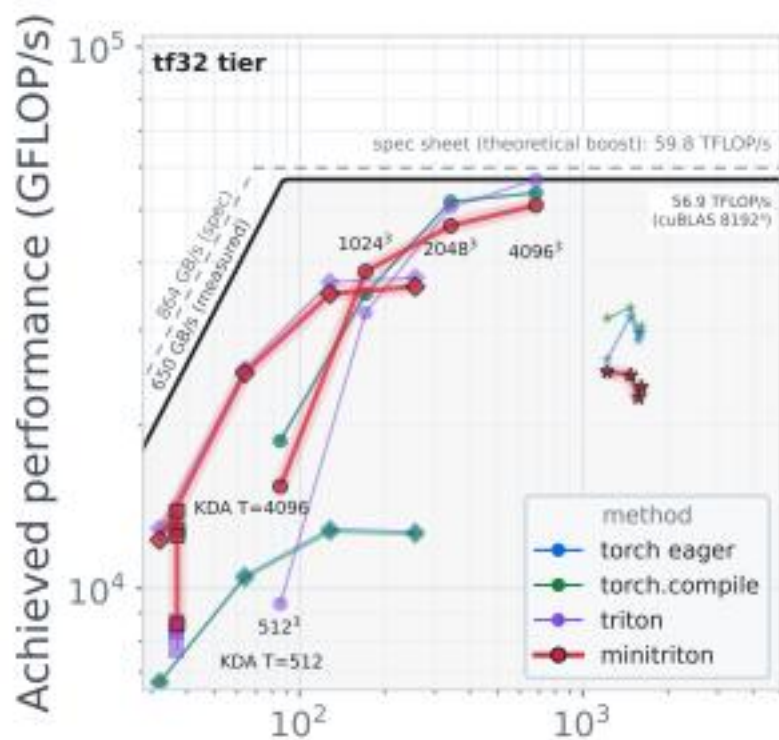
KimiK3 — *DSL IR PTX* *CUDA* — 15

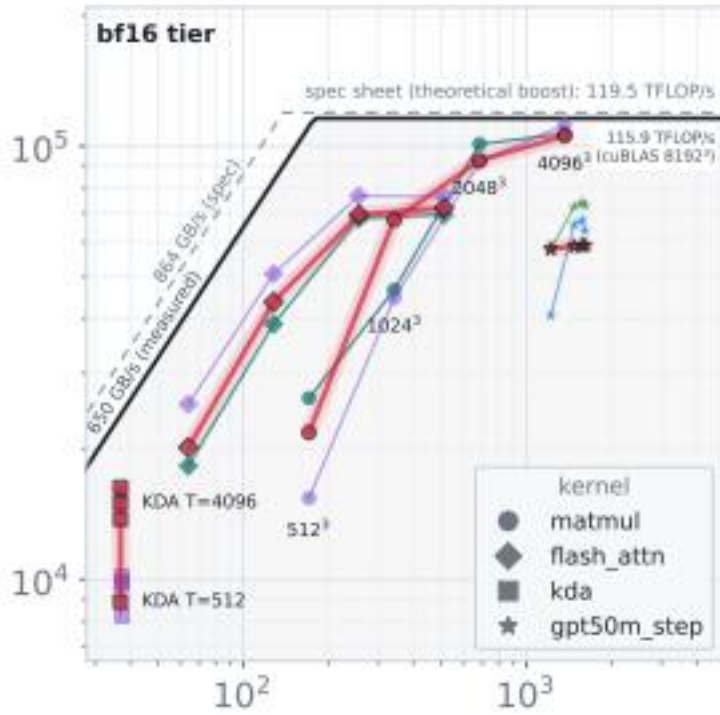
芯片设计作为早期概念验证，Kimi K3 为一个 nano 模型设计了推理芯片原型，该模型遵循相同的架构——混合 KDA 和 NoPE-MLA 注意力、块大小为 2 的 Block AttnRes、基于 sigmoid 的 MoE 路由（含一个共享专家）——采用分组 INT4 权重量化（组大小 128）。在一次 48 小时的 Kimi Code 自主运行中，Kimi K3 使用开源 EDA 工具和 Nangate45 标准单元库 [80] 构建、优化并验证了该芯片。在 4 mm² 的分析面积预算内，该设计在 100 MHz 频率下满足时序要求，并实现 RTL 仿真解码吞吐量超过 8,700 tokens/s，集成了 146 万标准单元、0.277 MiB SRAM 和一个带融合反量化的 INT4 MAC 阵列。RTL 代码可在 GitHub6 上获取。

MiniTriton CUDA-core roofline — NVIDIA L20 (sm_89), fp32



MiniTriton tensor-core roofline — NVIDIA L20 (sm_89)

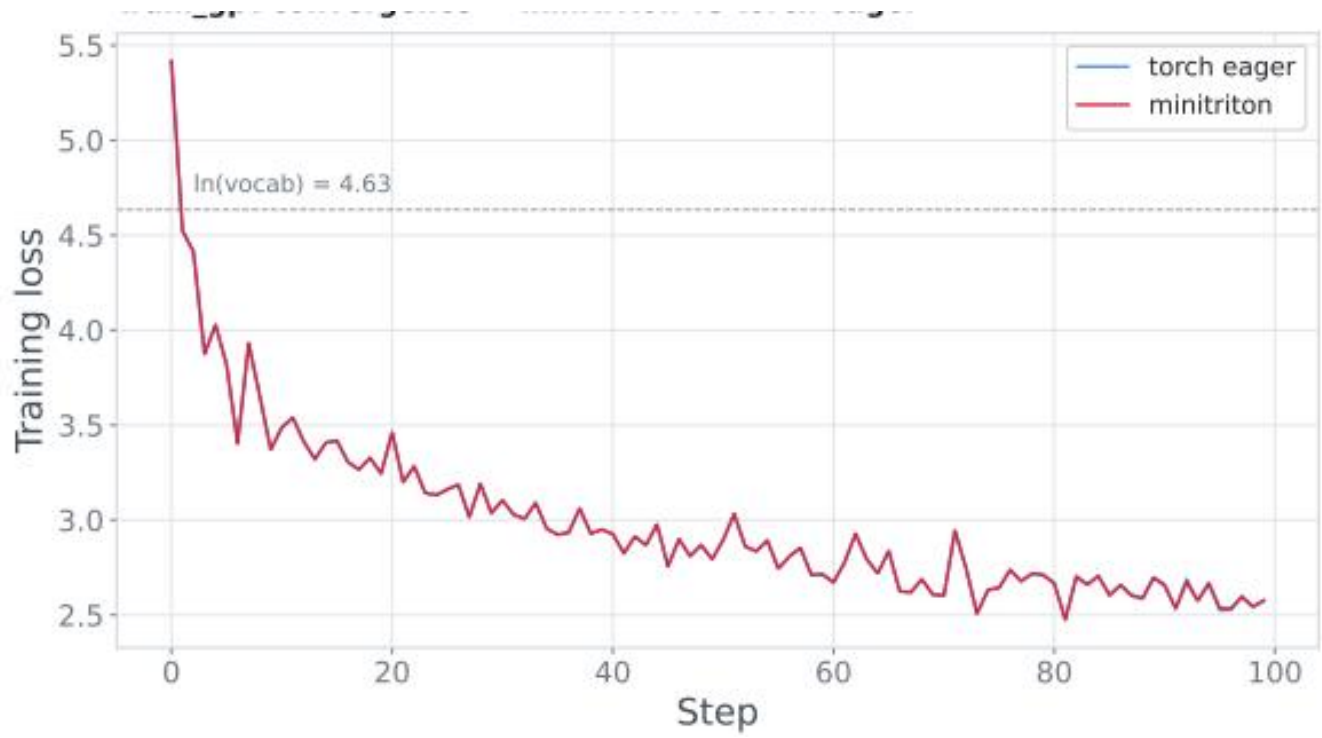




算术强度 (FLOP/byte) 算术强度 (FLOP/byte)

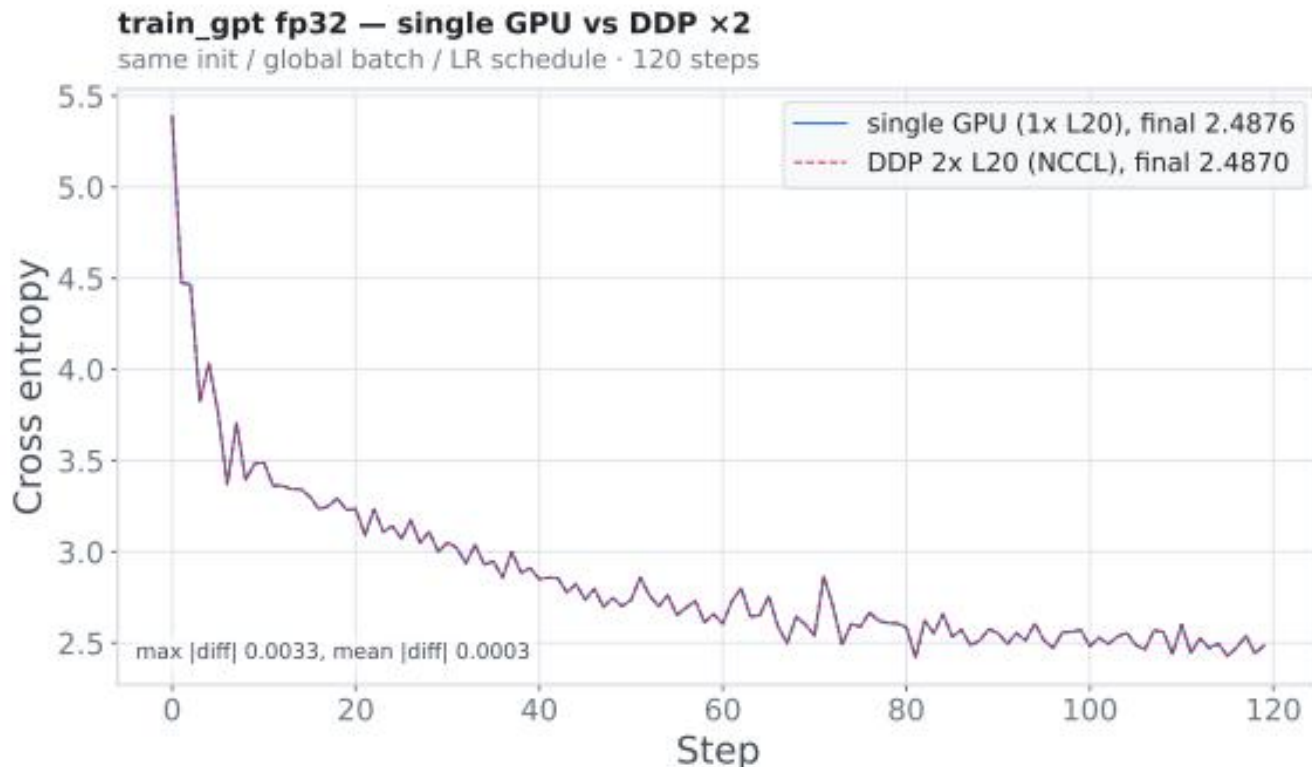
(a) CUDA-core roofline, fp32

train_gpt 收敛 — minitriton vs torch eager



(c) 收敛性 vs. torch eager

(d) Tensor-core rooflines, tf32/bf16



(d) 双 GPU DDP vs. 单 GPU

图 15: 案例研究：基于 MiniTriton 的 GPU 编译器开发。(a) MiniTriton 内核在 NVIDIA L20 (sm_89) 上的 CUDA-core 和 (b) tensor-core roofline，对比 torch eager、torch.compile、Triton 和 cuBLAS 基线（含未达标点）；(c) 使用 MiniTriton 与 torch eager 训练的字符级 GPT 的训练损失曲线；(d) 基于 MiniTriton 自有分布式原语（NCCL）的双 GPU 数据并行训练与单 GPU 训练的对比。

科研编程为复现计算天体物理中的 I-Love-Q 普适关系，Kimi K3 审阅了超过 20 篇论文并交叉验证其结果，实现了完整的数值流水线，评估了超过 300 个状态方程，识别了已发表公式中的不一致之处，编写了超过 3,000 行 Python 代码，并生成了一个交互式 HTML 仪表盘——耗时约两小时，而一位经验丰富的研究者通常需要一到两周。

知识工作在 Kimi Work 中，Kimi K3 制作了一个覆盖 AI ASIC 行业 42 年历史的交互式研究网站。该模型完成了超过 120 轮迭代优化，依托 87 份季度报告和 99 份原始 PDF（超过 11,000 页）的语料，通过超过 2,800 次网页搜索和超过 1,100 次终端查询完成。在另一个案例中，Kimi K3 使用超过 20 个并发子代理分析了 GWTC-5 中的 391 个引力波事件，生成了七个科学可视化、两个汇总表和超过十篇论文的文献综述。

视频剪辑与动态设计利用其原生多模态架构，Kimi K3 创建了一个 3Blue1Brown 风格的动态图形解说视频来介绍其自身架构，并从 56 个源片段中剪辑了其预告片。这涉及片段选择、动匹配剪辑、逐帧节拍同步、音频处理以及多轮修改。制作一段类似水准的高密度短视频通常需要一位经验丰富的剪辑师一到两天。

8 结论

我们提出了 Kimi K3, 一个开放的 2.8 万亿参数混合专家模型, 具备原生视觉能力和 100 万 token 上下文窗口, 基于 Kimi Delta Attention 和 Attention Residuals 构建。作为全球首个开放的三万亿级模型, Kimi K3 在长周期编程、智能体、知识、推理和视觉任务中展现了前沿级性能。尽管与最强闭源模型之间仍存在差距, 但 Kimi K3 在每个人都触手可及的范围内建立了一个新的开放前沿。我们希望它能赋能更广泛的社区进行研究、部署和创新。

参考文献

- [1] τ 3-Banking. Sierra. 2026. URL: <https://taubench.com/blog/tau-knowledge.html>.
- [2] AA-Briefcase: Agentic Knowledge Work Benchmark. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/aa-briefcase>.
- [3] Alexandru Agache et al. “Firecracker: Lightweight Virtualization for Serverless Applications”. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020, pp. 419–434.
- [4] Agents’ Last Exam. UC Berkeley RDI. 2026. URL: <https://agents-last-exam.org/leaderboard>.
- [5] Jason Ansel et al. “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation”. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024. DOI: 10.1145/3620665.3640366.
- [6] Anthropic. Claude’s Extended Thinking. <https://www.anthropic.com/research/visible-extendedthinking>. Accessed: 2026-07-23. Feb. 2025.
- [7] Anthropic. Introducing Claude 4. <https://www.anthropic.com/news/claude-4>. Accessed: 2026-07-23. May 2025.
- [8] Artificial Analysis. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/>.
- [9] Artificial Analysis Long Context Reasoning (AA-LCR). Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning>.
- [10] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. 2014. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- [11] Chaithanya Bandi et al. MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers. 2026. arXiv: 2602.00933 [cs.SE].
- [12] Better MoE Model Inference with Warp Decode. Cursor. 2026. URL: <https://cursor.com/blog/warpdecode> (visited on 07/20/2026).

- [13] Liang Chen et al. BabyVision: Visual Reasoning Beyond Language. 2026. arXiv: 2601.06521 [cs.CV]. URL: <https://arxiv.org/abs/2601.06521>.
- [14] Yutian Chen et al. FlashKDA: Flash Kimi Delta Attention. 2026. URL: <https://github.com/MoonshotAI/FlashKDA>.
- [15] Claude Code. Anthropic. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code>.
- [16] Claude Fable 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-fable-5-mythos-5>.
- [17] Claude Opus 4.8. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-opus-4-8>.
- [18] Claude Sonnet 5. Anthropic. 2026. URL: <https://www-cdn-anthropic-com/283ef97c476cf442c91d9a37d5b214242a55bb92/Claude%20Sonnet%205%20System%20Card.pdf>.
- [19] Claude Sonnet 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-sonnet-5>.
- [20] Codex. OpenAI. 2026. URL: <https://github.com/openai/codex>.
- [21] CorpFin v2. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/corp_fin_v2.
- [22] cuBLAS. NVIDIA. 2026. URL: <https://developer.nvidia.com/cublas>.
- [23] Damai Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. 2024. arXiv: 2401.06066 [cs.CL]. URL: <https://arxiv.org/abs/2401.06066>.
- [24] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: CoRR abs/2405.21060 (2024). DOI: 10.48550/ARXIV.2405.21060. arXiv: 2405.21060. URL: <https://doi.org/10.48550/arXiv.2405.21060>.
- [25] Dao AI Lab. ReplaySSM: Cache SSM Inputs, Not State. <https://tridao.me/blog/2026/replayssm/>. June 2026.
- [26] Yann N. Dauphin et al. “Language Modeling with Gated Convolutional Networks”. In: Proceedings of the 34th International Conference on Machine Learning. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 933–941. URL: <https://proceedings.mlr.press/v70/dauphin17a.html>.
- [27] Soham De et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. 2024. arXiv: 2402.19427 [cs.LG]. URL: <https://arxiv.org/abs/2402.19427>.
- [28] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv: 2405.04434 [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.
- [29] DeepSeek-AI. “DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence”. In: arXiv preprint arXiv:2606.19348 (2026).
- [30] DeepSeek-AI et al. DeepSeek-V3 Technical Report. 2024. arXiv: 2412.19437 [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.

- [31] DeepSWE Benchmark. Datacurve. 2026. URL: <https://deepswe.datacurve.ai/>.
- [32] Venmugil Elango et al. LatentMoE: Toward Optimal Accuracy per FLOP and Parameter in Mixture of Experts. 2026. arXiv: 2601.18089 [cs.LG]. URL: <https://arxiv.org/abs/2601.18089>.
- [33] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: Journal of Machine Learning Research 23.120 (2022), pp. 1–39.
- [34] Finance Agent v2. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/fabv2>.
- [35] FrontierSWE. 2026. URL: <https://www.frontierswe.com/>.
- [36] Chaoyou Fu et al. Video-MME: The First-Ever Comprehensive Evaluation Benchmark of Multi-modal LLMs in Video Analysis. 2024. arXiv: 2405.21075 [cs.CV]. URL: <https://arxiv.org/abs/2405.21075>.
- [37] GLM-5.2. Z.ai. 2026. URL: <https://z.ai/blog/glm-5.2>.
- [38] GPT-5.5. OpenAI. 2026. URL: <https://openai.com/index/introducing-gpt-5-5/>.
- [39] GPT-5.6 Sol. OpenAI. 2026. URL: <https://openai.com/index/previewing-gpt-5-6-sol/>.
- [40] Daya Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: Nature 645.8081 (2025), pp. 633–638. ISSN: 1476-4687. DOI: 10 . 1038 / s41586 - 025 - 09422 - z. URL: [http : //dx.doi.org/10.1038/s41586-025-09422-z](http://dx.doi.org/10.1038/s41586-025-09422-z).
- [41] Wentao Guo et al. SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations. 2025. arXiv: 2512. 14080 [cs.LG]. URL: <https://arxiv.org/abs/2512.14080>.
- [42] Harvey LAB: Legal Agent Benchmark. Harvey. 2026. URL: <https://www.harvey.ai/blog/introducingharveys-legal-agent-benchmark>.
- [43] Kaiming He et al. Deep Residual Learning for Image Recognition. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
- [44] Hermes Agent. Nous Research. 2026. URL: <https://hermes-agent.nousresearch.com/docs/>.
- [45] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. arXiv: 2203 . 15556 [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- [46] Shengding Hu, Yuge Tu, Xu Han, et al. “MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies”. In: (2024). arXiv: 2404.06395 [cs.CL].
- [47] Ailin Huang, Ang Li, Aobo Kong, et al. “Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters”. In: arXiv preprint arXiv:2602.10604 (2026).
- [48] Yanping Huang et al. “Gpipe: Efficient training of giant neural networks using pipeline parallelism”. In: Advances in neural information processing systems 32 (2019).

- [49] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018, pp. 2704–2713.
- [50] Sam Ade Jacobs et al. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. 2023. arXiv: 2309.14509 [cs.LG]. URL: <https://arxiv.org/abs/2309.14509>.
- [51] Peijie Jiang et al. PowLU: An Activation Function for Stable Pre-Training of LLMs. 2026. arXiv: 2605.25704 [cs.CL]. URL: <https://arxiv.org/abs/2605.25704>.
- [52] JobBench: Aligning Agent Work with Human Will. July 24, 2026. URL: <https://job-bench.github.io/>.
- [53] Keller Jordan et al. Muon: An Optimizer for Hidden Layers in Neural Networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.
- [54] Jared Kaplan et al. Scaling Laws for Neural Language Models. 2020. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.
- [55] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: Proceedings of ICML. Ed. by Hal Daumé III and Aarti Singh. PMLR, 2020, pp. 5156–5165. URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [56] Kimi CLI. Moonshot AI. 2026. URL: <https://www.kimi.com/code>.
- [57] Kimi Team. Attention Residuals. Preprint. 2026.
- [58] Kimi Team. Kimi K2: Open Agentic Intelligence. 2025. arXiv: 2507.20534 [cs.LG].
- [59] Kimi Team. “Kimi K2.5: Visual Agentic Intelligence”. In: arXiv preprint arXiv:2602.02276 (2026).
- [60] Kimi Team. Kimi K3: Open Frontier Intelligence. Moonshot AI. July 16, 2026. URL: <https://www.kimi.com/blog/kimi-k3>.
- [61] Kimi Team. “Kimi-vl technical report”. In: arXiv preprint arXiv:2504.07491 (2025).
- [62] Kimi Team. PerceptionBench: Evaluating Atomic Visual Perception in Multimodal Large Language Models. Moonshot AI. 2026. URL: <https://www.kimi.com/blog/perception-bench>.
- [63] Kimi Team et al. Kimi Linear: An Expressive, Efficient Attention Architecture. 2025. arXiv: 2510.26692 [cs.CL].
- [64] Chris Lattner et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation”. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). 2021, pp. 2–14. DOI: 10.1109/CGO51591.2021.9370308.
- [65] Legal Research Bench. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/legal_research.
- [66] Dmitry Lepikhin et al. “Gshard: Scaling giant models with conditional computation and automatic sharding”. In: arXiv preprint arXiv:2006.16668 (2020).

- [67] Mike Lewis et al. “BASE Layers: Simplifying Training of Large, Sparse Models”. In: Proceedings of ICML. 2021.
- [68] Huiba Li et al. “DADI: Block-Level Image Service for Agile and Elastic Application Deployment”. In: 2020 USENIX Annual Technical Conference (USENIX ATC). 2020, pp. 727–740.
- [69] Junlong Li et al. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. ICLR 2026. 2025. arXiv: 2510.25726 [cs.CL].
- [70] Yuetai Li et al. “JobBench: Aligning Agent Work With Human Will”. In: (2026). arXiv: 2605.26329 [cs.AI].
- [71] Yuhui Li et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. 2025. arXiv: 2503.01840 [cs.CL]. URL: <https://arxiv.org/abs/2503.01840>.
- [72] Hao Liu, Matei Zaharia, and Pieter Abbeel. “Ring Attention with Blockwise Transformers for Near-Infinite Context”. In: (2023). arXiv: 2310.01889 [cs.CL]. URL: <https://arxiv.org/abs/2310.01889>.
- [73] Jingyuan Liu et al. Muon is Scalable for LLM Training. 2025. arXiv: 2502.16982 [cs.LG]. URL: <https://arxiv.org/abs/2502.16982>.
- [74] LMArena Leaderboard. LMArena. 2026. URL: <https://lmarena.ai/leaderboard>.
- [75] Kevin Lu and Thinking Machines Lab. On-policy distillation. Thinking Machines Lab: Connectionism. 2025. URL: <https://thinkingmachines.ai/blog/on-policy-distillation/>.
- [76] Bohan Lyu et al. “MLS-Bench: A Holistic and Rigorous Assessment of AI Systems on Building Better AI”. In: (2026). arXiv: 2605.08678 [cs.LG].
- [77] Eric Martin and Chris Cundy. “Parallelizing Linear Recurrent Neural Nets Over Sequence Length”. In: Proceedings of ICLR. 2018. URL: <https://openreview.net/forum?id=HyUNwulC->.
- [78] Mike A Merrill et al. “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces”. In: arXiv preprint arXiv:2601.11868 (2026).
- [79] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. 2018. arXiv: 1805.02867 [cs.PF]. URL: <https://arxiv.org/abs/1805.02867>.
- [80] Nangate, Inc. Nangate 45nm Open Cell Library. <https://si2.org/open-cell-library/>. Version PDKv1_3_v2010_12. Donated to and distributed by the Silicon Integration Initiative (Si2). 2010. (Visited on 07/27/2026).
- [81] Deepak Narayanan et al. “Efficient large-scale language model training on gpu clusters using megatron-lm”. In: Proceedings of the international conference for high performance computing, networking, storage and analysis. 2021, pp. 1–15.
- [82] NCCL: The NVIDIA Collective Communications Library. NVIDIA. 2026. URL: <https://developer.nvidia.com/nccl>.

- [83] OpenAI. Introducing OpenAI o3 and o4-mini. Apr. 2025. URL: <https://openai.com/index/introducingo3-and-o4-mini/>.
- [84] OpenAI. Learning to Reason with LLMs. <https://openai.com/index/learning-to-reason-withllms/>. Accessed: 2026-07-23. 2024.
- [85] OpenAI Harmony Response Format. OpenAI. 2025. URL: <https://github.com/openai/harmony>.
- [86] OpenClaw. OpenClaw. 2026. URL: <https://docs.openclaw.ai/>.
- [87] Krista Opsahl-Ong et al. “OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning”. In: (2026). arXiv: 2603.08655.
- [88] Linke Ouyang et al. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. 2025. arXiv: 2412.07626 [cs.CV]. URL: <https://arxiv.org/abs/2412.07626>.
- [89] Adam Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. arXiv: 1912.01703 [cs.LG].
- [90] Tejal Patwardhan et al. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks. 2025. arXiv: 2510.04374 [cs.LG]. URL: <https://arxiv.org/abs/2510.04374>.
- [91] Bo Peng et al. RWKV-7 “Goose” with Expressive Dynamic State Evolution. 2025. arXiv: 2503.14456 [cs.CL].
- [92] Bowen Peng et al. “Yarn: Efficient context window extension of large language models”. In: arXiv preprint arXiv:2309.00071 (2023).
- [93] Long Phan et al. Humanity’s Last Exam. 2025. arXiv: 2501.14249 [cs.LG]. URL: <https://arxiv.org/abs/2501.14249>.
- [94] PostTrainBench. 2026. URL: <https://posttrainbench.com/>.
- [95] ProgramBench. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/programbench>.
- [96] Ruoyu Qin et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. 2024. arXiv: 2407.00079 [cs.DC].
- [97] Zhen Qin et al. HGRN2: Gated Linear RNNs with State Expansion. 2024. arXiv: 2404.07904 [cs.CL]. URL: <https://arxiv.org/abs/2404.07904>.
- [98] Haiquan Qiu and Quanming Yao. “Why Low-Precision Transformer Training Fails: An Analysis on Flash Attention”. In: International Conference on Learning Representations (ICLR). 2026. arXiv: 2510.04212 [cs.LG].
- [99] Zihan Qiu et al. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. 2025. arXiv: 2505.06708 [cs.CL].

- [100] Samyam Rajbhandari et al. “Zero: Memory optimizations toward training trillion parameter models”. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE. 2020, pp. 1–16.
- [101] David Rein et al. “Gpqa: A graduate-level google-proof q&a benchmark”. In: First Conference on Language Modeling. 2024.
- [102] Jonathan Roberts et al. ZeroBench: An Impossible Visual Benchmark for Contemporary Large Multi-modal Models. 2025. arXiv: 2502.09696 [cs.CV]. URL: <https://arxiv.org/abs/2502.09696>.
- [103] Bitu Darvish Rouhani et al. “Microscaling Data Formats for Deep Learning”. In: arXiv preprint arXiv:2310.10537 (2023). arXiv: 2310.10537 [cs.LG].
- [104] Alexander Samarin et al. LK Losses: Direct Acceptance Rate Optimization for Speculative Decoding. 2026 arXiv: 2602.23881 [cs.LG]. URL: <https://arxiv.org/abs/2602.23881>.
- [105] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. “Linear Transformers Are Secretly Fast Weight Programmers”. In: Proceedings of ICML. Ed. by Marina Meila and Tong Zhang. PMLR, 2021, pp. 9355–9366. URL: <https://proceedings.mlr.press/v139/schlag21a.html>.
- [106] Manasi Sharma et al. “ResearchRubrics: A Benchmark of Prompts and Rubrics For Evaluating Deep Research Agents”. In: The Fourteenth International Conference on Learning Representations. 2026. URL: <https://openreview.net/forum?id=ErnvfmSX0P>.
- [107] Noam Shazeer. GLU Variants Improve Transformer. 2020. arXiv: 2002.05202 [cs.LG]. URL: <https://arxiv.org/abs/2002.05202>.
- [108] Daniel Shepard and Robin Salimans. “AutomationBench”. In: (2026). arXiv: 2604.18934 [cs.AI].
- [109] Kean Shi et al. SaaS-Bench: Can Computer-Use Agents Leverage Real-World SaaS to Solve Professional Workflows? 2026. arXiv: 2605.15777 [cs.AI]. URL: <https://arxiv.org/abs/2605.15777>.
- [110] Benjamin F. Spector et al. “ThunderKittens: Simple, Fast, and Adorable Kernels”. In: The Thirteenth International Conference on Learning Representations. 2025. URL: <https://openreview.net/forum?id=0fJfVOSUra>.
- [111] Jianlin Su. Travels in MoE: 6. Promoting Load Balance via Optimal Assignment. Blog post (in Chinese). Feb. 2026. URL: <https://spaces.ac.cn/archives/11619>.
- [112] Hanchi Sun et al. Expert Threshold Routing for Autoregressive Language Modeling with Dynamic Computation Allocation and Load Balancing. 2026. arXiv: 2603.11535 [cs.AI].
- [113] Weigao Sun et al. “LASP-2: Rethinking Sequence Parallelism for Linear Attention and Its Hybrid”. In: (2025). arXiv: 2502.07563 [cs.LG]. URL: <https://arxiv.org/abs/2502.07563>.
- [114] Weigao Sun et al. “Linear Attention Sequence Parallelism”. In: (2024). arXiv: 2404.02882 [cs.LG]. URL: <https://arxiv.org/abs/2404.02882>.

- [115] Yiyao Sun et al. Agents’ Last Exam. 2026. arXiv: 2606.05405 [cs.AI]. URL: <https://arxiv.org/abs/2606.05405>.
- [116] Yuan Sun. Binary-Integer-Programming Based Algorithm for Expert Load Balancing in Mixture-of-Experts Models. 2025. arXiv: 2502.15451 [cs.LG].
- [117] SWE Marathon. 2026. URL: <https://www.swe-marathon.org/>.
- [118] Kimi Team. Kimi k1.5: Scaling Reinforcement Learning with LLMs. 2025. arXiv: 2501.12599 [cs.AI]. URL: <https://arxiv.org/abs/2501.12599>.
- [119] The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. July 24, 2026. URL: <https://toolathlon.xyz/introduction>.
- [120] Thinking Machines Lab. Inkling: Our Open-Weights Model. <https://thinkingmachines.ai/news/introducing-inkling/>. Accessed: 2026-07-23. July 2026.
- [121] Minyang Tian et al. SciCode: A Research Coding Benchmark Curated by Scientists. 2024. arXiv: 2407.13168 [cs.AI]. URL: <https://arxiv.org/abs/2407.13168>.
- [122] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations”. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL). 2019.
- [123] UK AI Security Institute and U.S. Center for AI Standards and Innovation. Preliminary Assessment of Kimi K3’s Cyber Capabilities. <https://www.aisi.gov.uk/blog/preliminary-assessment-of-kimi-k3cyber-capabilities>. July 2026.
- [124] Vals AI. Vals AI. 2026. URL: <https://www.vals.ai/>.
- [125] Ashish Vaswani et al. “Attention is All you Need”. In: Advances in NeurIPS. Ed. by I. Guyon et al. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [126] Nikhita Vedula et al. DeepSearchQA: Bridging the Comprehensiveness Gap for Deep Research Agents. 2025. URL: https://storage.googleapis.com/deepmind-media/DeepSearchQA/DeepSearchQA_benchmark_paper.pdf.
- [127] Bertie Vidgen et al. APEX-Agents. 2026. arXiv: 2601.14242 [cs.CL].
- [128] Ke Wang et al. “Measuring Multimodal Mathematical Reasoning with MATH-Vision Dataset”. In: Advances in NeurIPS. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 95095–95169. DOI: 10. 52202/079017-3014. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/ad0edc7d5fa1a783f063646968b7315b-Paper-Datasets_and_Benchmarks_Track.pdf.
- [129] Lei Wang et al. “TileLang: A Composable Tiled Programming Model for AI Systems”. In: arXiv preprint arXiv:2504.17577 (2025). URL: <https://arxiv.org/abs/2504.17577>.

- [130] Zirui Wang et al. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. 2024. arXiv: 2406.18521 [cs.CL]. URL: <https://arxiv.org/abs/2406.18521>.
- [131] Jason Wei et al. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. 2025. arXiv: 2504.12516 [cs.CL]. URL: <https://arxiv.org/abs/2504.12516>.
- [132] Xinming Wei et al. UltraEP: Unleash MoE Training and Inference on Rack-Scale Nodes with Near-Optimal Load Balancing. 2026. arXiv: 2606.04101 [cs.DC]. URL: <https://arxiv.org/abs/2606.04101>.
- [133] Zijian Wu et al. “MCPMark: A benchmark for stress-testing realistic and comprehensive mcp use”. In: arXiv preprint arXiv:2509.24002 (2025).
- [134] B. Xiao et al. “MiMo-V2-Flash Technical Report”. In: arXiv preprint arXiv:2601.02780 (2026).
- [135] Xiaomi MiMo Team. MiMo-V2.5-Pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>. 2026.
- [136] Tianbao Xie et al. “Introducing OSWorld-Verified”. In: xlang.ai (July 2025). URL: <https://xlang.ai/blog/osworld-verified>.
- [137] Zijie Yan et al. Scalable Training of Mixture-of-Experts Models with Megatron Core. 2026. arXiv: 2603.07685 [cs.DC]. URL: <https://arxiv.org/abs/2603.07685>.
- [138] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: Proceedings of ICLR. 2025. URL: <https://openreview.net/forum?id=r8H7xhYPwz>.
- [139] Songlin Yang and Yu Zhang. FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism. Jan. 2024. URL: <https://github.com/fla-org/flash-linear-attention>.
- [140] Songlin Yang et al. “Gated Linear Attention Transformers with Hardware-Efficient Training”. In: Proceedings of ICML. PMLR, 2024.
- [141] Songlin Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: Proceedings of NeurIPS. 2024.
- [142] Yaoyu Wang. Context Parallelism for DeltaNet. 2025. URL: <https://yywangcs.notion.site/DeltaNet-2a9fc9f5d8058013a498f34e0b25bd52>.
- [143] Mengqi Yuan et al. OSWorld2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. 2026. arXiv: 2606.29537 [cs.AI]. URL: <https://arxiv.org/abs/2606.29537>.
- [144] Xiang Yue et al. MMMU-Pro: A More Robust Multi-discipline Multimodal Understanding Benchmark. 2024. arXiv: 2409.02813 [cs.CL]. URL: <https://arxiv.org/abs/2409.02813>.
- [145] Aohan Zeng et al. GLM-5: from Vibe Coding to Agentic Engineering. 2026. arXiv: 2602.15763 [cs.LG]. URL: <https://arxiv.org/abs/2602.15763>.

- [146] Biao Zhang and Rico Sennrich. “Root mean square layer normalization”. In: Advances in NeurIPS 32 (2019).
- [147] Chenggang Zhao et al. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. 2025.
- [148] Yilun Zhao et al. “MMVU: Measuring Expert-Level Multi-Discipline Video Understanding”. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). June 2025, pp. 8475–8489.
- [149] Runjie Zhou et al. WorldVQA: Measuring Atomic World Knowledge in Multimodal Large Language Models. 2026. arXiv: 2602.02537 [cs.CV]. URL: <https://arxiv.org/abs/2602.02537>.
- [150] Jian Zhu et al. “SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows”. In: (2026). arXiv: 2606.29955 [cs.SE].

9 贡献者名单

贡献者名单按姓氏字母顺序排列。

Tongtong Bai	Fuxuan Gao	Aidi Li	Shaoguang Mao
Yifan Bai	Hongcheng Gao	Cheng Li	Yuan Mei
Yiping Bao	Jingyue Gao	Chengyuan Li	Xin Men
M. C.	Tong Gao	Cong Li	Minqing Ni
Jianfeng Cai	WeiJia Gao	Fang Li	Yixuan Niu
Xinyuan Cai	Shangyi Geng	Guanyu Li	Siyuan Pan
Peizhou Cao	Jie Gong	Haoyang Li	Shujun Peng
Yuxuan Cao	Linhua Gong	Jia Li	Zhangyang Qi
Ziwei Chai	Shengao Gong	Junxiong Li	Ruoyu Qin
Y. Charles	Xiaochen Gong	Lei Li	ZeChao Qin
H.S. Che	Qizheng Gu	Letian Li	Zeyu Qin
Guanduo Chen	Yicheng Gu	Lincan Li	Haiquan Qiu
Guangyu Chen	Shuhao Guan	Weihong Li	Jianxin Qiu
Guanzheng Chen	Haiqing Guo	Wentao Li	Jiezhong Qiu
Huarong Chen	Shiqi Guo	Xintong Li	Bowen Qu
Jia Chen	Xiang Guo	Yang Li	Yuhao Qu
Jianlong Chen	Zhengyan Guo	Yishen Li	Zeyu Shang
Jun Chen	Beixi Hao	Yiwei Li	Youbo Shao
Kexin Chen	Wenxin Hao	Yuxiao Li	Han Shen
Peng Chen	Xiaoru Hao	Zhaowei Li	Jincheng Shi
Ruijue Chen	Dailan He	Zhaoxi Li	Juanfeng Shi
Wentao Chen	Haotian He	Zheming Li	Lidong Shi

Xin Chen	Lehan He	Zhengxiao Li	Shengyuan Shi
Yang Chen	Qi He	Zhiyuan Li	Wingchun Siu
Yanru Chen	Weiran He	Jiawei Lin	Pengwei Song
Yifei Chen	Xinran He	Xiaohan Lin	Xiaoxi Song
Yingjiang Chen	Xinyi He	Yibo Lin	Jianlin Su
Yuankun Chen	Yibo He	Zichao Lin	Yunfeng Su
Yujie Chen	Yunjia He	Ziyan Lin	Zhaochen Su
Yutian Chen	Chao Hong	Bill Liu	Lin Sui
Zhirong Chen	Tiange Hong	Boxiao Liu	Jingsong Sun
Dazhi Cheng	Hao Hu	Chuan Liu	Junyao Sun
Yean Cheng	Jiaxi Hu	Liang Liu	Shaoning Sun
Jialei Cui	Ruikun Hu	Shaowei Liu	Shuzhe Sun
Jingbing Cui	Weiming Hu	Shudong Liu	Tongyu Sun
Anqi Dai	Yangyang Hu	Shuran Liu	Yujun Sun
Jiaqi Deng	Zhenxing Hu	Tianwei Liu	Yunpeng Tai
Hao Ding	Liang Hua	Weizhou Liu	Chuning Tang
Rui Ding	Jinbin Huang	Yangyang Liu	Heyi Tang
Shaofeng Ding	Ke Huang	Yanming Liu	Sirui Tang
Mengfan Dong	Ruiyuan Huang	Yibo Liu	Zecheng Tang
Mengnan Dong	Siyang Huang	Yipeng Liu	Chaoran Tian
Yuhao Dong	Weixiao Huang	Zhengying Liu	Rongpeng Tian
Yuxin Dong	Yan Huang	Zhiheng Liu	Yu Tian
Ang'ang Du	Zhengjie Huang	Enzhe Lu	Wei Tu
Chenzhuang Du	Zhiqi Huang	Haoyu Lu	Chensi Wang
Dikang Du	Chaobo Jia	Linqiang Lu	Chuang Wang
Jusen Du	Yutong Jiang	Tingzhan Lu	Chunjie Wang
Yulun Du	Zhejun Jiang	Zhiyuan Lu	Dinglu Wang
Yu Fan	Zuoyou Jiang	Aotian Luo	Feng Wang
Jing Feng	Wenyi Jin	G. Luo	Hailong Wang
Qiulin Feng	Xinyi Jin	Junyu Luo	Haiming Wang
Yichen Feng	Yu Jing	Yifan Luo	Hao Wang
Kelin Fu	Huanjun Kong	B. Lyu	Hao Wang
Qiang Fu	Guokun Lai	Wenzhou Lyu	Huaqing Wang

Hui Wang Jiayi Wang Jinglong Wang Jinhong Wang Jiuzheng Wang Linian Wang Shaobo Wang Shenzhi Wang Shuyi Wang Si Wang Siyuan Wang Tianfu Wang Wenjue Wang Xingran Wang Xinmei Wang Xinyuan Wang Xusheng Wang Yalin Wang Yangkun Wang Yao Wang Yaoyu Wang Yejie Wang Yiqin Wang Yucheng Wang Yuzhi Wang Zhaoji Wang Zhaowei Wang Zhengtao Wang Zhenhao Wang Zhongsheng Wang Zifan

Wang Chu Wei Ming Wei Shouxin Wei Zichen Wen Fan Wu Haoning Wu Rucong Wu Wenhao Wu Xiaoxue
Wu Yingcong Wu Yongqi Wu Yuxin Wu Zijian Wu Xinglang Xian Chenxuan Xiang

Yuye Xiang Bocheng Xiao Chenjun Xiao Xin Xiao Jin Xie Xiaotong Xie Yifeng Xie Zhe Xie Bowei Xing
Yiming Xiong Baosheng Xu Boyu Xu Jiale Xu Jianfan Xu Jing Xu Jinjing Xu L.H. Xu Qingtao Xu Shuyao
Xu Suting Xu Tiantian Xu Tianxiang Xu Weixin Xu Xinran Xu Yangchuan Xu Ye Xu Yueni Xu Ziyao Xu
Haonan Xue Junjie Yan Yaoyao Yan Fan Yang Guangyao Yang Hao Yang Junwei Yang Ruoyu Yang Wenjie
Yang Xiaofei Yang Xinyu Yang Yi Yang Yiling Yang Ying Yang Yuchen Yang Zhen Yang Zhilin Yang Zian
Yang

Zuhao Yang Haotian Yao Dan Ye Haoran Ye Wenjie Ye Zhanbo Ye Bohong Yin Haoxiang Yin Xietong Yin
Chengzhen Yu Haozhen Yu Longhui Yu Shengnan Yu Shuying Yu Tianxiang Yu Enming Yuan Mengjie Yuan
Tongtian Yue Wei Yue Yang Yue Dunyuan Zha Haobing Zhan B.H. Zhang Dehao Zhang Fei Zhang Hao Zhang
Haoyuan Zhang Huanyu Zhang Jiawei Zhang Jiapei Zhang Jiaxuan Zhang Jin Zhang Kaiyi Zhang Miaozhen Zhang Puqi
Zhang Qinglei Zhang Rong Zhang Rui Zhang Shaoshuai Zhang Shiyi Zhang Xiaobin Zhang Xiaoyun Zhang
Y. Zhang Yangkun Zhang Ye Zhang Yichi Zhang Yikun Zhang

Yizhi Zhang Yongting Zhang Yu Zhang Yutao Zhang Yutong Zhang Zheng Zhang Zijing Zhang Bin Zhao
Chenguang Zhao Feifan Zhao Jinglun Zhao Jinxiang Zhao Shuai Zhao Wenshuo Zhao Xiangyu Zhao Xuanle
Zhao Yikai Zhao Zijia Zhao Haozhi Zheng Huabin Zheng Ruihan Zheng Shaojie Zheng Tengyang Zheng
Haofeng Zhong Lei Zhong Longguang Zhong M. Zhou Qianqiang Zhou Runjie Zhou Ruozhang Zhou Xinyu
Zhou Yiqiao Zhou Zaida Zhou Jinguo Zhu Liya Zhu Xinhao Zhu Yangjunfeng Zhu Yuxuan Zhu Zhen Zhu
Chen Zhuang Weiyu Zhuang Xinxing Zu Kimi K3

10 Sigmoid Tanh Unit GLU 细节

SiTU-GLU (§2.3.2) 的设计目标是在不丢弃 Swish 特征形状的前提下, 对 SwiGLU 乘积进行有界约束: 即在原点附近近似线性响应, 且负尾部趋于零。图 4 展示了门控分支和上投影分支及其完整的标量响应。

平滑约束两个分支 SiTU 将 Swish 的线性因子限制为 $\beta_1 \tanh(\mathbf{W}_g \mathbf{x} / \beta_1)$, 同时保留 sigmoid 因子 [60]。由于 sigmoid 已经将负门控响应推向零, 此改动主要控制大的正激活值而不会消除负尾部。Kimi K3 对上投影分支采用相同的构造, 即 $\beta_2 \tanh(\mathbf{W}_u \mathbf{x} / \beta_2)$, 防止任一分支主导乘积。

局部与极限行为对于原点附近的标量 z , 缩放后的 $\tanh$ 满足

$$\beta \tanh\left(\frac{z}{\beta}\right) = z + O\left(\frac{z^3}{\beta^2}\right). \quad (18)$$

因此, SiTU-GLU 在原点附近与 SwiGLU 一阶匹配。当 $\beta_1, \beta_2 \rightarrow \infty$ 时, 它也逐点恢复为 SwiGLU。

有界输出由于 $|\tanh(z)| < 1$ 且 $0 < \text{Sigmoid}(z) < 1$, 每个输出坐标满足

$$\|\text{SiTU} - \text{GLU}(\mathbf{x})\|_\infty \leq \beta_1 \beta_2 = 100, \quad (19)$$

其中 $\beta_1 = 4$, $\beta_2 = 25$ 。与对门控预激活进行硬截断不同, 平滑约束在远离饱和边界处保留了非零梯度, 我们发现这能带来更好的训练行为。

11 分位数均衡的推导

本附录从最优均衡分配出发, 推导 §2.3 中使用的分位数均衡 (Quantile Balancing, QB) 更新, 推导遵循 [111]; 关于专家负载均衡的分配视角可追溯到 BASE Layers [67] 和 BIP [116]。令 $\mathbf{s} \in \mathbb{R}^{m \times n}$ 收集 m 个 token 在 n 个专家上的路由分数, 其中每个 token 恰好选择 k 个专家, $x_{i,j} \in \{0, 1\}$ 指示 token i 是否分配给专家 j 。最大分数均衡分配——即每个专家恰好服务 mk/n 个 token (假设为整数)——为

$$\max_{x_{i,j} \in \{0,1\}} \sum_{i,j} x_{i,j} s_{i,j} \quad \text{s.t.} \quad \sum_j x_{i,j} = k, \quad \sum_i x_{i,j} = \frac{mk}{n}. \quad (20)$$

线性松弛与对偶将 $x_{i,j} \in \{0, 1\}$ 松弛为 $x_{i,j} \in [0, 1]$ 将式 20 转化为线性规划, 其最优解根据二部图 b-匹配多面体的标准整性是整数的; 因此该松弛是精确的。引入自由乘子 α_i 和 β_j 分别对应 token 侧和专家侧的等式约束, 松弛问题可写成 max-min 形式:

$$\max_{x_{i,j} \in [0,1]} \min_{\alpha_i, \beta_j} \sum_{i,j} x_{i,j} s_{i,j} - \sum_i \alpha_i \left(\sum_j x_{i,j} - k \right) - \sum_j \beta_j \left(\sum_i x_{i,j} - \frac{mk}{n} \right). \quad (21)$$

目标函数关于 x, α , 和 β 都是线性的, 且可行集是凸的, 因此 minimax 定理允许交换优化顺序:

$$\min_{\alpha_i, \beta_j} \max_{x_{i,j} \in [0,1]} \sum_{i,j} x_{i,j} (s_{i,j} - \alpha_i - \beta_j) + k \sum_i \alpha_i + \frac{mk}{n} \sum_j \beta_j. \quad (22)$$

内层最大值对每个元素可分, 其中若 $s_{i,j} - \alpha_i - \beta_j > 0$ 则 $x_{i,j}^* = 1$, 若 $s_{i,j} - \alpha_i - \beta_j < 0$ 则 $x_{i,j}^* = 0$; 在实际中平局情况测度为零。代入 x^* 得到凸的对偶目标:

$$\min_{\alpha_i, \beta_j} \mathcal{L}(\alpha, \beta) := \sum_{i,j} \max(0, s_{i,j} - \alpha_i - \beta_j) + k \sum_i \alpha_i + \frac{mk}{n} \sum_j \beta_j. \quad (23)$$

算法 1: 交替 QB 求解器。

输入: 分数矩阵 $s \in \mathbb{R}^{m \times n}$ 输出: 分配 $x \in \{0, 1\}^{m \times n}$ 1 初始化 $\beta = 0_{1 \times n}$;

2 for $t = 1, 2, \dots, T$ do

3 $\alpha \leftarrow \text{desc_sort}(s - \beta, \text{axis} = 1)_{[:,k:k+1]}$ 4 $\beta \leftarrow \text{desc_sort}(s - \alpha, \text{axis} = 0)_{[mk/n:mk/n+1]}$ 5 end

6 返回 x , 其中若 $j \in_k (s_i - \beta)$ 则 $x_{i,j} = 1$, 否则为 0

精确坐标最小化我们通过交替求解固定 β 时的 α 和固定 α 时的 β 来最小化式 23；每个子问题都有闭式精确解。固定 β 时，问题在 token 间解耦，对 token i 求解：

$$\min_{\alpha} k\alpha + \sum_j \max(0, s_{i,j} - \beta_j - \alpha). \quad (24)$$

该目标关于 α 是分段线性的，斜率为 k 减去超过 α 的 margin $s_{i,j} - \beta_j$ 的数量；因此当恰好有 k 个 margin 在 α 之上时目标被精确最小化，即对任意位于 $s_i - \beta$ 的第 k 大和第 $(k+1)$ 大元素之间的 α_i^* 。按惯例我们取第 $(k+1)$ 大元素，等价地即 $(1 - k/n)$ 分位数：

$$\alpha_i^* = \text{quantile}_{1-k/n}(s_i - \beta). \quad (25)$$

对称地，固定 α 时，专家 j 求解 $\min_{\beta} \frac{mk}{n}\beta + \sum_i \max(0, s_{i,j} - \alpha_i - \beta)$ ，其极小值为 $s_{:,j} - \alpha$ 的第 $(mk/n + 1)$ 大元素，同样是 $(1 - k/n)$ 分位数：

$$\beta_j^* = \text{quantile}_{1-k/n}(s_{:,j} - \alpha). \quad (26)$$

两个更新因此分别是沿 token 轴和专家轴的相同分位数，此即该方法命名的由来。图 5 将专家侧更新展示为使每个专家 margin 分布的被接受上尾均衡化，算法 1 总结了由此得到的交替求解器。

从分配到路由在式 23 的最优解处， $x_{i,j}^* = 1$ 当且仅当 $s_{i,j} - \alpha_i^* - \beta_j^* > 0$ ；结合 token 约束 $\sum_j x_{i,j}^* = k$ ，被选中的专家恰好是 $s_i - \beta^*$ 的 Top- k 项。因此路由只需专家阈值 $\beta \in \mathbb{R}^n$ （等价地，式 13 中的偏置 $\mathbf{b} = -\beta$ ），而 token 阈值 $\alpha \in \mathbb{R}^m$ 是与动态训练批次绑定的中间变量，将被丢弃。这种不对称性保持了训练-推理一致性：在部署时，路由是带冻结偏置的固定 Top- k 选择，无需进行分位数计算。

与基于符号的无损失更新的关系式 26 背后的专家侧子问题具有（次）梯度

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \frac{mk}{n} - \sum_{i=1}^m \chi(s_{i,j} - \alpha_i - \beta_j > 0), \quad (27)$$

即目标负载减去专家 j 的实际负载。在该目标上进行 SignSGD 步骤即可恢复辅助无损失均衡 [30] 的固定步长符号更新，除了符号约定 $\mathbf{b} = -\beta$ ：符号更新只保留式 27 中负载误差的方向，而 QB 直接跳转到同一对偶目标的精确坐标极小点。这一视角既解释了为何 QB 无需类似学习率的超参数，也解释了为何它在即使接近 10^3 个专家时也能在几步更新内达到均衡。QB 同样与 BIP [116] 相关，后者以不等式约束 $\sum_j x_{i,j} \leq k$ 和 $\sum_i x_{i,j} \leq mk/n$ 求解同一分配问题；由此引入的 α 和 β 上的非负性约束为两个更新增加了 $\max(0, \cdot)$ 截断，这只能抑制被过度选择的专家而无法促进被不足选择的专家，并且在我们实验中显著减慢了均衡速度。最后，由此得到的固定 Top- k 路由与专家特定阈值路由相关，但不同于 Expert Threshold 路由——后者维护 EMA 阈值并允许每个 token 选择可变数量的专家 [112]。

12 基于直方图的分位数估计

式 14 的 QB 更新需要在整个训练步上取分位数：对 n 个专家中的每一个，取 margin $s_{i,j} - \alpha_i$ 的 $(1 - k/n)$ 分位数，其中 token 数量 m 跨越分布在数据并行 rank 和梯度累积步上的数百万 token。在训练循环内收集 $O(mn)$ 个 margin 用于精确分位数计算是不切实际的。关键观察在于，更新从不需要 margin 本身，只需要它们每个专家的分布，而直方图可以以固定成本进行概括。因此，Kimi K3 为每个专家维护一个分箱直方图并从中读取分位数。具体地，我们对所需偏置 $r_{i,j} := \alpha_i - s_{i,j}$ （即恰好将专家 j 置于 token i 的截止门槛的偏置）进行直方图统计；对 margin 取负反转了它们的顺序，因此式 14 的 QB 目标 $\hat{b}_j$ 恰好是 $r_{:,j}$ 的 (k/n) 分位数。

分箱范围第一个问题是需要哪个区间上分箱，这里所需偏置提供了帮助：其范围由当前偏置本身界定。路由分数是 sigmoid 输出，因此 $s_{i,j} \in (0, 1)$ ，而截止门槛 α_i 本身是某个专家 j' 的带偏置分数 $s_{i,j'} + b_{j'}$ ，因此它位于 $(b_{\min}, 1 + b_{\max})$ 内，其中 $b_{\min}$ 和 $b_{\max}$ 为当前偏置的极值。因此每个 $r_{i,j}$ 落在 $[b_{\min} - 1, b_{\max} + 1]$ 内。我们将该区间划分为 B 个均匀分箱，实践中这已足够，并且每步重新计算范围，因此箱宽度 $w = (b_{\max} - b_{\min} + 2)/B$ 会随着偏置在修正不均衡过程中扩散而保持适应。

累积与恢复程序的其余部分遵循训练步的结构。在每个前向传播过程中，每个 rank 将其本地的 $r_{i,j}$ 值 scatter-add 到每个专家的计数矩阵 $\mathbf{H} \in \mathbb{N}^{n \times B}$ 中，在所有微批次上累积且无需通信。在步骤结束时，一次 all-reduce 将本地计数求和为全局直方图，每个 rank 从同一汇总计数中恢复分位数。每个专家的直方图对每个 token 计数一次，因此目标秩恰好是 §2.3.3 的目标负载 $q = mk/n$ ，现在对全步骤取：我们选择累积计数首次达到 $\lceil q \rceil$ 的分箱，并在其中进行线性插值。若选中分箱 β_j ，其之前累积计数为 c_j ，箱内计数为 h_j ，则

$$\hat{b}_j = b_{\min} - 1 + \left(\beta_j + \text{clip} \left(\frac{q - c_j}{h_j}, 0, 1 \right) \right) w,$$

得到的偏置按式 14 进行均值中心化。

性质三个性质使得该估计器在大规模下切实可行。第一，精度高：累积计数在箱边界处是精确的，因此真实分位数与其估计值位于同一分箱中，误差以箱宽度 w 为界；当 $B = 1000$ 时，这最多为几 10^{-3} ，我们未观察到可测量的残余负载不均衡。第二，成本低：唯一的通信是每层每步一次 nB 个整数的 all-reduce，与 m 无关，在我们配置中低于在每微批次上跨进程组交换原始 margin 成本的 1%，后者是自然替代方案。第三，估计正确的量：因为计数是可加的，全局直方图对 token 如何跨 rank 或累积步分区是完全不变的，估计值是汇总全局批次的分位数，而不是各 rank 分位数的平均值，后者通常不同。作为进一步改进，在各步之间维护估计分位数的指数移动平均可以减少批次间的采样噪声，并可能进一步改善负载均衡。

13 MoonEP 通用上界证明

令 $m_r(P)$ 表示在方案 P 下放置在 rank r 上的冗余专家数量。对路由器输出 I ，规划目标是最小化任一 rank 上的最大冗余专家数，即 $M(I) = \min_P \max_r \{m_r(P)\}$ 。我们证明 $M(I) \leq E/R$ 总是成立（定理 1），且该上界本质上是紧的：存在路由器输出使得 $M = \lceil E(R-1)/R^2 \rceil \approx E/R$ （定理 2）。

定理 1 的证明（通用上界）目标是证明对任意路由器输出 I ， $M(I) \leq E/R$ 成立。关键引理：存在一个方案 P^* 使得每个 EP rank 接收完全相同数量的 token ($S \times K$)，且每个 rank 的远程 token 仅来自一个其他 EP rank。构造如下：初始时，每个 rank 仅持有本地 token，rank 被相应划分为欠载和过载。我们重复选取一个

欠载 rank 和一个过载 rank，将 token 从过载 rank 迁移以恰好将欠载 rank 填至均衡值 $S \times K$ ；过载 rank 可能仍然过载、变为恰好均衡或变为欠载，并被放回相应集合中。重复此过程直到所有 rank 完美均衡。每次填充使一个欠载 rank 达到均衡且此后不再变化，因此该过程在至多 $R - 1$ 次填充后终止；同时，每个 rank 至多被填充一次，因此其远程 token 来自单个 rank，从而证明了引理。由此，假设 rank r 的所有远程 token 来自 rank s ；这些 token 属于 rank s 上至多 E/R 个本地专家，因此 $m_r(P^*) \leq E/R$ ，进而

$$M(I) = \min_P \max_r \{m_r(P)\} \leq \max_r \{m_r(P^*)\} \leq \frac{E}{R} \quad (28)$$

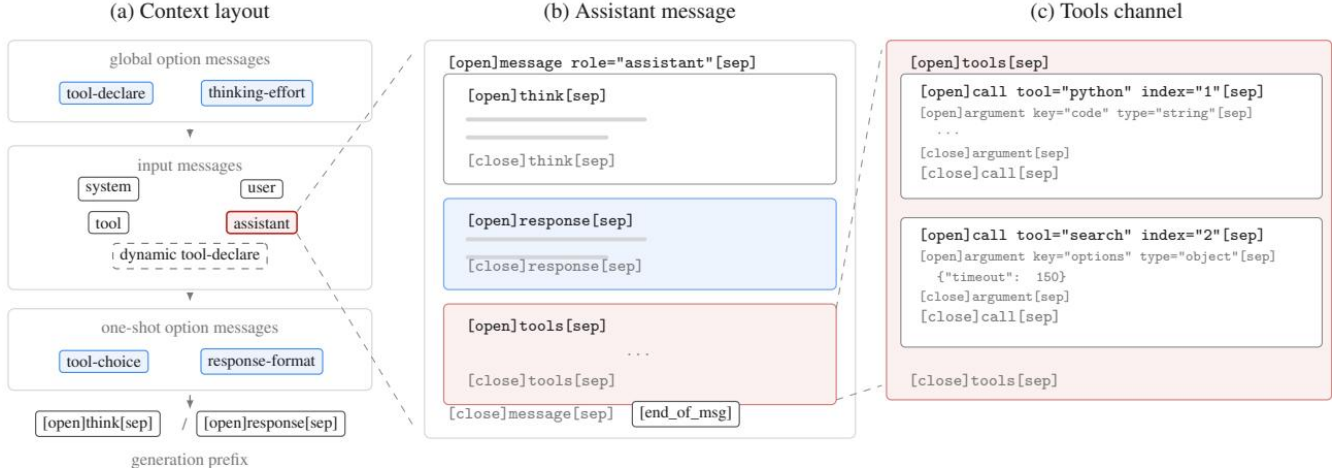


图 16: Kimi K3 聊天模板的结构。(a) 上下文布局：全局选项消息位于输入消息之前，而单次选项消息紧随其后，因此每次请求的选项不会影响历史 KV 缓存；动态加载的工具以输入选项消息的形式在会话中注入（虚线）。(b) 助手消息的解剖结构：正文组织为思考（think）、回复（response）和工具（tools）通道。(c) 工具通道的展开：并行的工具调用带有索引，以便工具结果能与其调用匹配，且参数带有类型标注。

定理 2 的证明（上界的紧性）构造路由器输出 I^* 如下：EP rank 0 上的专家不接收任何 token，而其余 $R - 1$ 个 rank 上的所有专家均匀共享所有 token。则所有 $S \times K \times R$ 个 token 均匀分配给 $E(R - 1)/R$ 个专家，因此每个专家接收 $\frac{SKR^2}{E(R-1)}$ 个 token。在任意方案 P 下，rank 0 必须接收 $S \times K$ 个 token，这些全是远程 token，且这些 token 至少涉及 $\frac{SK}{\frac{SKR^2}{E(R-1)}} = \frac{E(R-1)}{R^2}$ 个不同专家；取上取整，rank 0 需要至少 $\lceil \frac{E(R-1)}{R^2} \rceil$ 个冗余专家，因此 $M(I^*) \geq \lceil \frac{E(R-1)}{R^2} \rceil$ 。反之，通过使用定理 1 证明中的填充过程并优先按专家迁移 token 来构造方案，可以使每个 rank 上的冗余专家数保持在该值以内，因此等式成立。由于当 R 较大时 $\lceil \frac{E(R-1)}{R^2} \rceil \approx \frac{E}{R}$ ，定理 1 的上界本质上是紧的：不存在显著小于 E/R 的通用上界。

14 聊天模板

Kimi K3 的聊天模板围绕三个目标重新设计。第一是可扩展性：新能力应通过向后兼容的消息格式而非模板修订来引入，使得单一模板服务于整个模型世代。第二是低对齐代价：格式应以最少的监督数据即可学习，支持经过轻度微调的预训练模型可直接进入强化学习的流水线。第三是解码友好性：结构应容纳简单的编码器、流式解析器和语法约束执行器。为此，模板采用 XTML (eXtensible Token Markup Language, 可扩展 Token